

INTRODUCTION **MACHINE LEARNING**

Tiếp theo

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Trong Linear Regression phần trước ta được biết.
Các thông số dự đoán đều có sai số (**error**)

Giảm sai số càng nhỏ

Mất mát (loss) càng nhỏ

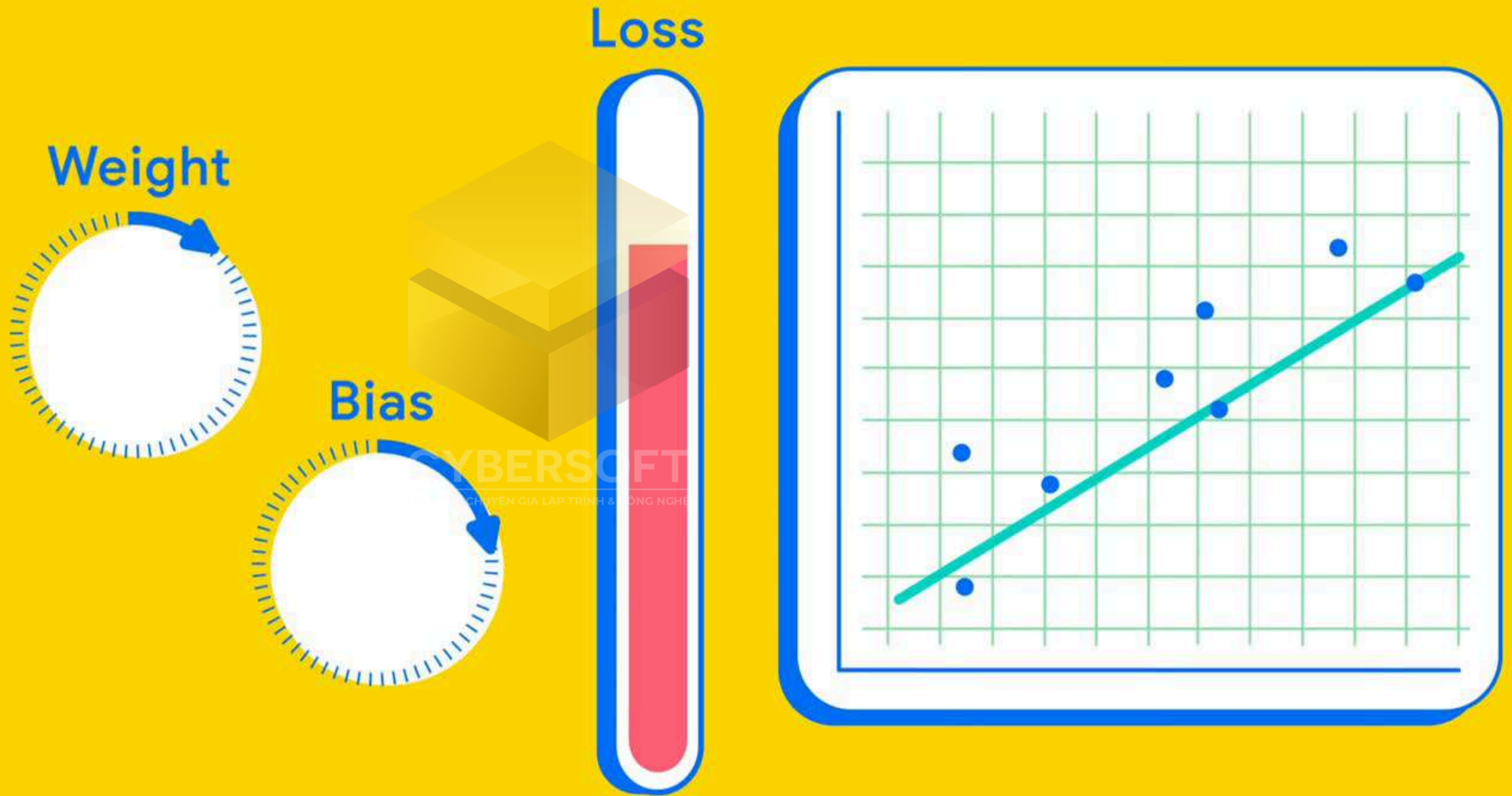
Các dự đoán (Prediction) càng chính xác



Vậy Làm sao giảm thiểu
mất mát (loss) ?



ỨNG DỤNG GRADIENT DESCENT





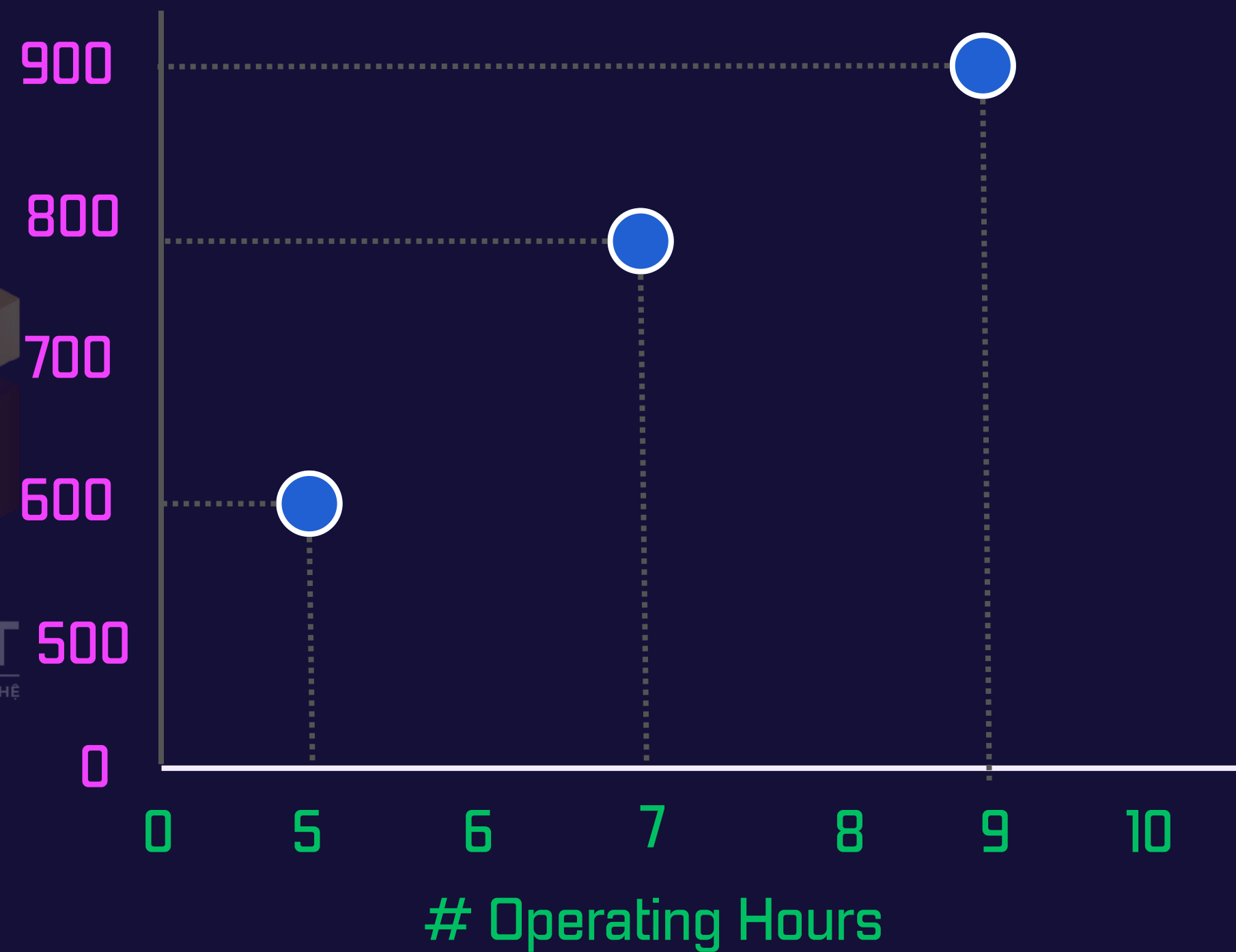
Bài toán Starbucks London

Features Label
#Operating Hours (hours) #Daily Sales (USD)

5	600
7	790
9	900



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ





Bài toán Starbucks London

Features

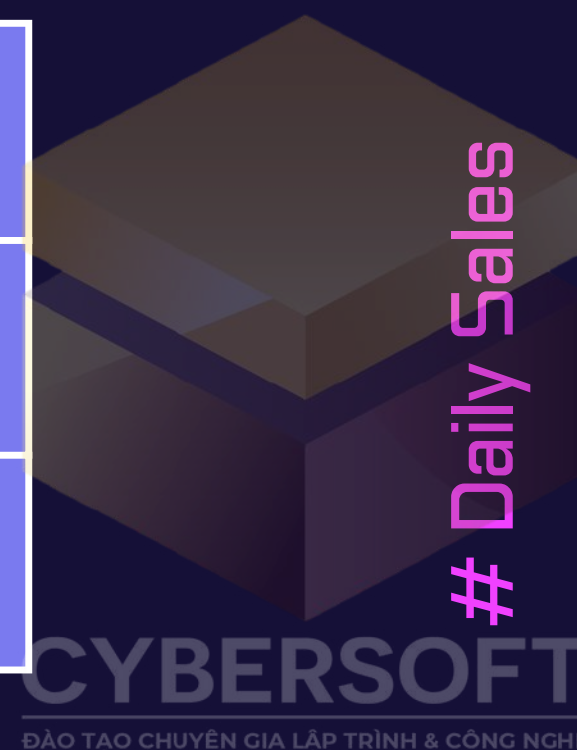
Label

Predict

#Operating Hours (hours)

#Daily Sales (USD)

5	600	0
7	790	0
9	900	0



Daily Sales

900

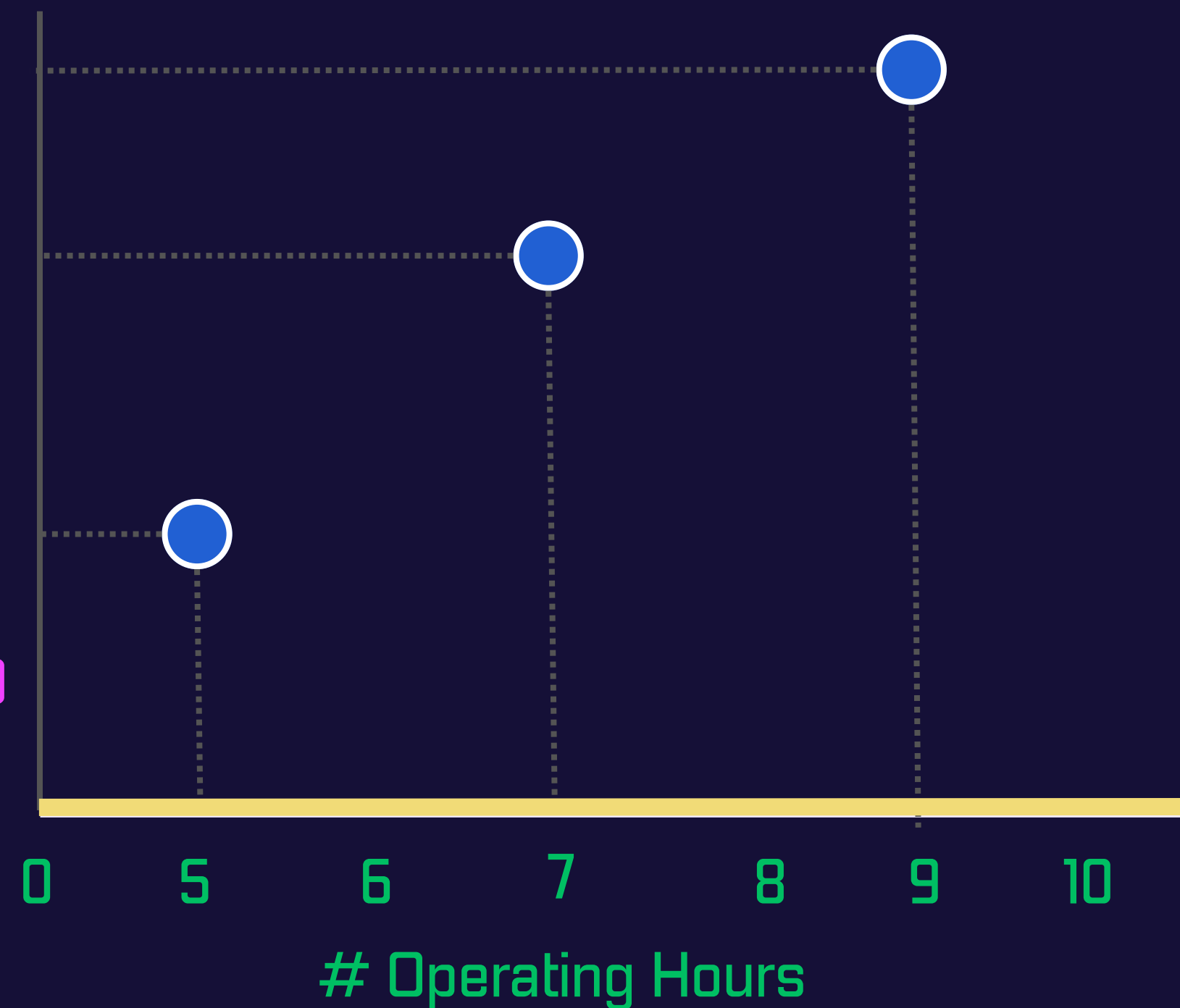
800

700

600

500

0



Linear Regression bắt đầu bằng:

$$Y = \omega_1 X + \omega_0$$

$$\text{weight } (\omega_1) = 0$$

$$\text{bias } (\omega_0) = 0$$



Bài toán Starbucks London

Features

Label

Predict

#Operating Hours (hours)

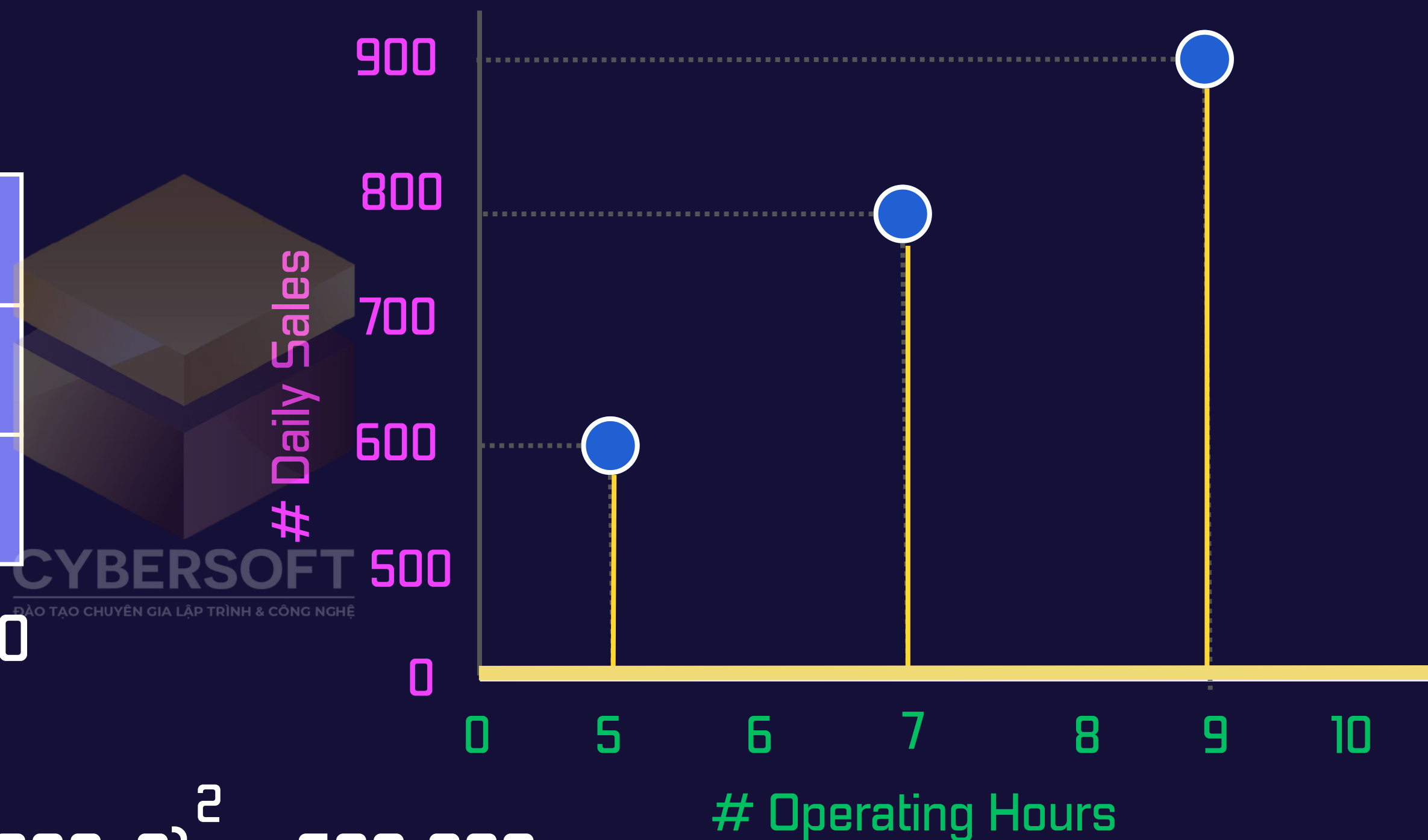
#Daily Sales (USD)

5	600	0
7	790	0
9	900	0

bias (w_1) = 0 weight (w_0) = 0

Loss

$$MSE = \frac{(600-0)^2 + (790-0)^2 + (900-0)^2}{3} = 598\,033$$





Bài toán Starbucks London

Features **Label** **Predict**
#Operating Hours (hours) #Daily Sales (USD)

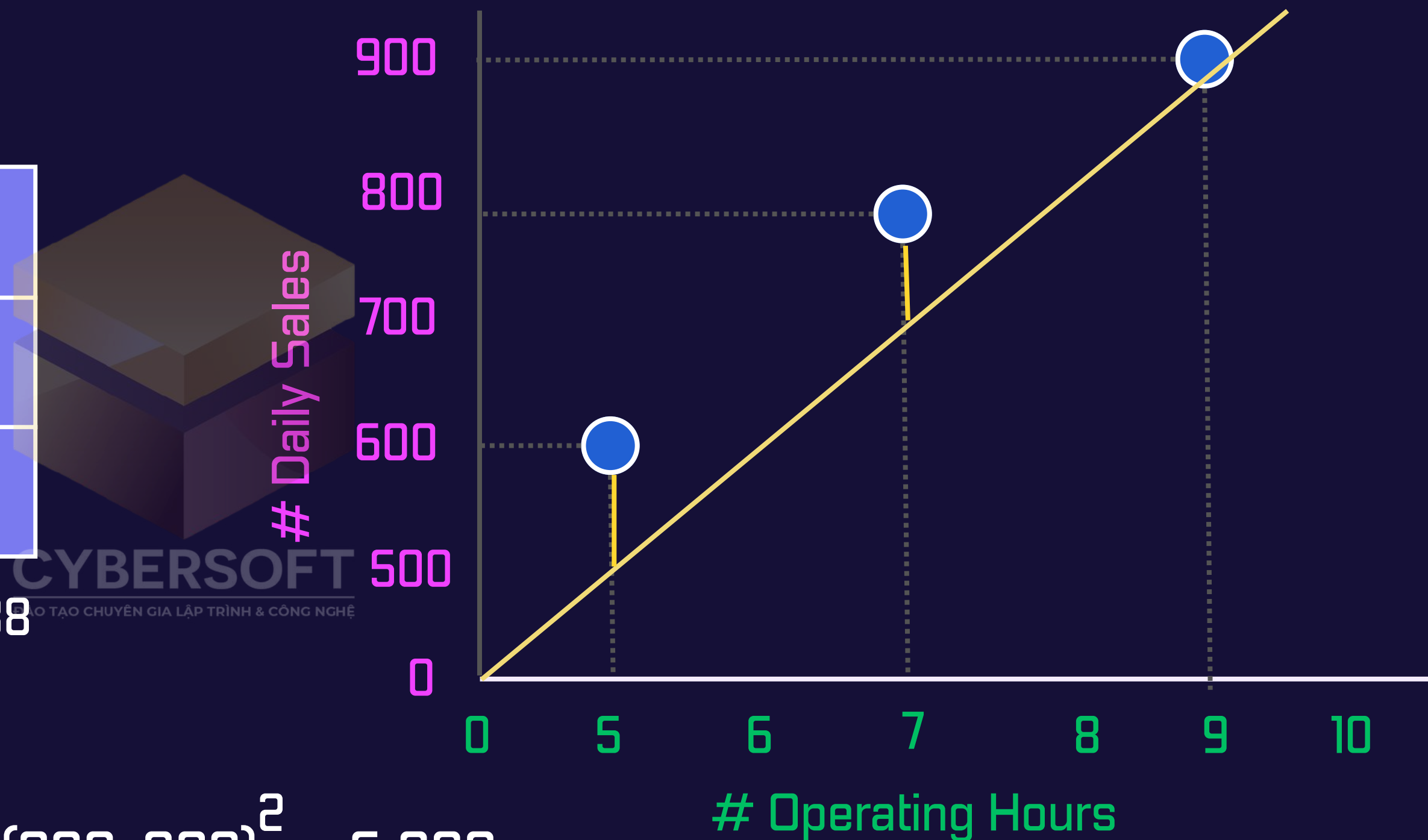
5	600	500
7	790	720
9	900	890

bias (w_1) = 75 weight (w_0) = 238

Linear Regression cân bằng

Loss

$$\text{MSE} = \frac{(600-500)^2 + (790-720)^2 + (900-890)^2}{3} = 5000$$





Bài toán Starbucks London

Features **Label** **Predict**
#Operating Hours (hours) #Daily Sales (USD)

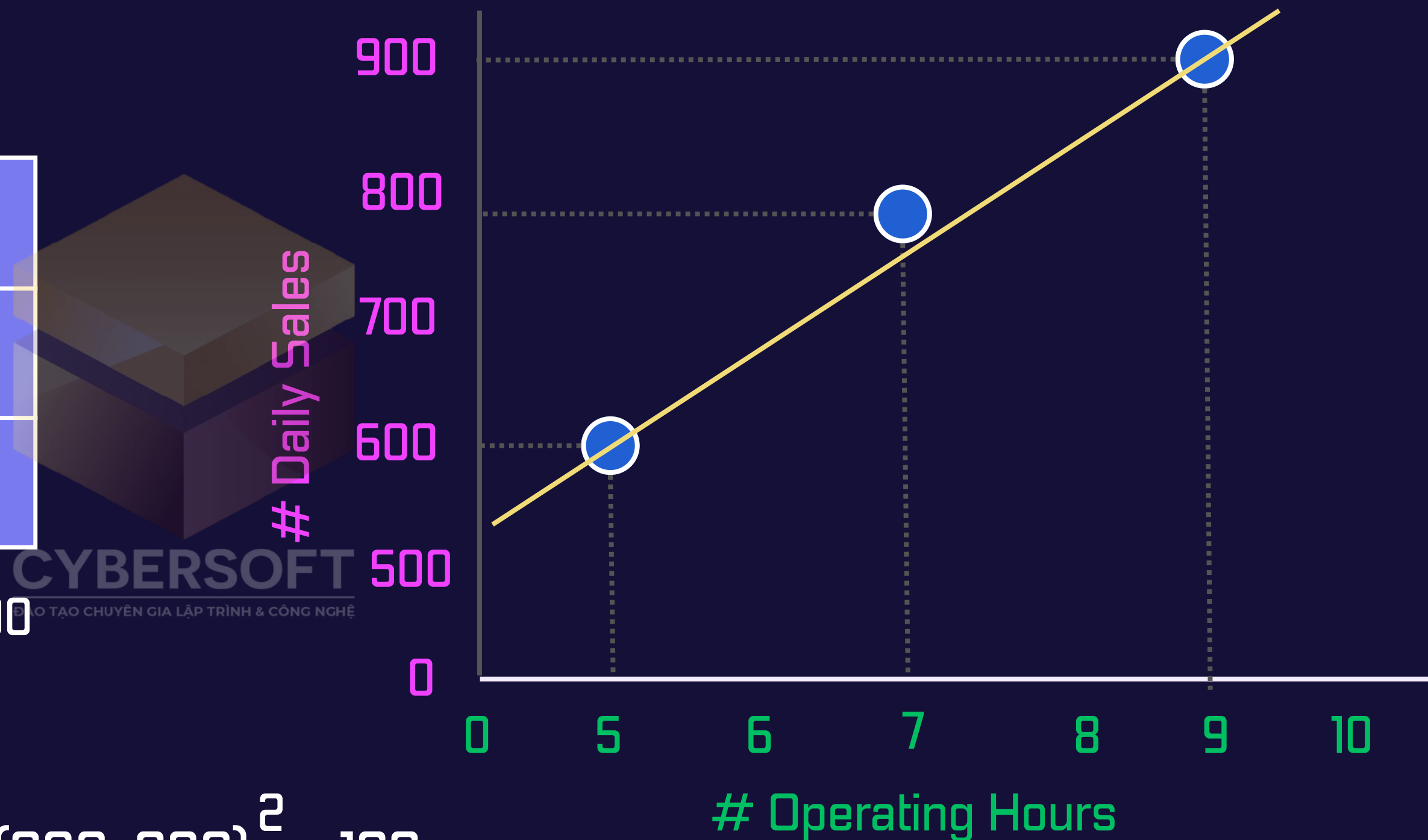
5	600	600
7	790	770
9	900	900

bias (w_1) = 70 weight (w_0) = 200

Linear Regression tối ưu

Loss

$$\text{MSE} = \frac{(600-600)^2 + (790-770)^2 + (900-900)^2}{3} = 133$$



Cơ sở toán học về Gradient Descent

Giả sử chúng áp dụng hàm Loss MSE – Nhắc lại

Công thức MSE:

$$L = \frac{1}{n} \sum_{i=1}^n (y_{\text{pred},i} - y_i)^2$$

- n : Số lượng mẫu dữ liệu.
- $y_{\text{pred},i} = \mathbf{w}^T \cdot \mathbf{x}_i + b$: Giá trị dự đoán, với $\mathbf{w}$ là vector trọng số, $\mathbf{x}_i$ là vector đặc trưng của mẫu i , và b là hằng số.
- y_i : Giá trị thực tế của mẫu i .

Mục tiêu: Tìm $\mathbf{w}$ và b để làm nhỏ nhất L , tức là giảm sai số giữa dự đoán và thực tế.

Gradient (Độ dốc): Kí hiệu **Nabla** (∇) → Gradient là vector chứa thông tin về **hướng** và **độ lớn** mà **hàm mất mát** thay đổi khi ta điều chỉnh các tham số (**trọng số w** và **hằng số b**). Gradient chỉ hướng mà hàm mất mát tăng nhanh nhất.

Đạo hàm (Derivative)

Đạo hàm để tính tốc độ thay đổi của một hàm tại một điểm theo một biến số. Trong trường hợp này, ta cần đạo hàm của L theo các tham số w và b

Cơ sở toán học về Gradient Descent

KÍ HIỆU:

Nếu hàm mất mát là $L(w_1, w_2)$ (phụ thuộc vào hai trọng số w_1 và w_2)

Đạo hàm của L theo w_1 : $\partial L / \partial w_1$ cho ta biết ảnh hưởng của w_1 đến mất mát.

Gradient descent: sử dụng đạo hàm để "đi ngược" hướng tăng của hàm mất mát, tức là trừ đi gradient nhân với learning rate để giảm mất mát.

Tìm Gradient cho: $f(\theta)$

- θ (theta) là một vector, thường được dùng để ký hiệu tập hợp các tham số của một mô hình cần tối ưu (trong Linear Regression thì các tham số chính là hệ số w)
- Đạo hàm của hàm số đó tại một điểm θ bất kỳ được ký hiệu là $\nabla_{\theta} f(\theta)$

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

viết gọn $\rightarrow$

$$\theta = \theta - \eta \nabla_{\theta} f(\theta)$$

Quy tắc cần nhớ: Gradient Descent luôn luôn đi ngược hướng với đạo hàm.

Cập nhật bias (w_1) và weight (w_0)

Gradient của MSE đối với weight

$$\frac{\partial \text{MSE}}{\partial w} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_i$$

Gradient của MSE đối với bias

$$\frac{\partial \text{MSE}}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Cập nhật weight

$$w = w - \alpha \cdot \frac{\partial \text{MSE}}{\partial w}$$

Cập nhật bias

$$b = b - \alpha \cdot \frac{\partial \text{MSE}}{\partial b}$$

Trong đó:

- α : Learning rate (tỷ lệ học), một hằng số nhỏ (ví dụ: 0.0001) để kiểm soát bước cập nhật. (η (Eta) – một số tài liệu học thuật, công thức toán có thể dùng kí hiệu này)
- Dấu trừ đảm bảo rằng w và b được điều chỉnh theo hướng giảm MSE.

Gradient Descent hoạt động



MSE

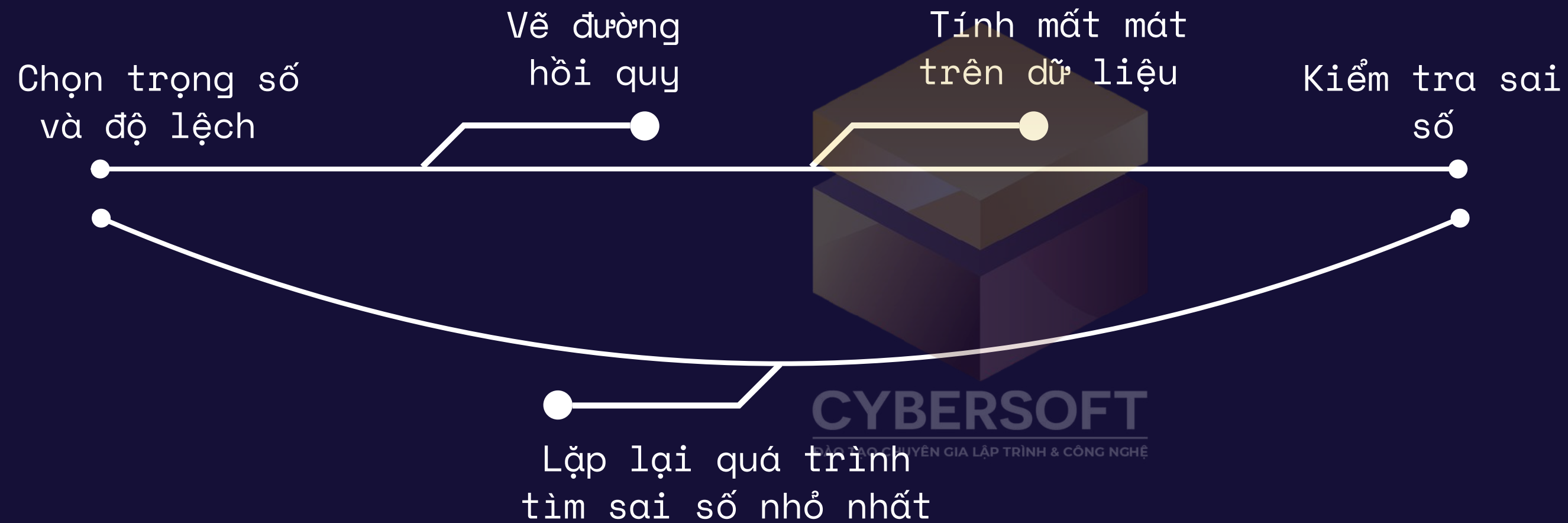
598 033



5 000



133



Features	Label	Predict
#Operating Hours (hours)	#Daily Sales (USD)	
5	600	600
7	790	770
9	900	900

#Bắt đầu giả định

$$\text{bias } (w_1) = 1$$

$$\text{weight } (w_0) = 1$$

$$\text{weight } (\omega_1) * \text{Features} + \text{bias } (\omega_0) = \text{Predict} - \text{Label} = \text{Loss}$$

1	5	1	6	600	-594
	7		8	790	-782
	9		10	900	-890

```
import numpy as np
```

```
weight = 1  
bias = 1
```

```
features = np.array([5, 7, 9])  
label = np.array([600, 790, 900])  
predictions = weight * features + bias  
error = predictions - label  
print("Predictions:", predictions)  
print("Error:", error)
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



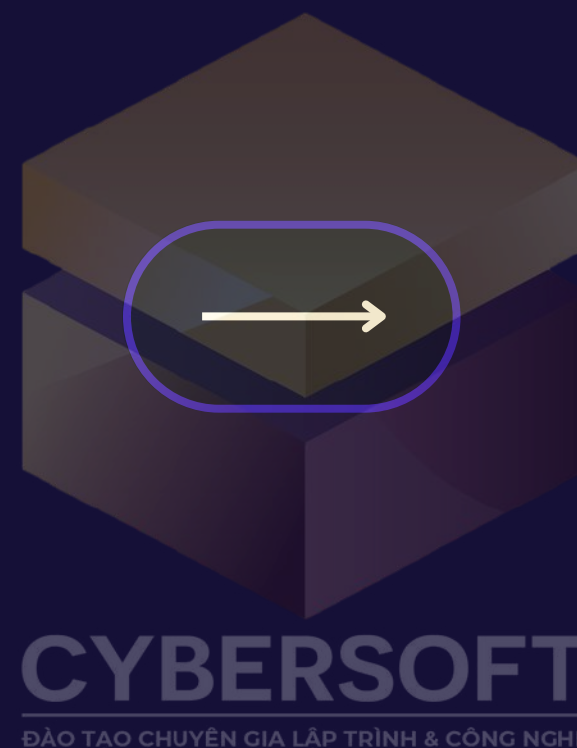
Sai số cao cần cải thiện

```
Predictions: [ 6  8 10]  
Error: [-594 -782 -890]
```

```
import numpy as np

weight = 1
bias = 1

features = np.array([5, 7, 9])
label = np.array([600, 790, 900])
predictions = weight * features + bias
error = predictions - label
print("Predictions:", predictions)
print("Error:", error)
```



```
features = np.array([5, 7, 9])
labels = np.array([600, 790, 900])

weight = 0.0
bias = 0.0

learning_rate = 0.01
epochs = 1000

n = len(features)

# Gradient Descent Loop
for i in range(epochs):
    predictions = weight * features + bias
    error = predictions - labels

    weight_grad = (2/n) * np.dot(error, features)
    bias_grad = (2/n) * np.sum(error)
    weight -= learning_rate * weight_grad
    bias -= learning_rate * bias_grad

# In kết quả
print(f"Final weight: {weight:.4f}")
print(f"Final bias: {bias:.4f}")
```

=> Có thể gọi quá trình này là mô hình huấn luyện (Model Training)

Vector đặc trưng

Vector đặc trưng là cách biểu diễn các đặc trưng này dưới dạng một danh sách số (một hàng hoặc cột số). Ví dụ:

- Nhà 1: `[100 m2, 3 phòng, 1 vị trí]` → vector đặc trưng là `[100, 3, 1]`.
- Mỗi số trong vector đại diện cho một đặc trưng (diện tích, số phòng, vị trí).

```
features = np.array([5, 7, 9])  
labels = np.array([600, 790, 900])
```

Tích vô hướng

```
weight_grad = (2/n) * np.dot(error, features)
```

`.dot` là phép toán trong Numpy được dùng để tính **tích vô hướng** (dot product)

Giả sử:

- `error = [1, -0.5, 0.2]` (sai số của 3 mẫu dữ liệu).
- `features = [[100, 3, 1], [200, 4, 2], [150, 2, 0]]` (danh sách vector đặc trưng của 3 mẫu).

Kết quả của: `np.dot(error, features)`

Nhân từng phần tử của `error` với các phần tử tương ứng trong từng vector `feature`, rồi cộng lại.
Kết quả là một vector mới, ví dụ: `[100*1 + 3*(-0.5) + 1*0.2, 200*1 + 4*(-0.5) + 2*0.2, ...]`.

- Gradient Descent Là Gì
- "Bước Chân" quyết định của mọi mô hình AI
- Tại Sao Gradient Descent Quan Trọng
- Cách Gradient Descent Hoạt Động

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

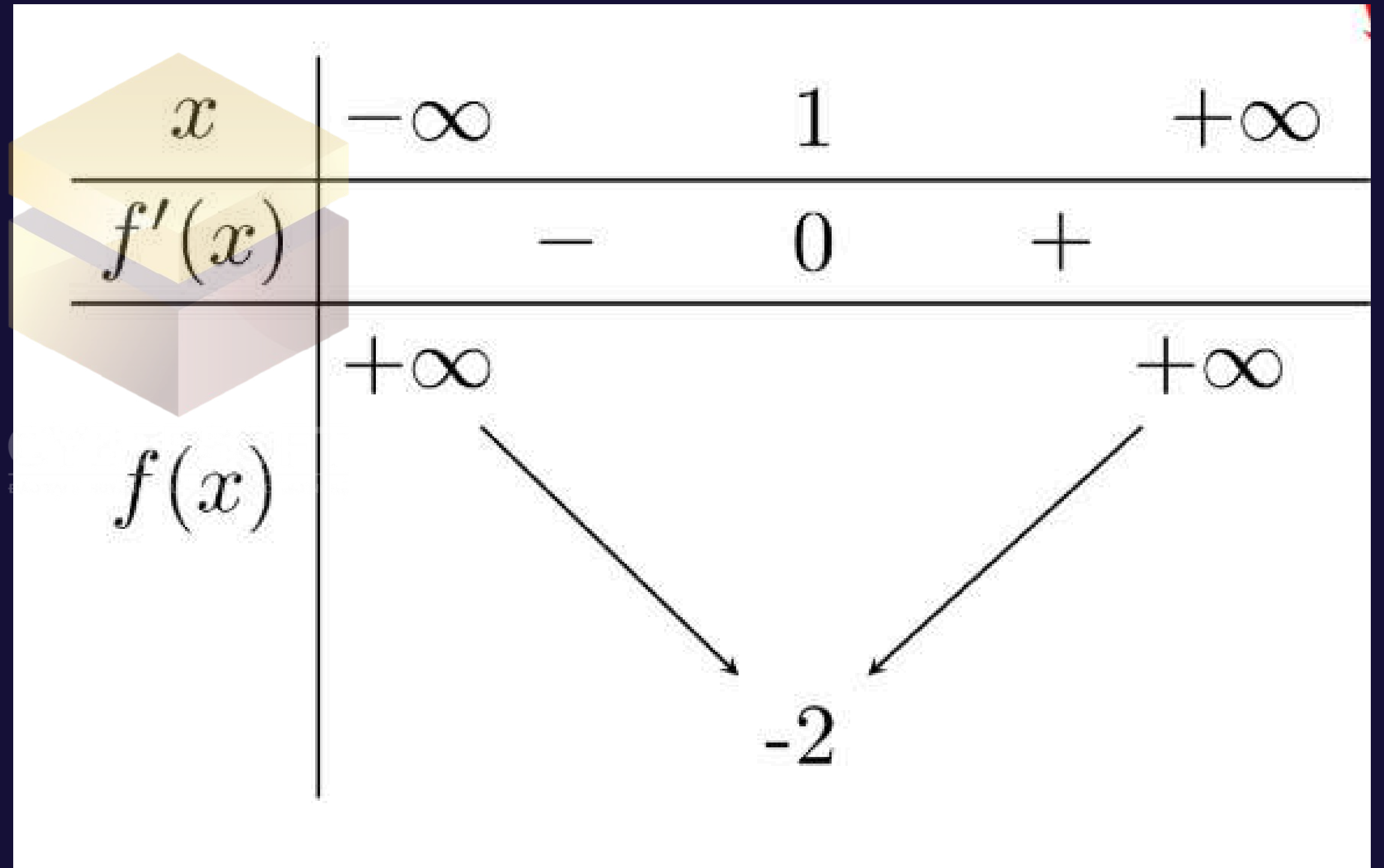


KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

Chắc các bạn còn nhớ đạo hàm ?

$$F(x) = \frac{1}{2}(x-1)^2 - 2$$

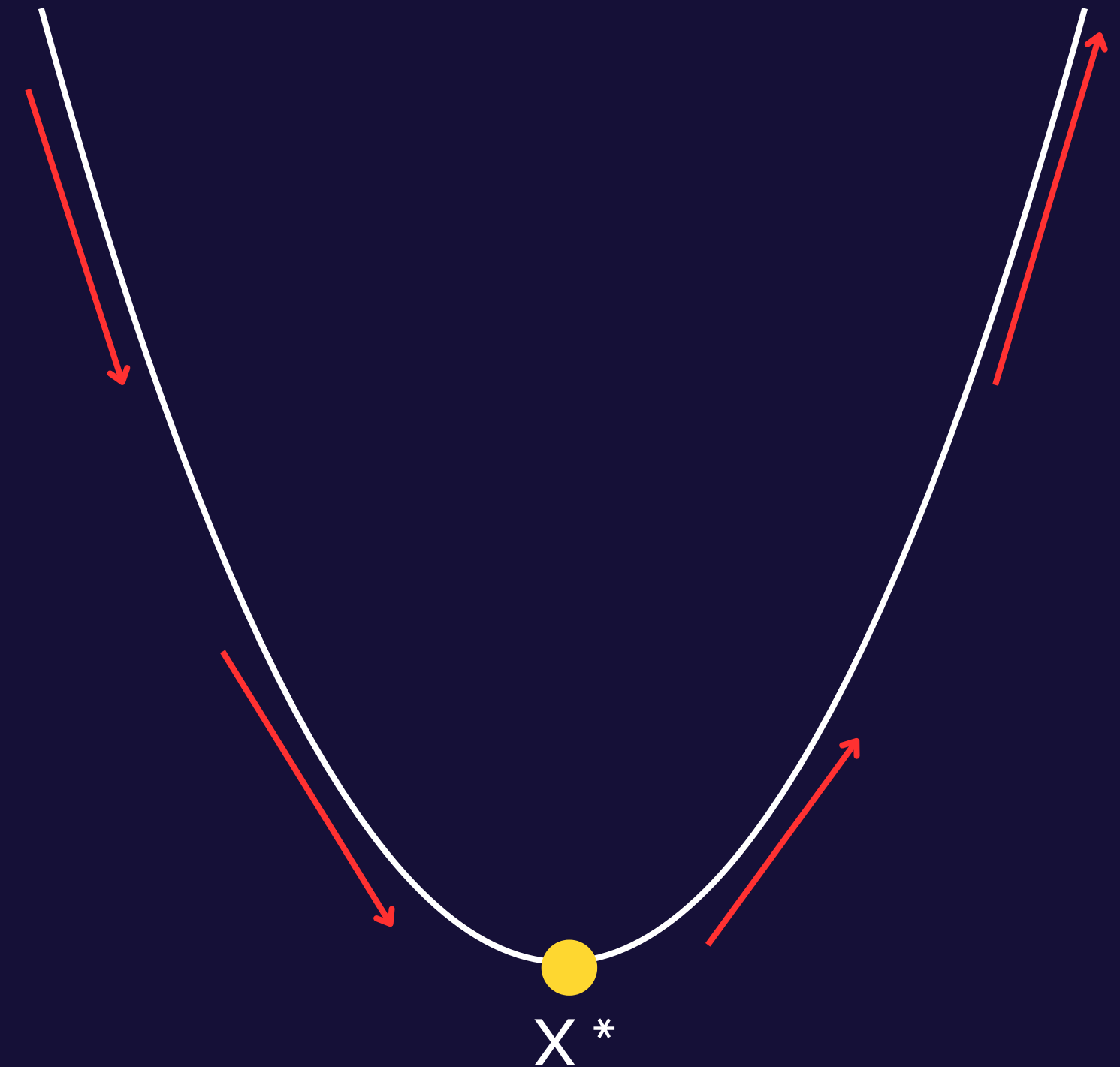
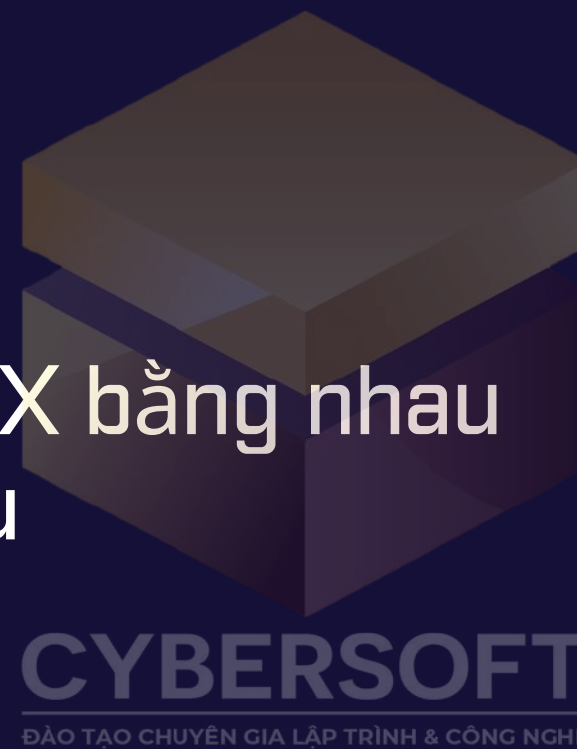
$$\Rightarrow F'(x) = x - 1$$



KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

Với hàm $F(x)$ có 1 cực trị

Các tốc độ học n của từng điểm X bằng nhau
thì độ dốc của cả hai là như nhau



Cực trị

KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

Với hàm có nhiều cực trị nhánh phải

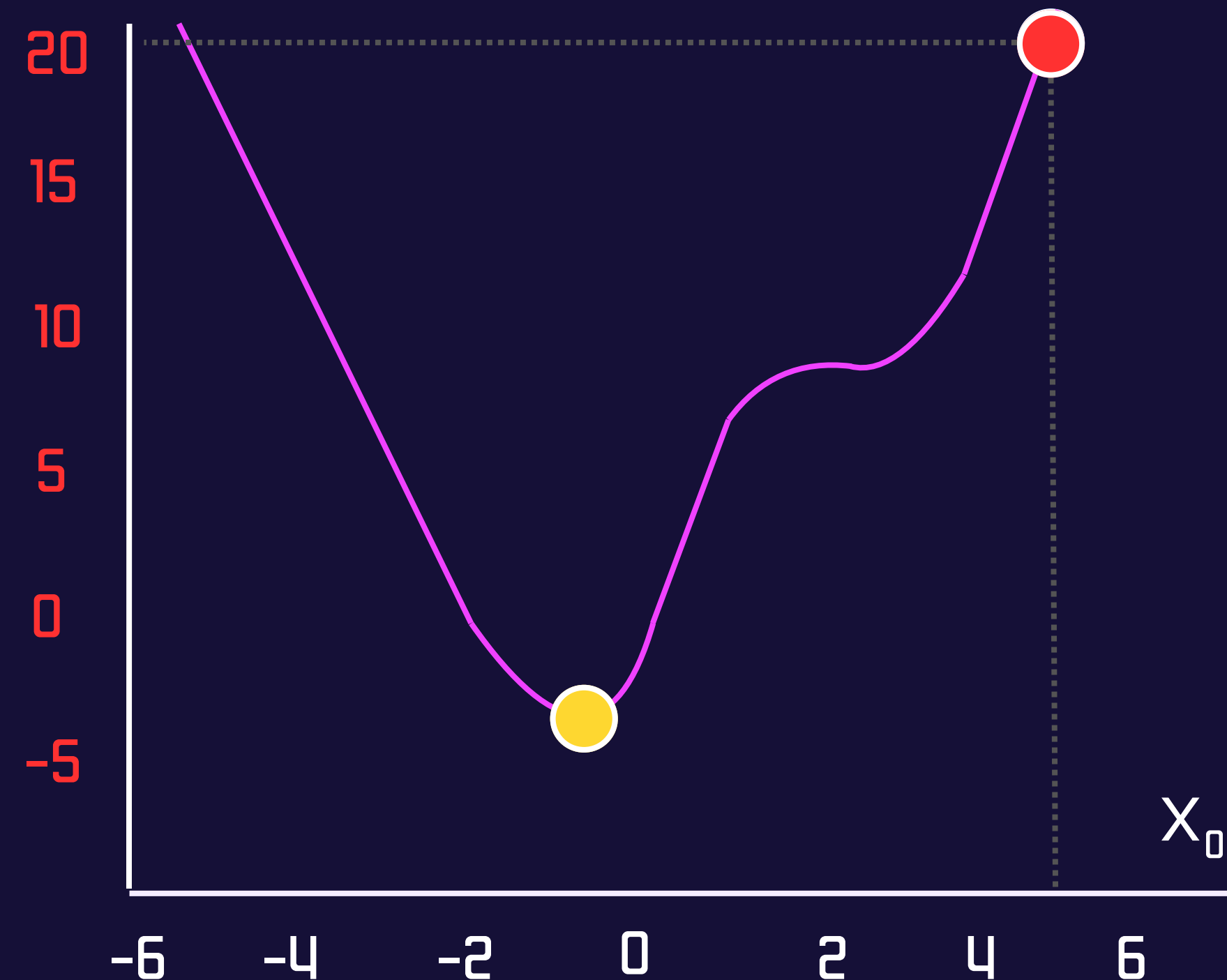
➤ $F(x) = x^2 + 5\sin(x)$

➤ Với $x_0 = 5$

➤ Learning rate: $n = 0,1$



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



→ Tại Iteration = 0 , Cost = 20,21 , $X_0 = 5$

↓
Là số Epoch mà mô hình đã học

● Nơi mô hình tìm được điểm cực tiểu (học thành công)

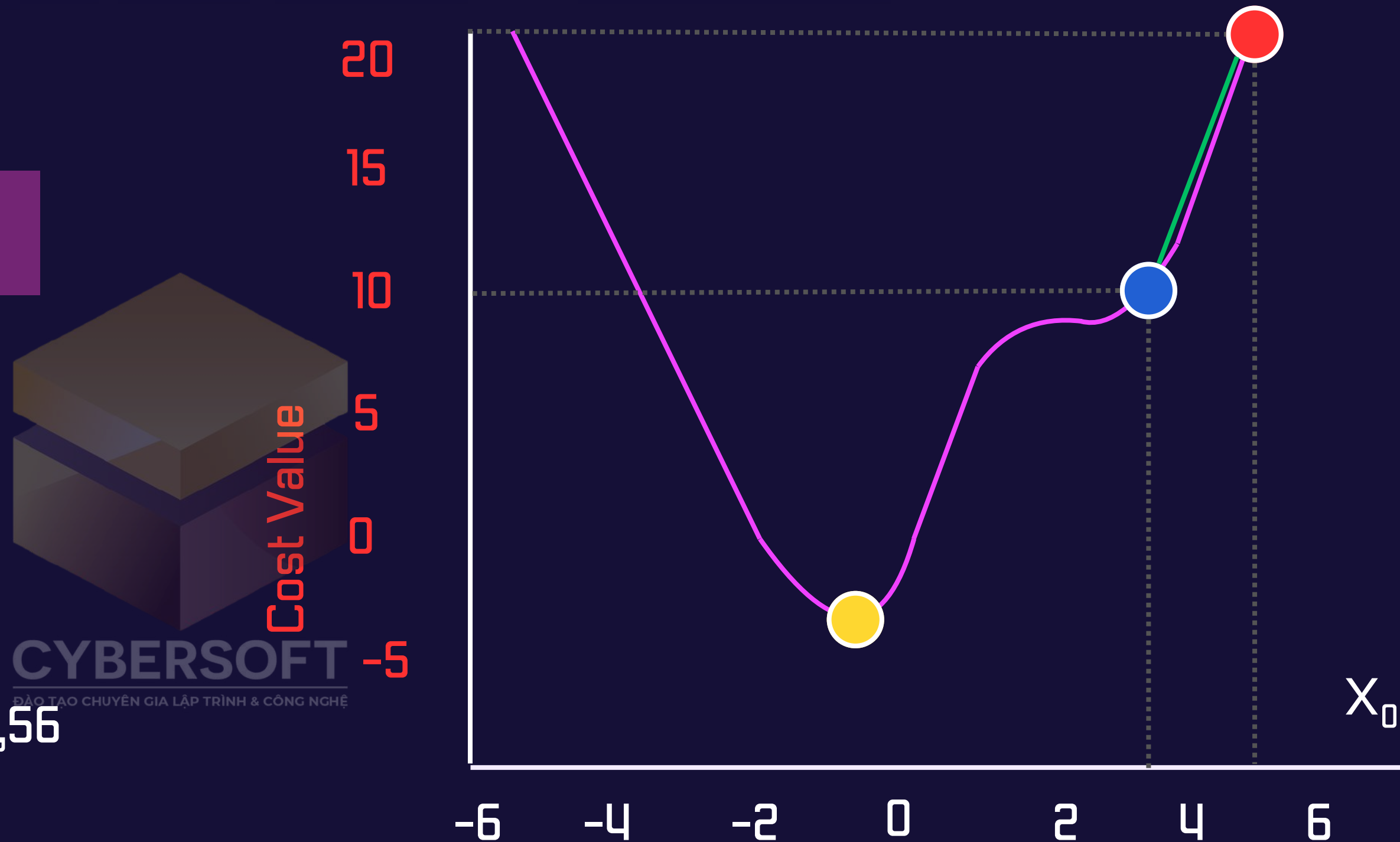
KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

➤ $F(x) = x^2 + 5\sin(x)$

➤ Với $x_0 = 5$

➤ Learning rate: $n = 0,1$

→ Tại Iteration = 2, Cost = 10, $X_0 = 3,56$

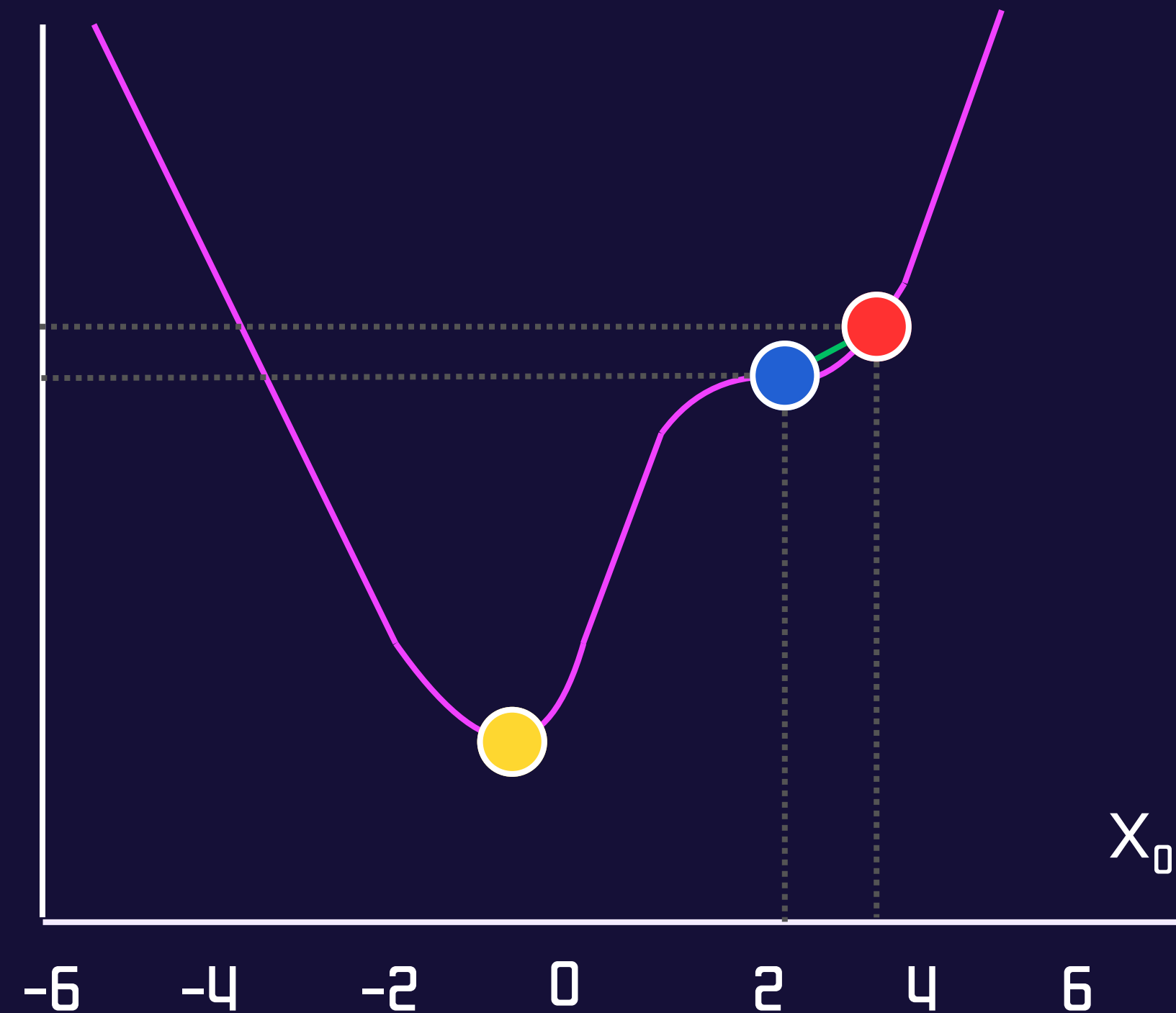


KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

$$F(x) = x^2 + 5\sin(x)$$

Tại Iteration = 3 trở đi mô hình bắt đầu học chậm lại, vì gradient rất nhỏ, các bước cập nhật của nó cũng sẽ rất nhỏ

=> Thuật toán la cà hoặc tiến rất chậm trong vùng đó, mất nhiều lần lặp hơn để thoát ra khỏi khu vực đó



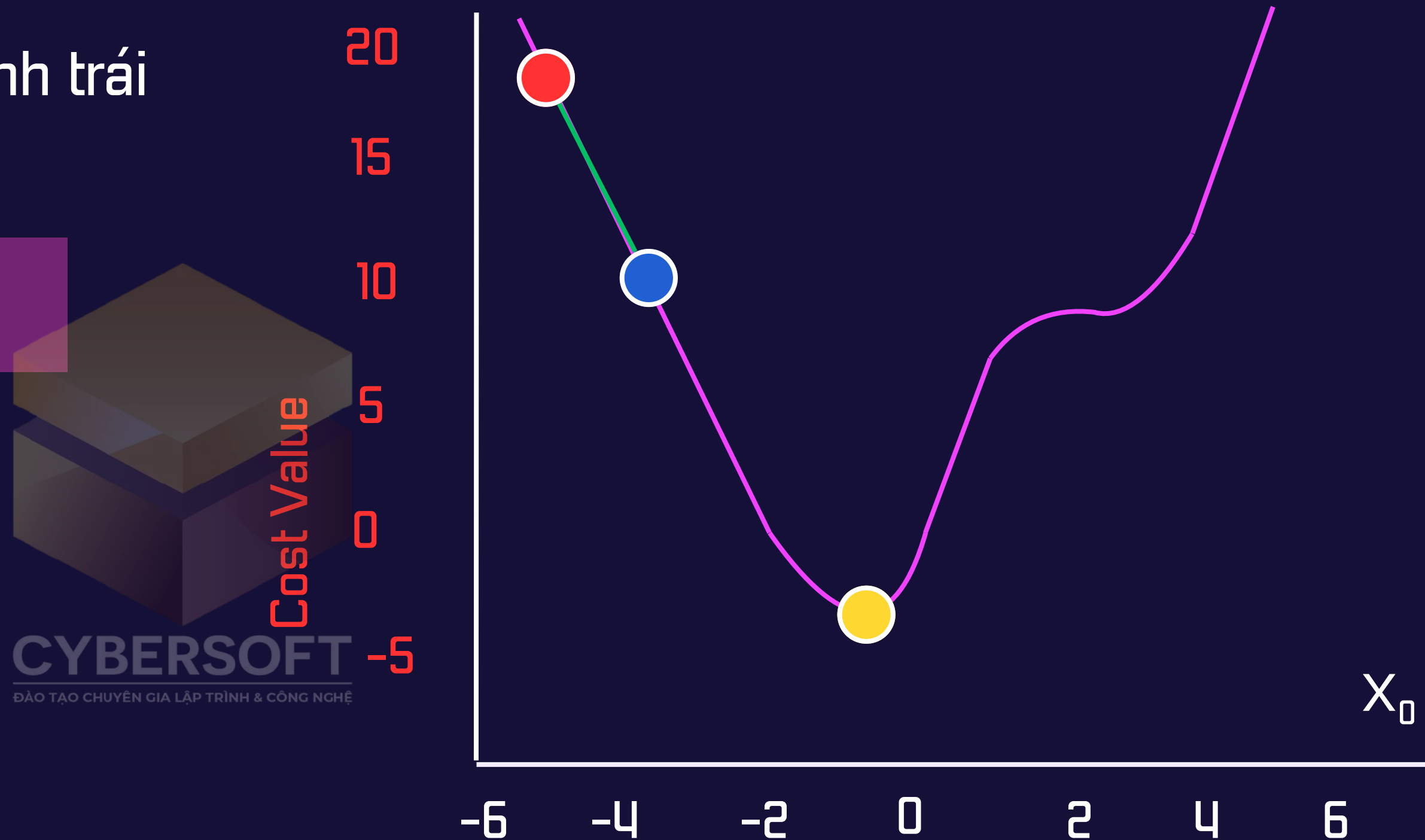
KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

Ta xét các điểm cực trị bên nhánh trái

➤ $F(x) = x^2 + 5\sin(x)$

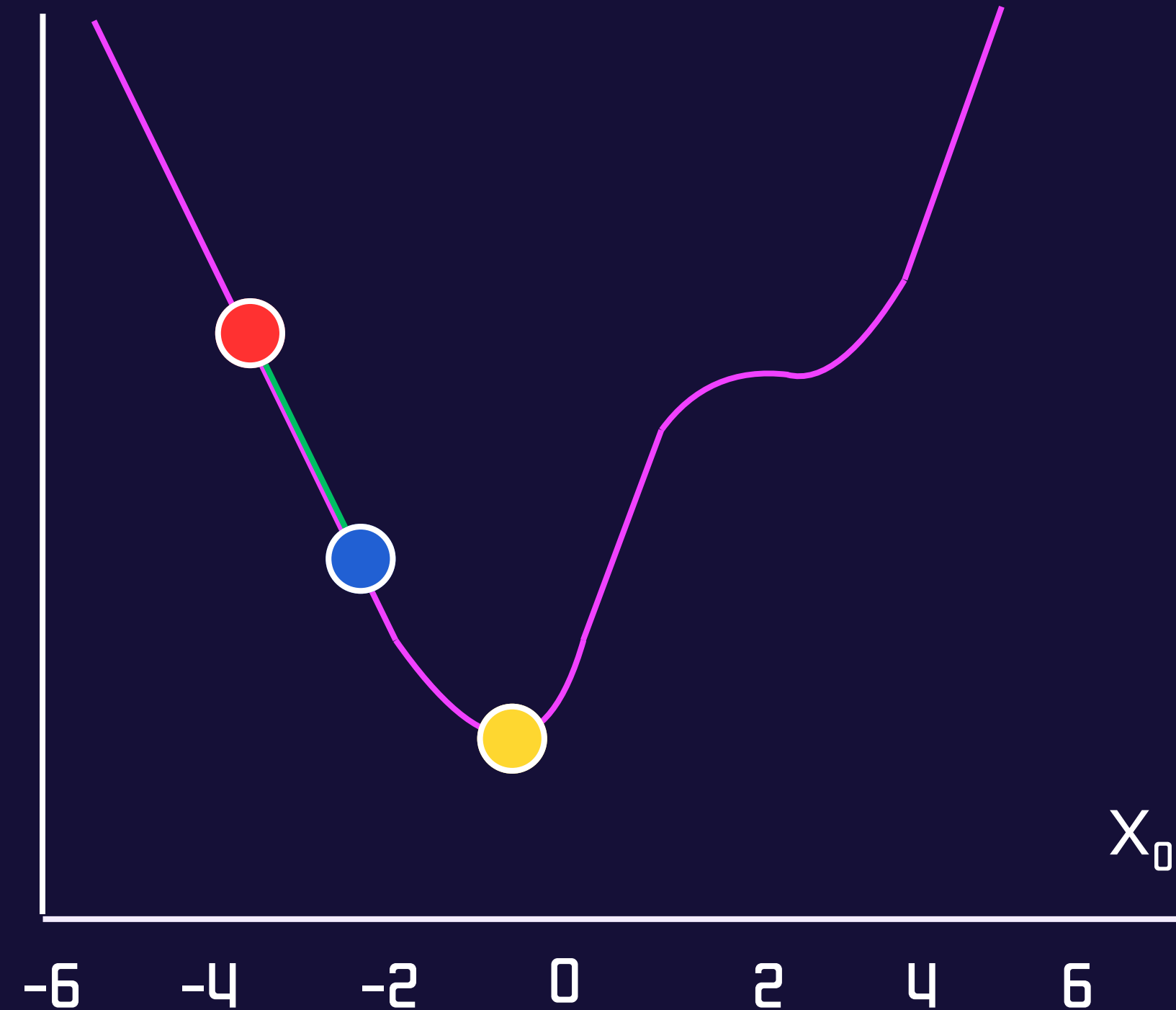
➤ Với $x_0 = -5$

➤ Learning rate: $n = 0,1$

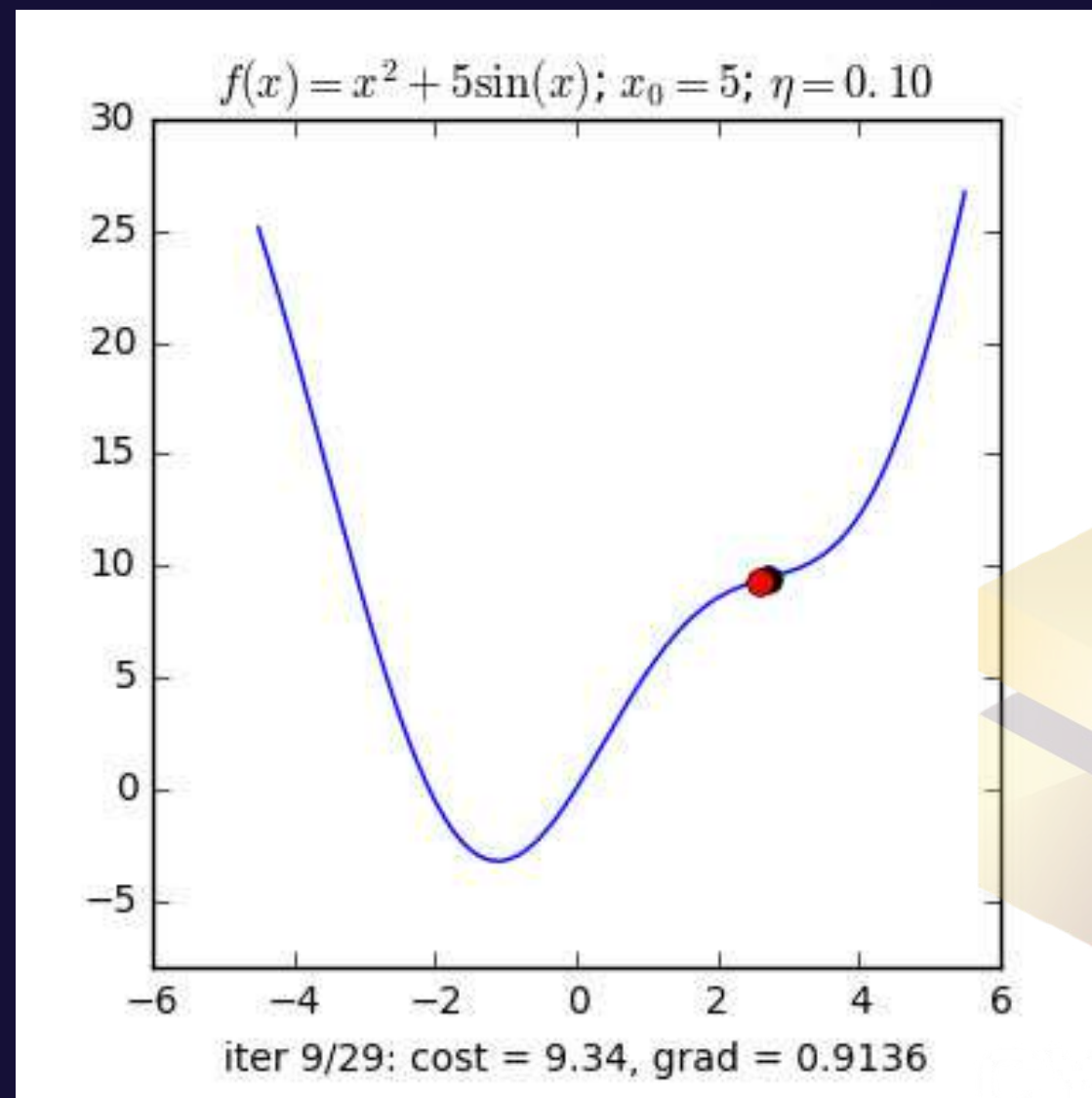


KIẾN THỨC TOÁN HỌC GRADIENT DESCENT

- Tại đây, Vị trí cực tiểu toàn cục của hàm số này nằm gần $x \approx -1$.
- Vì điểm khởi tạo $X_0 = -5$ gần với cực tiểu này, thuật toán có thể hội tụ đến nó nhanh hơn



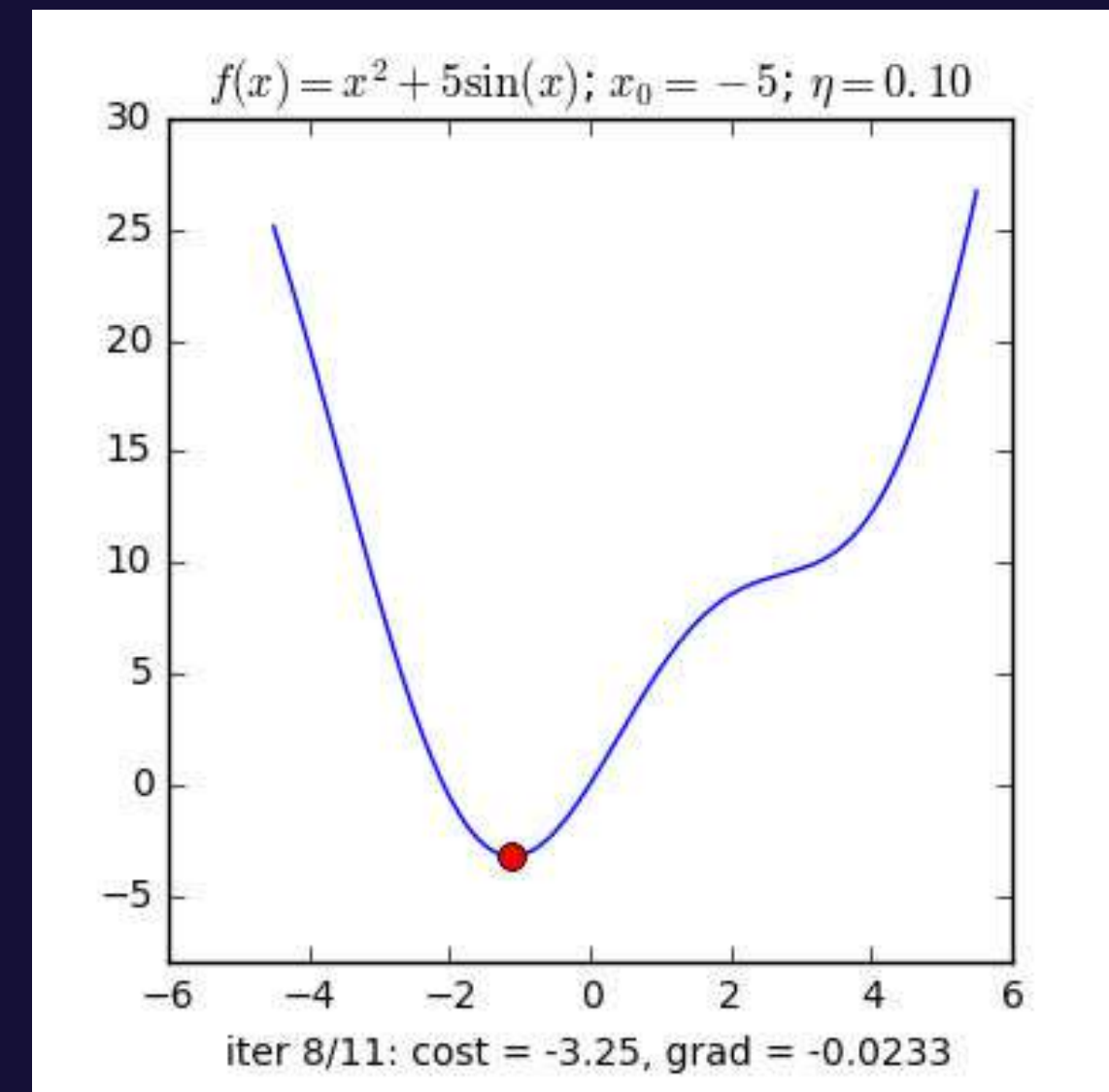
KIẾN THỨC TOÁN HỌC GRADIENT DESCENT



Với $x_0 = 5$

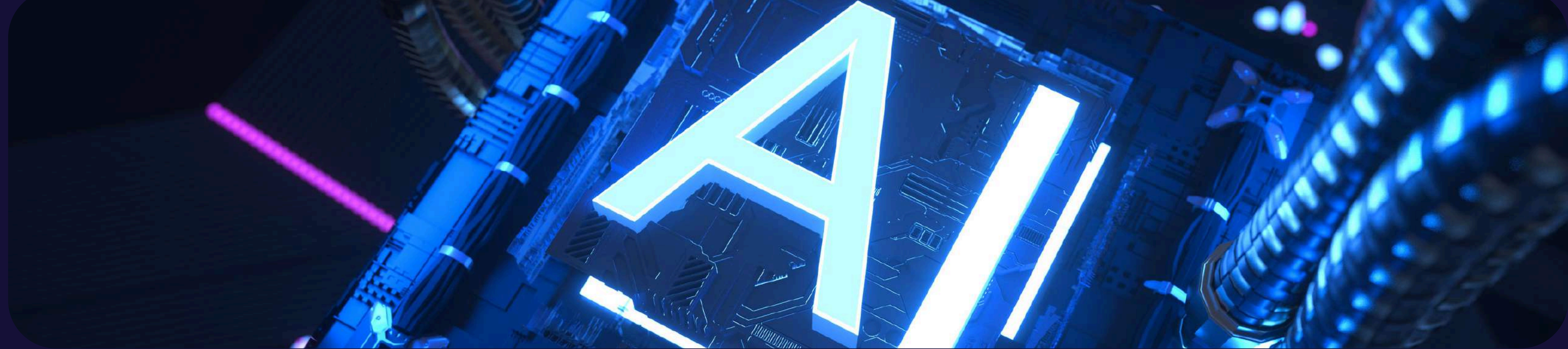
Chậm hơn. Do gradient nhỏ, các bước di chuyển nhỏ hơn, đặc biệt ở vùng "bằng phẳng".
=> Khả năng vẫn tìm được cực tiểu toàn cục, nhưng tốn nhiều thời gian hơn để vượt qua các "bẫy" cực bộ hoặc vùng bằng phẳng.

<https://cybersoft.edu.vn>



Với $x_0 = -5$

Nhanh hơn. Do gradient lớn, thuật toán thực hiện các bước di chuyển lớn về phía cực tiểu.
=> Khả năng cao sẽ tìm được cực tiểu toàn cục do khởi tạo gần đó.



Bài tập thực hành

Supervised Learning

Dự đoán giá nhà bằng Linear Regression.

Hiện Trạng

Ý nghĩa của việc dự đoán giá nhà



Giúp người mua tìm được nhà phù hợp ngân sách

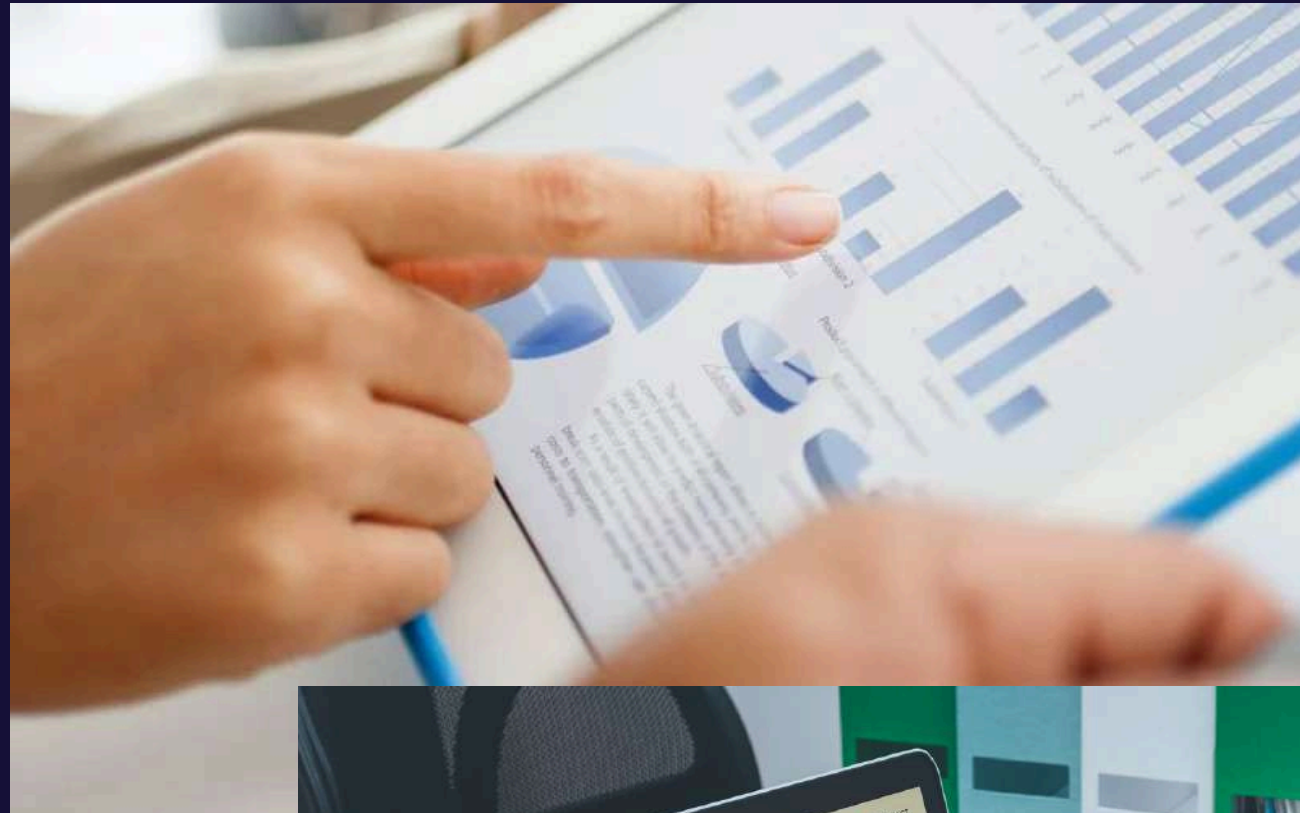
CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



Hỗ trợ nhà đầu tư tối ưu hóa lợi nhuận

Hiện Trạng

Phương pháp truyền thống



Sử dụng thống kê cơ bản, chuyên gia địa phương, xu hướng thị trường

=> Thiếu độ chính xác, không mở rộng được



Hiện Trạng

Xu hướng mới với Machine Learning



Khai thác dữ liệu lớn, phân tích nhiều yếu tố cùng lúc

Cải thiện độ chính xác và khả năng thích ứng với thị trường



Kết quả bài tập

House Price Prediction

The image shows a web application for house price prediction. On the left, a sidebar titled 'Input Features' contains five input fields: 'Square Feet' (500), 'parking' (1), 'Totalrooms(including Bedrooms,Bathrooms,guestrooms)' (4), 'Furnish Status' (semi-furnished), and 'Heating_And_Air-conditioning' (Both). These fields are highlighted with a red border. An arrow points from this sidebar to the text 'Các đầu vào' (Inputs). The main area on the right has a large 'House Price Prediction' title. Below it is a 'Predict Price' button. An arrow points from this button to the text 'Giá dự đoán' (Predicted Price). Below the button, a green box displays the 'Predicted Price: \$4,035,523.33'. The background features a 3D cube graphic and the 'CYBERSOFT' logo with the tagline 'ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ'.

Input Features

Square Feet

500

parking

1

Totalrooms(including Bedrooms,Bathrooms,guestrooms)

4

Furnish Status

semi-furnished

Heating_And_Air-conditioning

Both

House Price Prediction

Predict Price

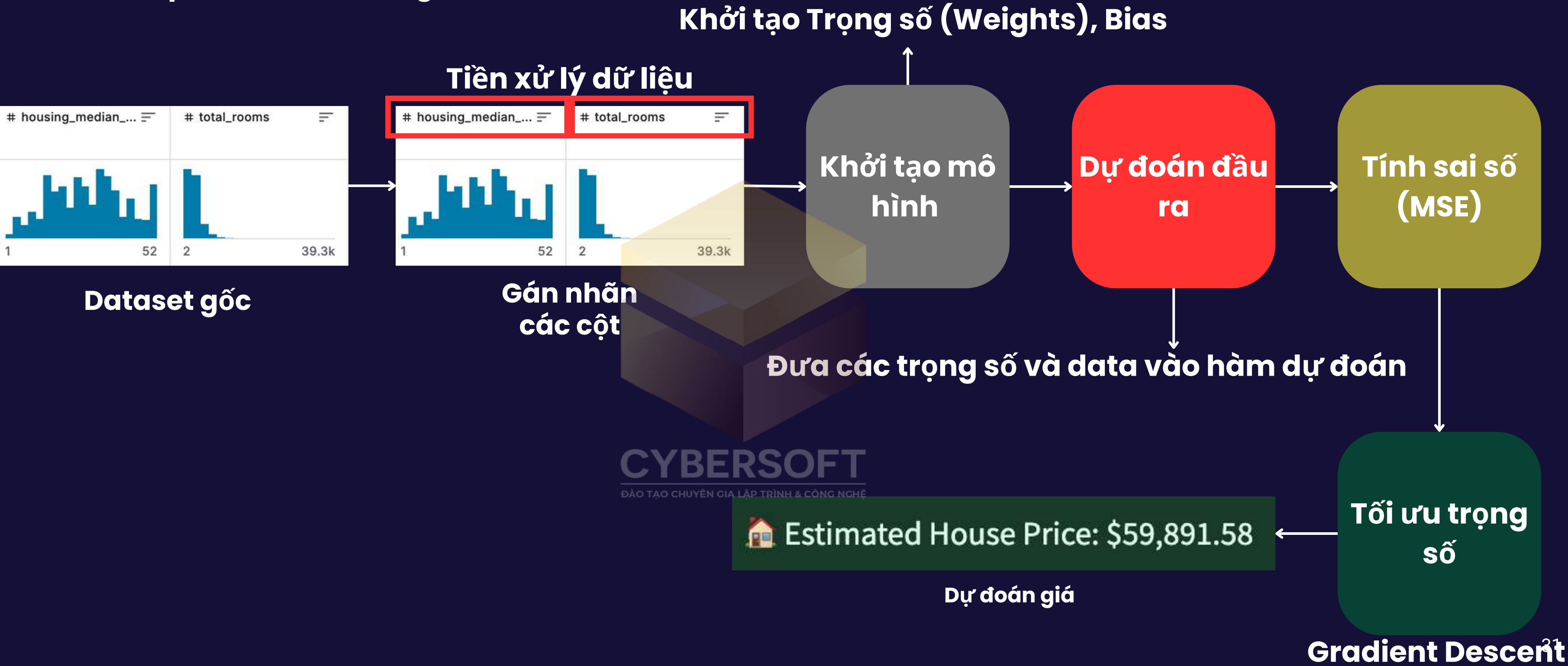
Predicted Price: \$4,035,523.33

Các đầu vào

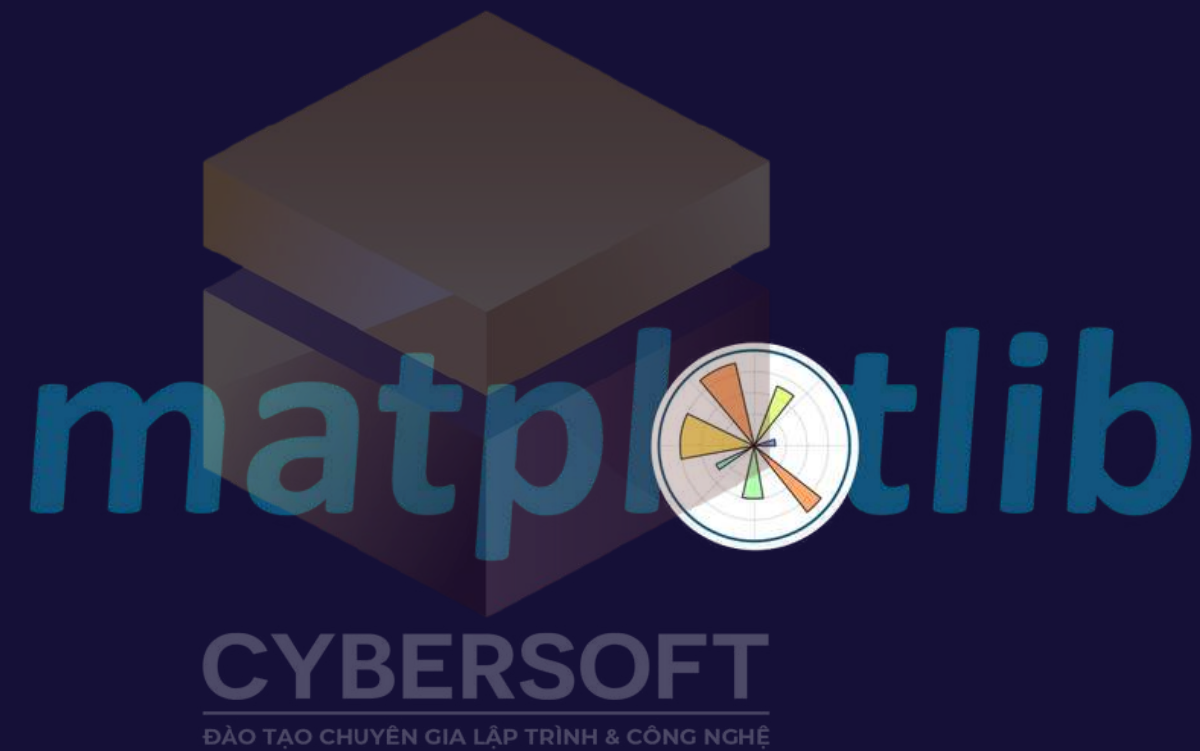
Giá dự đoán

Flow dự án

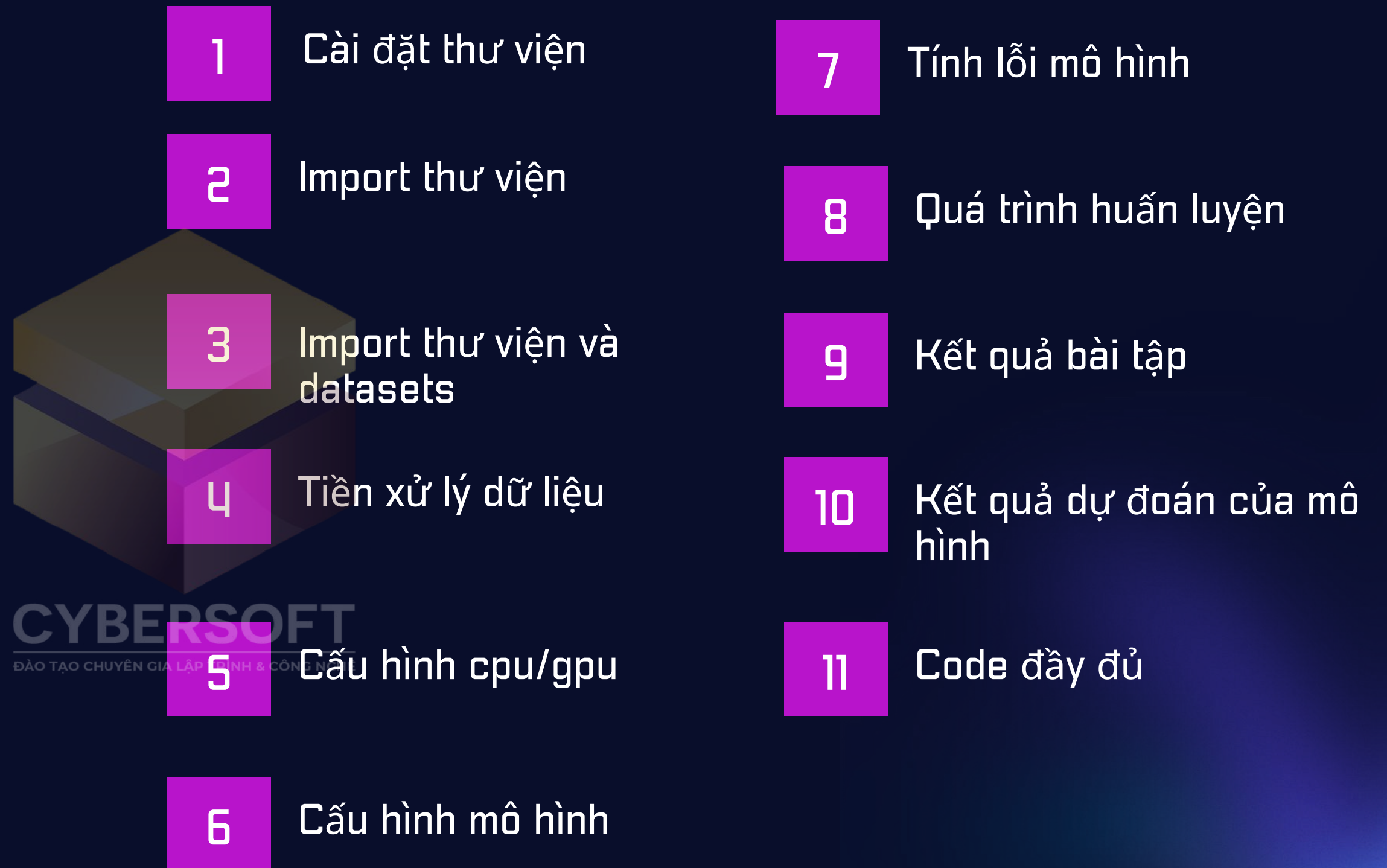
Supervised Learning



Thư viện áp dụng



Các bước thực hiện



Bài tập thực hành Boston Housing

1. Cài đặt thư viện

```
pip install torch numpy pandas scikit-learn seaborn matplotlib
```



Bài tập thực hành Boston Housing

2. Import thư viện

```
# Nhập thư viện numpy để xử lý mảng và tính toán số học
import numpy as np
# Nhập thư viện pandas để xử lý và phân tích dữ liệu dạng bảng
import pandas as pd
# Nhập hàm train_test_split từ sklearn để chia dữ liệu thành tập huấn luyện và kiểm tra
from sklearn.model_selection import train_test_split
# Nhập mô hình LinearRegression từ sklearn để xây dựng mô hình hồi quy tuyến tính
from sklearn.linear_model import LinearRegression
# Nhập các hàm mean_squared_error và r2_score từ sklearn để đánh giá hiệu suất mô hình
from sklearn.metrics import mean_squared_error, r2_score
# Nhập thư viện seaborn để vẽ biểu đồ trực quan hóa dữ liệu
import seaborn as sns
# Nhập thư viện matplotlib.pyplot để tạo và tùy chỉnh biểu đồ
import matplotlib.pyplot as plt
# Nhập thư viện torch để sử dụng các tính năng của PyTorch
import torch
```

Bài tập thực hành Boston Housing

3. Import thư viện và datasets

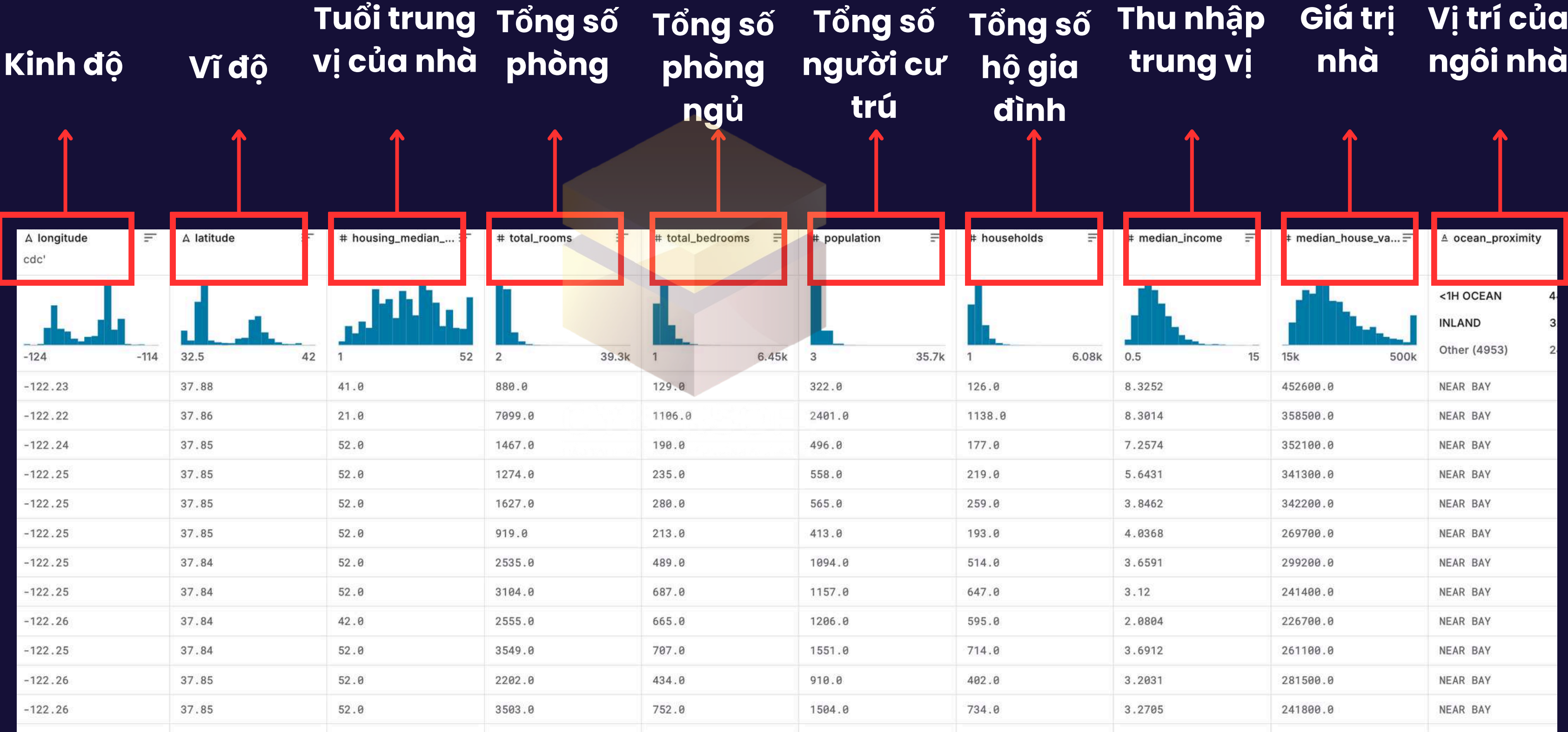
Link nguồn datasets: <https://www.kaggle.com/datasets/yasserh/housing-prices-dataset/data>

```
# Nhập thư viện kagglehub để tải dataset từ Kaggle
import kagglehub
# Tải dataset 'housing-prices-dataset' từ Kaggle và lưu đường dẫn vào biến
yasserh_housing_prices_dataset_path = kagglehub.dataset_download('yasserh/housing-prices-dataset')
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

```
# Đọc file CSV 'Housing.csv' vào DataFrame của pandas
df = pd.read_csv("/kaggle/input/housing-prices-dataset/Housing.csv")
```


Giải thích các column trong datasets



Bài tập thực hành Boston Housing

4. Tiền xử lý dữ liệu

Đối với các máy chỉ có cpu

Loại bỏ các hàng (row) chứa giá trị thiếu (NaN) trong DataFrame df

```
# Chọn các cột cần thiết
df = df[['price', 'area', 'bedrooms', 'bathrooms']]

# Xử lý missing và giá trị không hợp lệ
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)
```

```
# Chia input và output
X = df[['area', 'bedrooms', 'bathrooms']]
y = df['price']
```

Biến đầu vào, đầu ra (target) mà mô hình sẽ dự đoán

Bài tập thực hành Boston Housing

4. Tiền xử lý dữ liệu

Đối với các máy chỉ có cpu

Chia dữ liệu thành tập train và test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

20% dữ liệu sẽ dùng để kiểm tra (test),
80% dùng để huấn luyện (train).

Đặt hạt giống (seed) để kết quả chia là
giống nhau mỗi lần chạy.

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

4. Tiền xử lý dữ liệu

Đối với các máy có gpu

```
# Scale dữ liệu
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y.values.reshape(-1, 1))

# Chia train/test (20% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_scaled, test_size=0.2, random_state=42)

# Chuyển sang tensor
X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).to(device)
```



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

5. Cấu hình cpu/gpu

```
import torch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```



Đối với các máy chỉ có cpu

Using device: cpu



Đối với các máy có gpu

Using device: cuda

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Đối với máy sử dụng cpu

```
model = LinearRegression()
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Mô hình sẽ tự dùng epoch, learning-rate và bias theo cấu hình sẵn của mô hình để tối ưu tài nguyên

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Đối với máy sử dụng gpu

```
class LinearRegressionWithZeroBias(torch.nn.Module):  
    def __init__(self, input_dim):  
        super(LinearRegressionWithZeroBias, self).init()  
        self.weights = torch.nn.Parameter(torch.randn(input_dim, 1))  
        self.bias = torch.nn.Parameter(torch.zeros(1))  
  
    def forward(self, x):  
        return x @ self.weights + self.bias
```

Khởi tạo ma trận trọng số ngẫu nhiên với kích thước phù hợp để nhân với đầu vào X

Khởi tạo Bias ngẫu nhiên cho mô hình

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Đối với máy sử dụng gpu

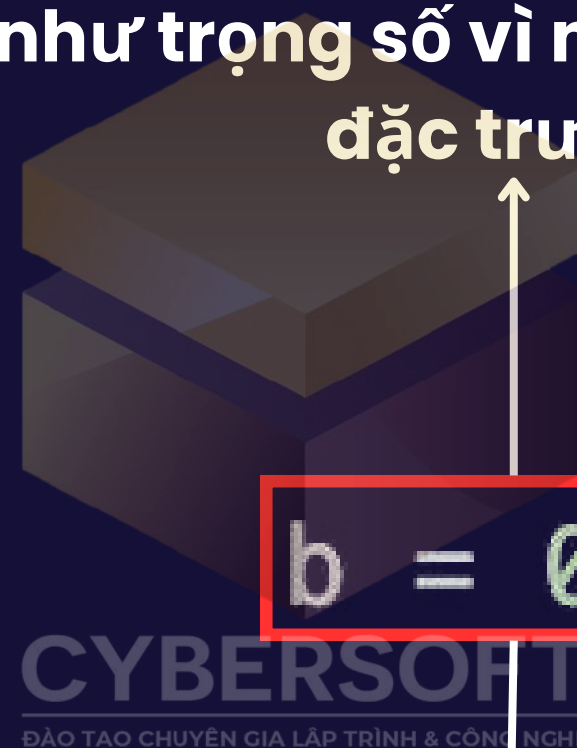
```
self.weights = torch.nn.Parameter(torch.randn(input_dim, 1))
```

Tạo ma trận ngẫu nhiên có kích thước $(input_dim, 1)$ với các giá trị từ phân phối chuẩn ($mean=0, std=1$).
→ Đại diện cho vector trọng số w trong hồi quy tuyến tính.

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Bias thường không cần ngẫu nhiên như trọng số vì nó không ảnh hưởng đến tính phân biệt giữa các đặc trưng.



$b = 0.0$

Khởi tạo bằng 0 để đơn giản hóa, không ảnh hưởng gradient

Nó đóng vai trò điều chỉnh đầu ra tổng thể (dịch đường dự đoán lên/xuống).

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

tốc độ học của mô hình

```
# Siêu tham số  
lr = 0.01  
epochs = 1000  
n = len(X)
```

Số lần lặp qua toàn bộ
tập dữ liệu để huấn
luyện mô hình

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

$lr = 0.01$

Hệ số quyết định bước nhảy khi cập nhật trọng số w và bias b trong thuật toán Gradient Descent.

Nếu lr quá lớn, mô hình có thể nhảy qua điểm tối ưu, gây dao động hoặc không hội tụ.

Nếu lr quá nhỏ, mô hình sẽ học rất chậm và mất nhiều thời gian huấn luyện.

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Learning rate

Giá trị trong các trường hợp
khác nhau

Linear / Logistic Regression
Dataset: Nhỏ (dưới 10.000 mẫu)
0.01 – 0.1

Mạng nơ-ron cơ bản (FNN nhỏ)
0.001 – 0.01

Deep Learning (ResNet, BERT, v.v.)
1e-5 – 5e-4

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Vòng lặp qua toàn bộ tập dữ liệu huấn luyện

epochs = 1000

Nhiều epochs có thể giúp mô hình học kỹ hơn,
nhưng quá nhiều dễ gây quá khớp (overfitting).

Bài tập thực hành Boston Housing

6. Cấu hình mô hình

Epoch

Giá trị trong các trường hợp khác nhau



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Linear / Logistic Regression
Dataset: Nhỏ (dưới 10.000 mẫu)
500 – 2000

=> lr cao giúp hội tụ nhanh hơn

Mạng nơ-ron cơ bản (FNN nhỏ)
50 – 200

=> lr quá lớn có thể làm mất ổn định

Deep Learning (ResNet, BERT, v.v.)
2 – 10

=> Sử dụng Optimizer như AdamW,
có lr warmup

Bài tập thực hành Boston Housing

7. Tính lỗi mô hình

```
y_pred = X @ w + b
```

→ Tính giá trị dự đoán

```
# Tính lỗi  
error = y_pred - y  
  
# Hàm mất mát MSE  
loss = np.mean(error ** 2)
```

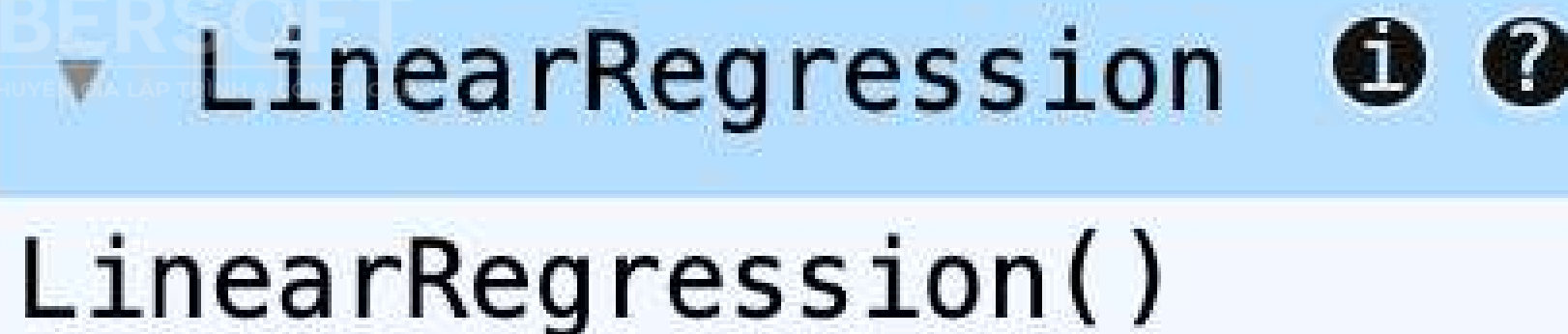
↓
Tính sai số giữa dự đoán và thực tế

Bài tập thực hành Boston Housing

8. Quá trình huấn luyện

Đối với máy sử dụng cpu

```
model.fit(X_train, y_train)
```

The image shows a Jupyter Notebook interface. At the top, there is a light blue header bar with the text 'LinearRegression' in black, followed by an information icon (i) and a help icon (?). Below this header, the text 'LinearRegression()' is displayed in black on a white background. A faint 'CYBERSOFT' watermark is visible in the background of the notebook interface.

Mô hình sẽ tự động train bằng các chỉ số cấu hình
mặc định của mô hình

Bài tập thực hành Boston Housing

8. Quá trình huấn luyện

Đối với máy sử dụng gpu

```
model = LinearRegressionWithZeroBias(input_dim=3).to(device)

# Cấu hình huấn luyện
learning_rate = 0.01
epochs = 1000
criterion = torch.nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

# Huấn luyện
for epoch in range(epochs):
    model.train()
    outputs = model(X_train_tensor)
    loss = criterion(outputs, y_train_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 50 == 0:
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}")
```

```
Epoch [50/1000], Loss: 0.8129
Epoch [100/1000], Loss: 0.5061
Epoch [150/1000], Loss: 0.4599
Epoch [200/1000], Loss: 0.4503
Epoch [250/1000], Loss: 0.4479
Epoch [300/1000], Loss: 0.4472
Epoch [350/1000], Loss: 0.4470
Epoch [400/1000], Loss: 0.4470
Epoch [450/1000], Loss: 0.4469
Epoch [500/1000], Loss: 0.4469
Epoch [550/1000], Loss: 0.4469
Epoch [600/1000], Loss: 0.4469
Epoch [650/1000], Loss: 0.4469
Epoch [700/1000], Loss: 0.4469
Epoch [750/1000], Loss: 0.4469
Epoch [800/1000], Loss: 0.4469
Epoch [850/1000], Loss: 0.4469
Epoch [900/1000], Loss: 0.4469
Epoch [950/1000], Loss: 0.4469
Epoch [1000/1000], Loss: 0.4469
```

Mean Squared Error: 2750037622784.00
R-squared Score: 0.4559

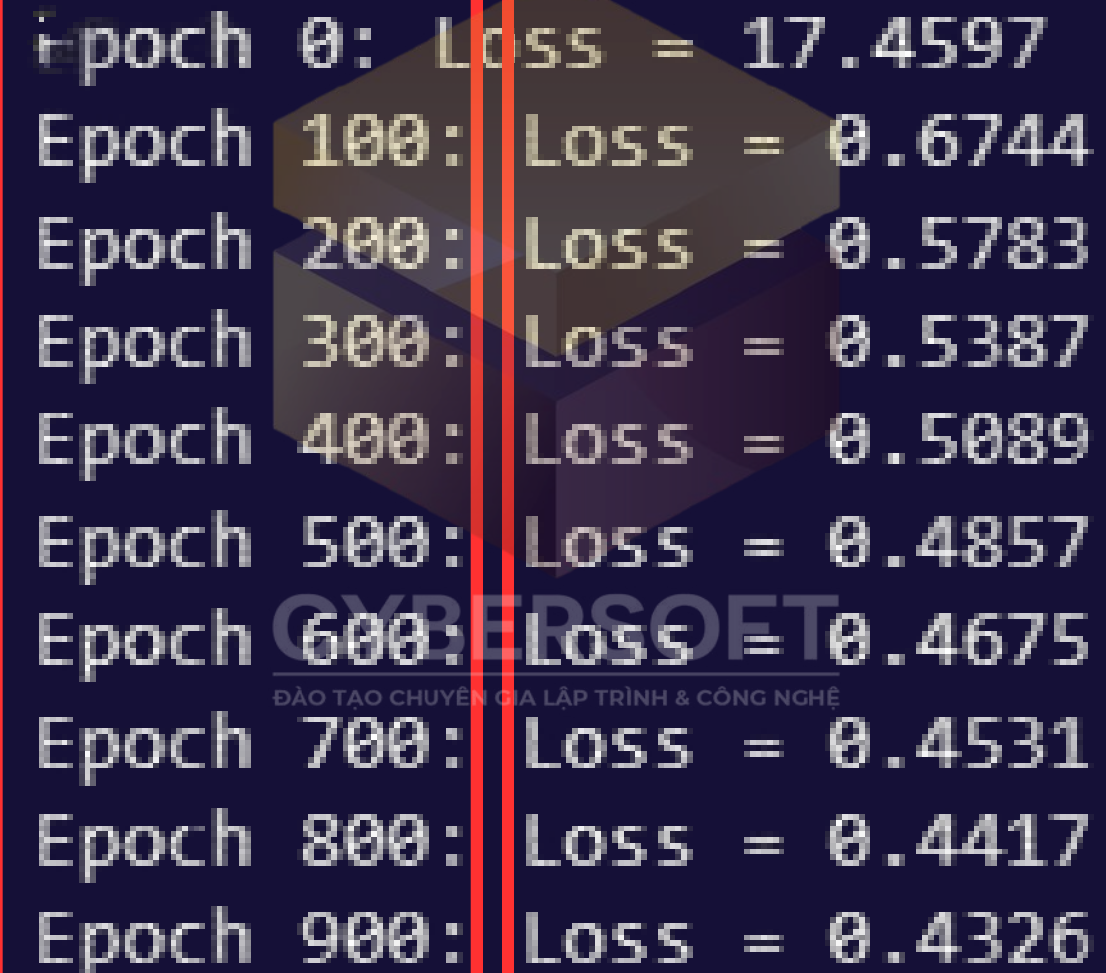
**Mô hình sử dụng cấu hình
mà bạn dùng để train**

Bài tập thực hành Boston Housing

9. Kết quả

- Loss giảm mạnh từ 17.4597 ở epoch đầu tiên xuống chỉ còn 0.4326 sau 900 epoch.
- Quá trình học diễn ra ổn định, thể hiện qua việc loss giảm đều theo từng mốc epoch 100.
- Điều này chứng tỏ learning rate và số epoch đã được lựa chọn phù hợp, giúp mô hình hội tụ tốt mà không bị dao động.
- Nếu học tiếp đến 1000 epoch, mô hình có loss có thể sẽ tiếp tục giảm nhưng giảm ít hoặc giảm không đều => Mô hình bị overfit

Tham số mất mát qua từng bước

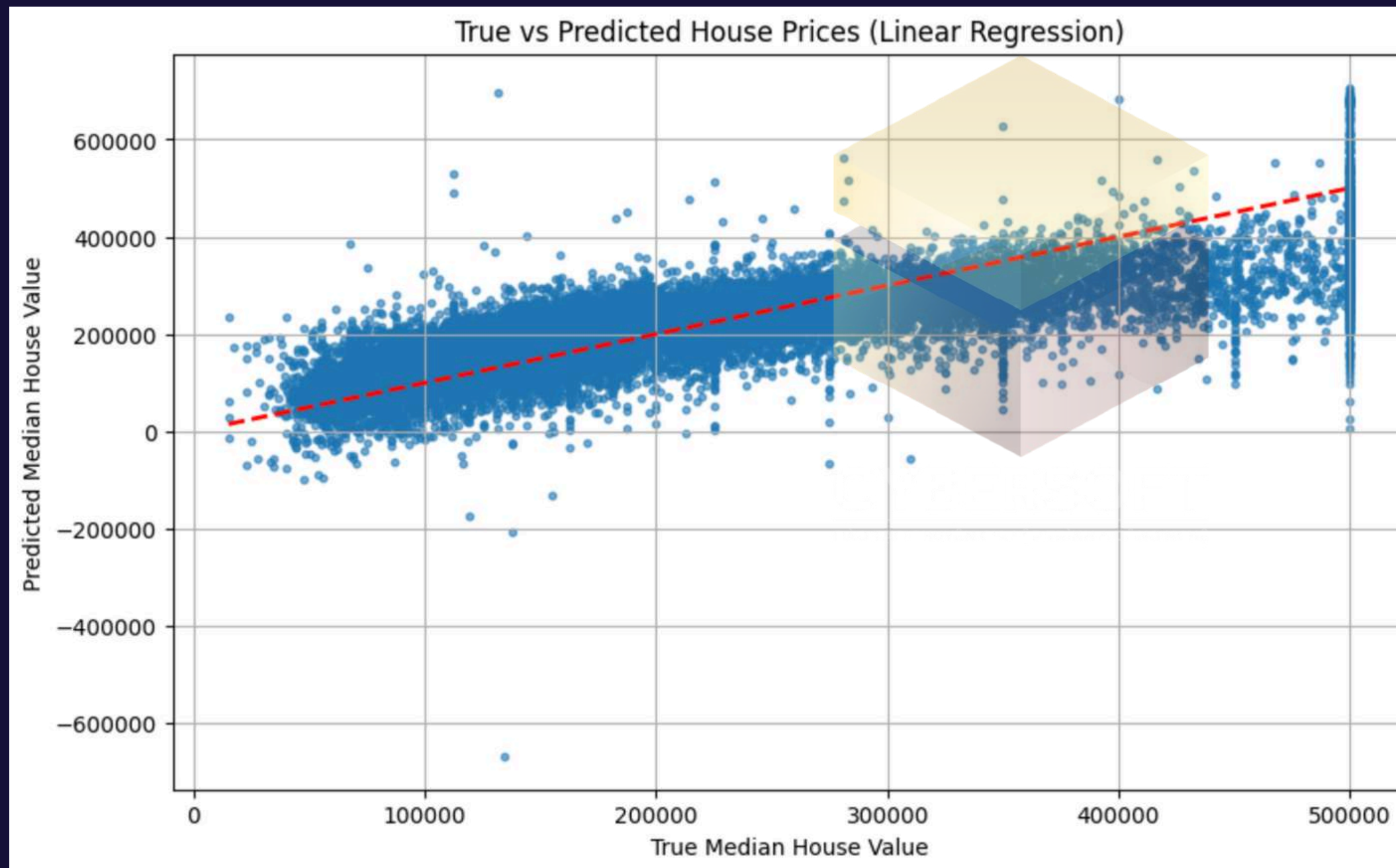


```
Epoch 0: Loss = 17.4597
Epoch 100: Loss = 0.6744
Epoch 200: Loss = 0.5783
Epoch 300: Loss = 0.5387
Epoch 400: Loss = 0.5089
Epoch 500: Loss = 0.4857
Epoch 600: Loss = 0.4675
Epoch 700: Loss = 0.4531
Epoch 800: Loss = 0.4417
Epoch 900: Loss = 0.4326
```

Các bước đã học

Bài tập thực hành Boston Housing

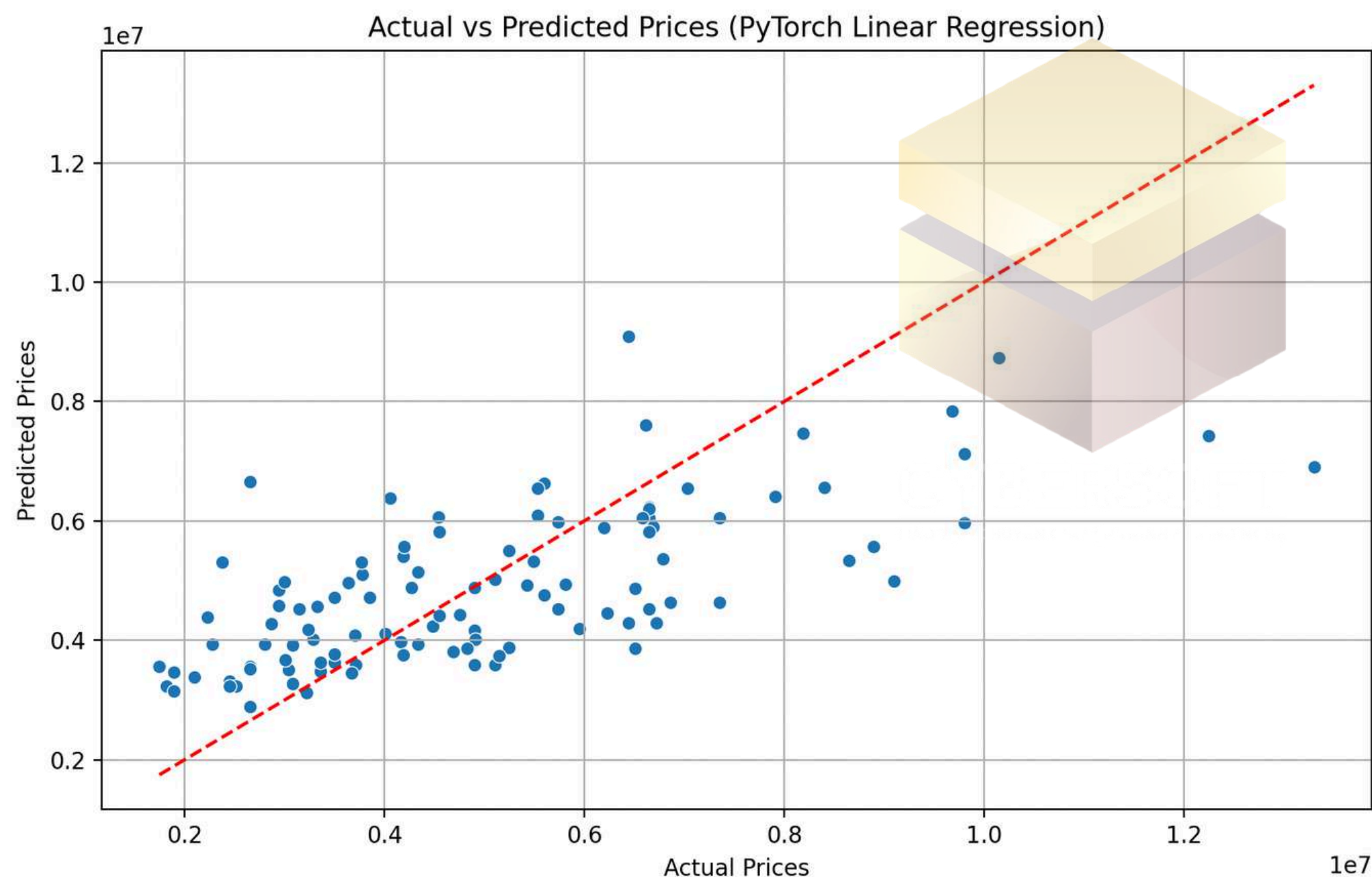
9. Kết quả



Mô hình hồi quy tuyến tính thể hiện xu hướng dự đoán tổng thể đúng, nhưng vẫn còn thiếu chính xác ở các vùng giá trị cao, và dự đoán âm là điểm yếu nghiêm trọng.

Bài tập thực hành Boston Housing

10. Kết quả dự đoán của mô hình



1. Phân bố điểm có xu hướng tăng theo đường chéo
2. → Dù còn phân tán, nhưng nhiều điểm nằm gần đường lý tưởng (đường đỏ) cho thấy mô hình nắm được xu hướng tăng của giá nhà theo đặc trưng.
3. Không bị lệch nghiêm trọng
4. → Không có cụm điểm nào lệch quá xa khỏi đường chéo, tức là mô hình không bị bias mạnh, đặc biệt với dữ liệu phổ biến (giá trung bình).
5. Hiệu quả ở vùng giá phổ biến
6. → Với vùng giá từ ~2 triệu đến ~6 triệu, dự đoán khá gần giá thật → Mô hình hoạt động tốt trong khoảng dữ liệu chính của tập huấn luyện.

Bài tập thực hành Boston Housing

11. Lưu mô hình

Gpu

```
torch.save(model.state_dict(), "linear_regression_model.pkl")
import joblib
joblib.dump(scaler_X, "scaler_X.pkl")
joblib.dump(scaler_y, "scaler_y.pkl")
print("✅ Mô hình và scaler đã được lưu.")
```

Cpu

ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

```
# Lưu mô hình
joblib.dump(model, "linear_model.pkl")
print("✅ Model saved as linear_model.pkl")
```

Code đầy đủ

Cpu



Google Colab

 google.com

CYBERSOFT

ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Gpu

TITLE

LAST MODIFIED



deploy.py

Jun 12



housing.py

Jun 12



linear_regression_model.pth

Jun 12



scaler_X.pkl

Jun 12



scaler_y.pkl

Jun 12

Deploy mô hình lên Streamlit

Các bước thực hiện

Đối với máy sử dụng cpu



Bài tập thực hành Boston Housing

1. Cài đặt thư viện

```
pip install streamlit joblib numpy
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

2. Import thư viện

```
import streamlit as st  
import joblib  
import numpy as np
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

3. Import mô hình

```
# Load mô hình đã lưu  
model = joblib.load("linear_model.pkl")
```

Load mô hình vừa lưu trên colab

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

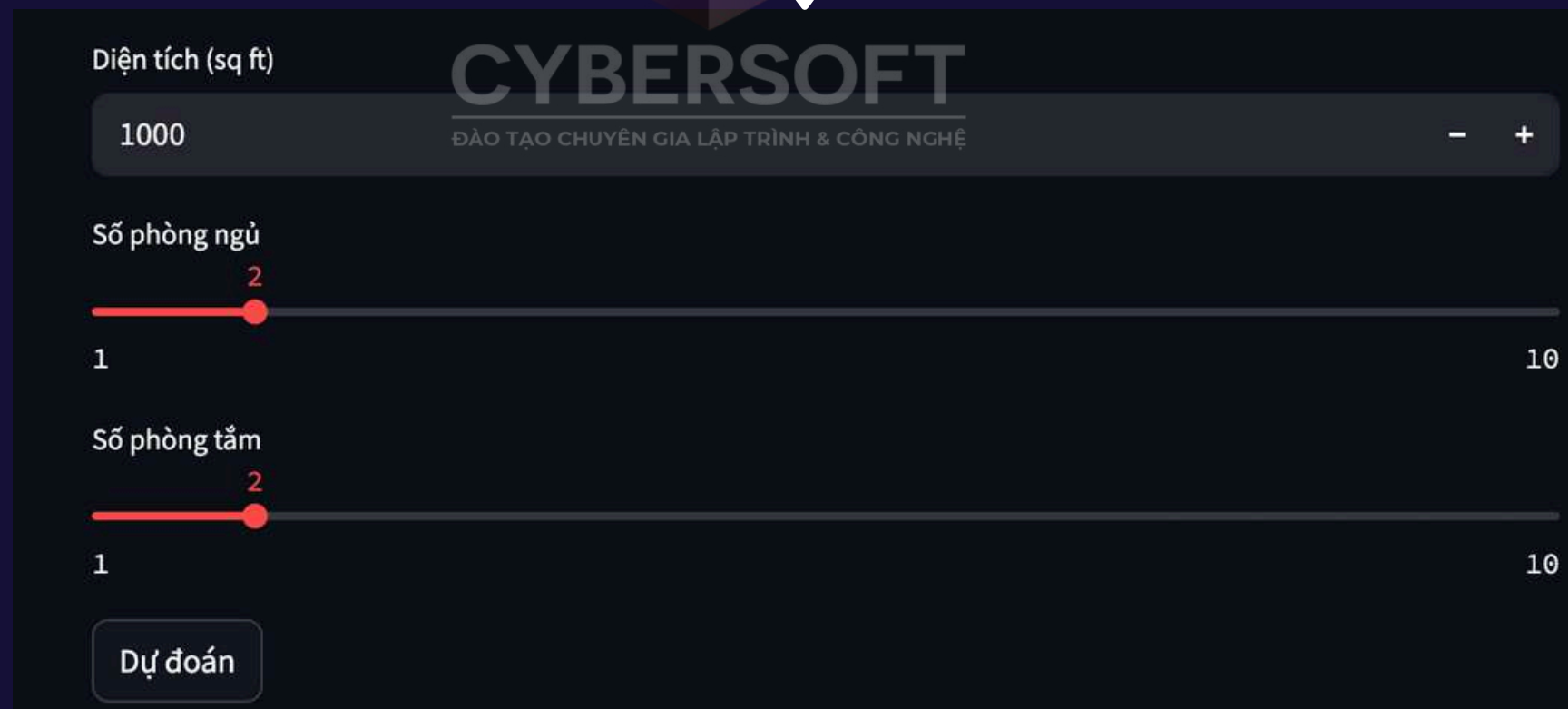
4. Tạo giao diện với Streamlit

```
# Giao diện nhập liệu
st.title("🏠 House Price Prediction App")
st.write("Nhập thông tin ngôi nhà để dự đoán giá:")
```

 **House Price Prediction App**

Nhập thông tin ngôi nhà để dự đoán giá:

```
area = st.number_input("Diện tích (sq ft)", min_value=100, max_value=10000, value=1000)
bedrooms = st.slider("Số phòng ngủ", 1, 10, 2)
bathrooms = st.slider("Số phòng tắm", 1, 10, 2)
```



Diện tích (sq ft)

1000

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Số phòng ngủ

2

1 10

Số phòng tắm

2

1 10

Dự đoán

Bài tập thực hành Boston Housing

5. Khởi chạy bài tập

Tạo mảng đầu vào từ các giá trị người dùng nhập.

```
# Khi nhấn nút "Dự đoán"
if st.button("Dự đoán"):
    input_data = np.array([[area, bedrooms, bathrooms]])
    prediction = model.predict(input_data)[0]
    st.success(f"💰 Giá nhà dự đoán: {prediction:,.0f} VND")
```

Dự đoán giá nhà bằng mô hình đã huấn luyện.

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



House Price Prediction App

Nhập thông tin ngôi nhà để dự đoán giá:

Diện tích (sq ft)

1000

- +

Số phòng ngủ

2

1

10

Số phòng tắm

2

1

10

Dự đoán

💰 Giá nhà dự đoán: 3,969,987 VND

Deploy mô hình lên Streamlit

Các bước thực hiện

Đối với máy sử dụng gpu



Bài tập thực hành Boston Housing

1. Cài đặt thư viện

```
pip install torch streamlit joblib numpy
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

2. Import thư viện

```
import streamlit as st
import torch
import torch.nn as nn
import numpy as np
import joblib
```



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

3. Import mô hình

Khởi tạo trọng số weights là ma trận ngẫu nhiên có kích thước [input_dim, 1]

Khởi tạo hệ số chệch (bias) bằng 0 (điểm đặc biệt của lớp này)

```
class LinearRegressionWithZeroBias(nn.Module):  
    def __init__(self, input_dim):  
        super(LinearRegressionWithZeroBias, self).__init__()  
        self.weights = nn.Parameter(torch.randn(input_dim, 1))  
        self.bias = nn.Parameter(torch.zeros(1))  
  
    def forward(self, x):  
        return x @ self.weights + self.bias
```

Viết lại hàm với Bias ở trên Visual Studio Code

Bài tập thực hành Boston Housing

3. Import mô hình

scaler_X: chuẩn hóa đầu vào (ví dụ: diện tích, phòng ngủ, phòng tắm).
scaler_y: chuẩn hóa đầu ra (giá nhà).

```
# Load scaler và model  
scaler_X = joblib.load("scaler_X.pkl")  
scaler_y = joblib.load("scaler_y.pkl")
```

```
model = LinearRegressionWithZeroBias(input_dim=3)  
model.load_state_dict(torch.load("linear_regression_model.pth", map_location=torch.device("cuda")))  
model.eval()
```

Tạo một instance của mô hình hồi quy tuyến tính với 3 đặc trưng đầu vào

Dùng `map_location=torch.device("cuda")` để load lên GPU

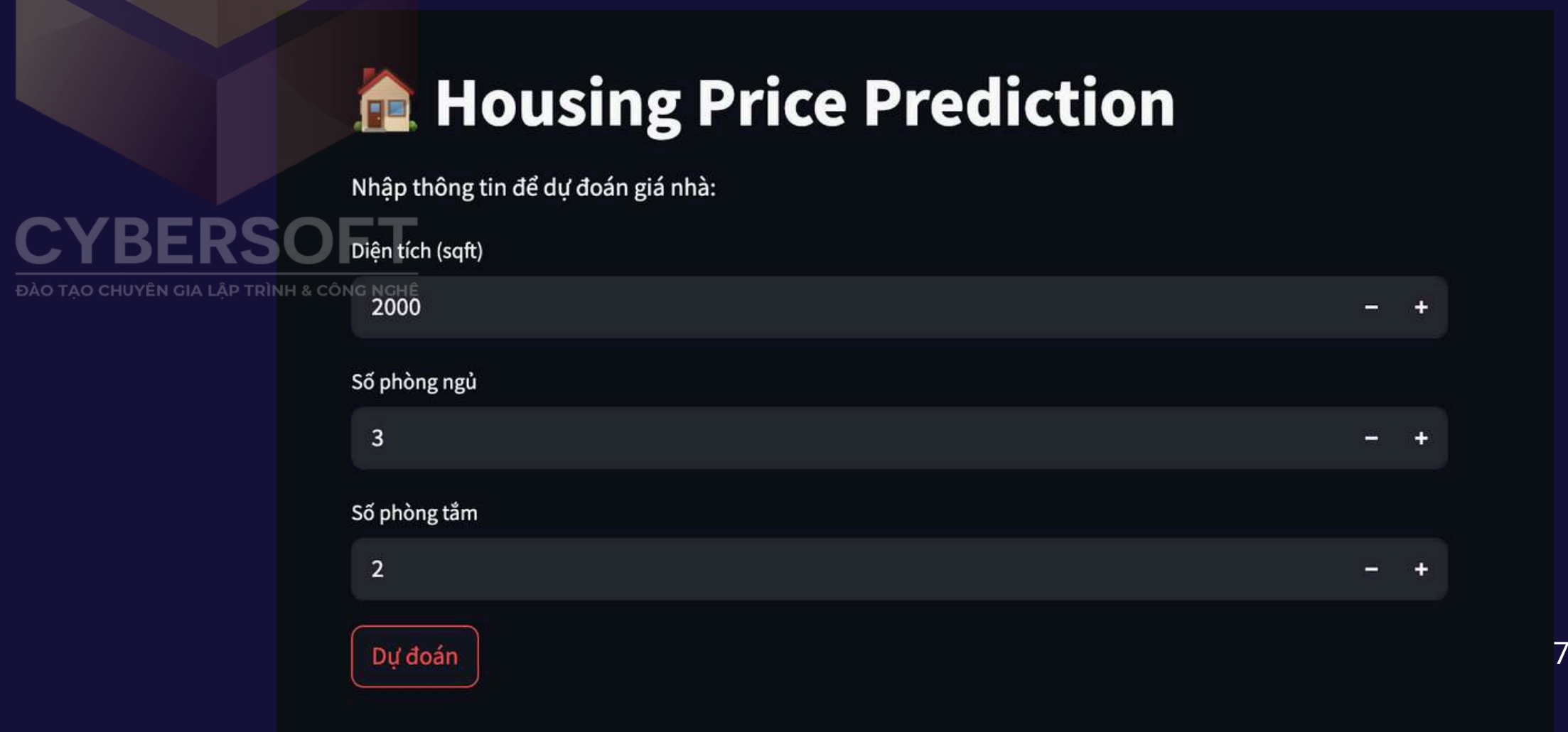
CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

4. Tạo giao diện với Streamlit

```
st.title("🏠 Housing Price Prediction")
st.write("Nhập thông tin để dự đoán giá nhà:")

# Input người dùng
area = st.number_input("Diện tích (sqft)", min_value=100, max_value=10000, value=2000)
bedrooms = st.number_input("Số phòng ngủ", min_value=1, max_value=10, value=3)
bathrooms = st.number_input("Số phòng tắm", min_value=1, max_value=10, value=2)
```



The screenshot shows a web application interface for housing price prediction. It features a title 'Housing Price Prediction' with a house icon, followed by the instruction 'Nhập thông tin để dự đoán giá nhà:'. There are three input fields: 'Diện tích (sqft)' with a value of 2000, 'Số phòng ngủ' with a value of 3, and 'Số phòng tắm' with a value of 2. Each input field has minus and plus buttons for adjustment. At the bottom, there is a red 'Dự đoán' button. A 'CYBERSOFT' watermark is visible on the left side of the interface.

Bài tập thực hành Boston Housing

5. Chuẩn hoá dữ liệu đầu vào

Tạo 1 hàng dữ liệu từ các giá trị area, bedrooms, bathrooms người dùng nhập

```
X_input = np.array([[area, bedrooms, bathrooms]])  
X_scaled = scaler_X.transform(X_input)  
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
```

Áp dụng chuẩn hóa (StandardScaler, MinMaxScaler, ...) đã được huấn luyện trước.
=> Giúp đầu vào nằm trong khoảng giá trị phù hợp với mô hình.

Chuyển từ numpy array → PyTorch tensor để đưa vào mô hình.

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập thực hành Boston Housing

6. Khởi chạy bài tập

```
with torch.no_grad():  
    y_scaled_pred = model(X_tensor)  
    y_pred = scaler_y.inverse_transform([y_scaled_pred.numpy()])  
  
st.success(f"💰 Giá nhà dự đoán: ${y_pred[0][0]:,.2f}")
```

streamlit run deploy.py

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Housing Price Prediction

Nhập thông tin để dự đoán giá nhà:

Diện tích (sqft)

2000

Số phòng ngủ

3

Số phòng tắm

2

Dự đoán

💰 Giá nhà dự đoán: \$4,675,671.00

Dự án trên lớp

- 📌 Dự án: Dự đoán điểm số sinh viên bằng Hồi quy tuyến tính.
- 📌 Chức năng chính: Phân tích dữ liệu giáo dục, tìm mối quan hệ giữa biến đầu vào và điểm số.
- 📌 Foundation: Ma trận, đạo hàm, tối ưu hóa hàm mất mát.
- 📌 Thuật toán: Hồi quy tuyến tính, Hồi quy logistic
- 📌 Thư viện: Scikit-learn (LinearRegression), Matplotlib.

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

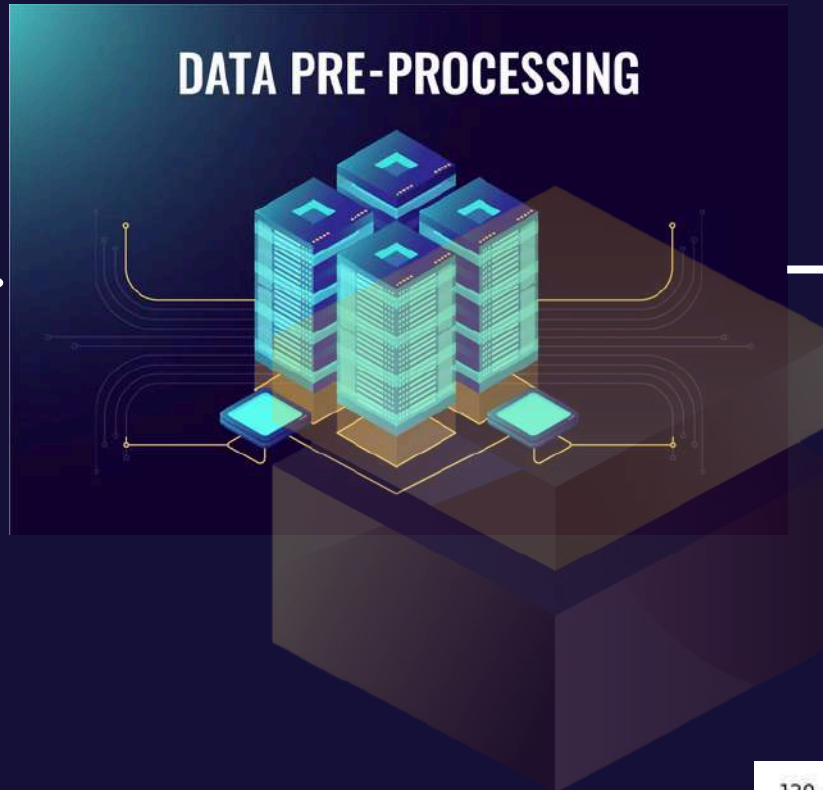
Flow dự án

Dataset

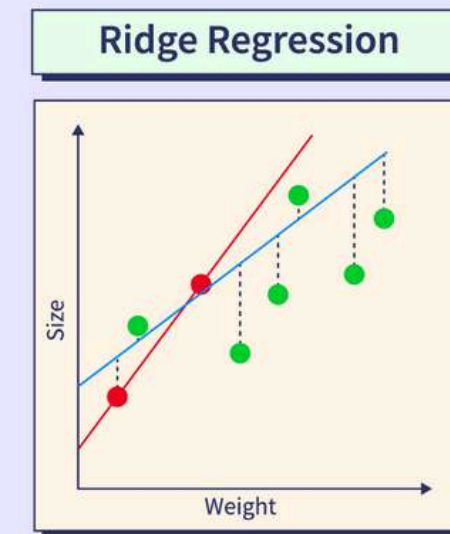
	Hours_Studied	Sleep_Hours	Extracurriculars	Health_Index	P
0	4.370861	9.556429	7.587945	6.387926	
1	1.185260	9.729189	8.491984	2.911052	
2	6.506676	2.255445	3.629302	4.297257	
3	6.467904	2.534717	1.585464	9.539970	
4	2.098344	5.456592	1.309497	9.183884	

Tiền xử lí dữ liệu

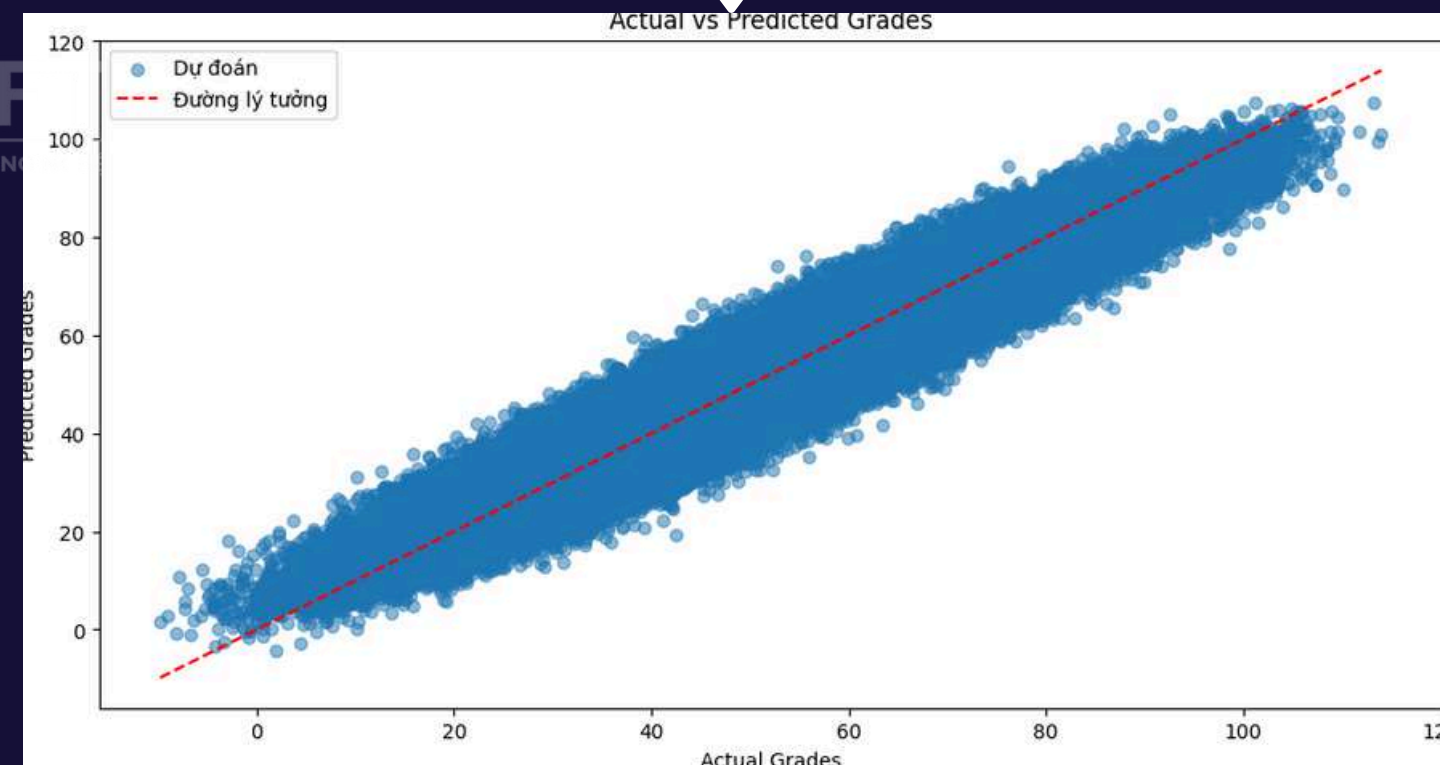
DATA PRE-PROCESSING



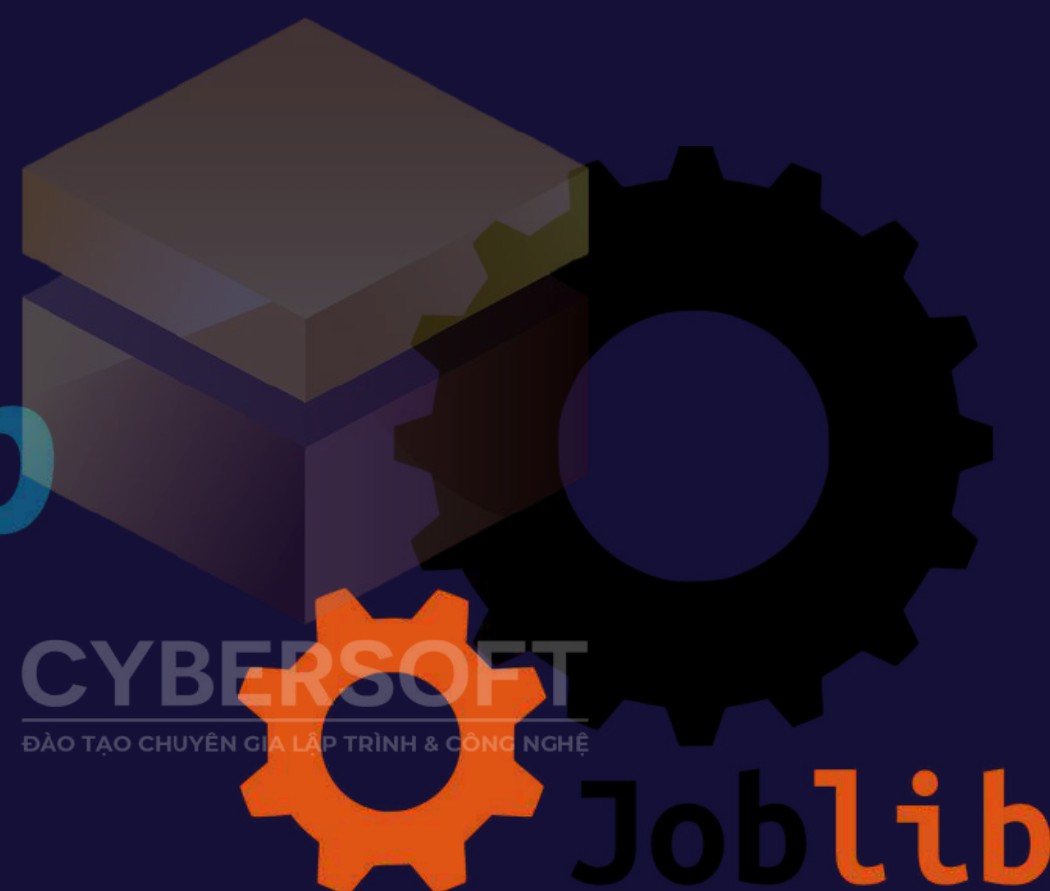
Ridge Regression



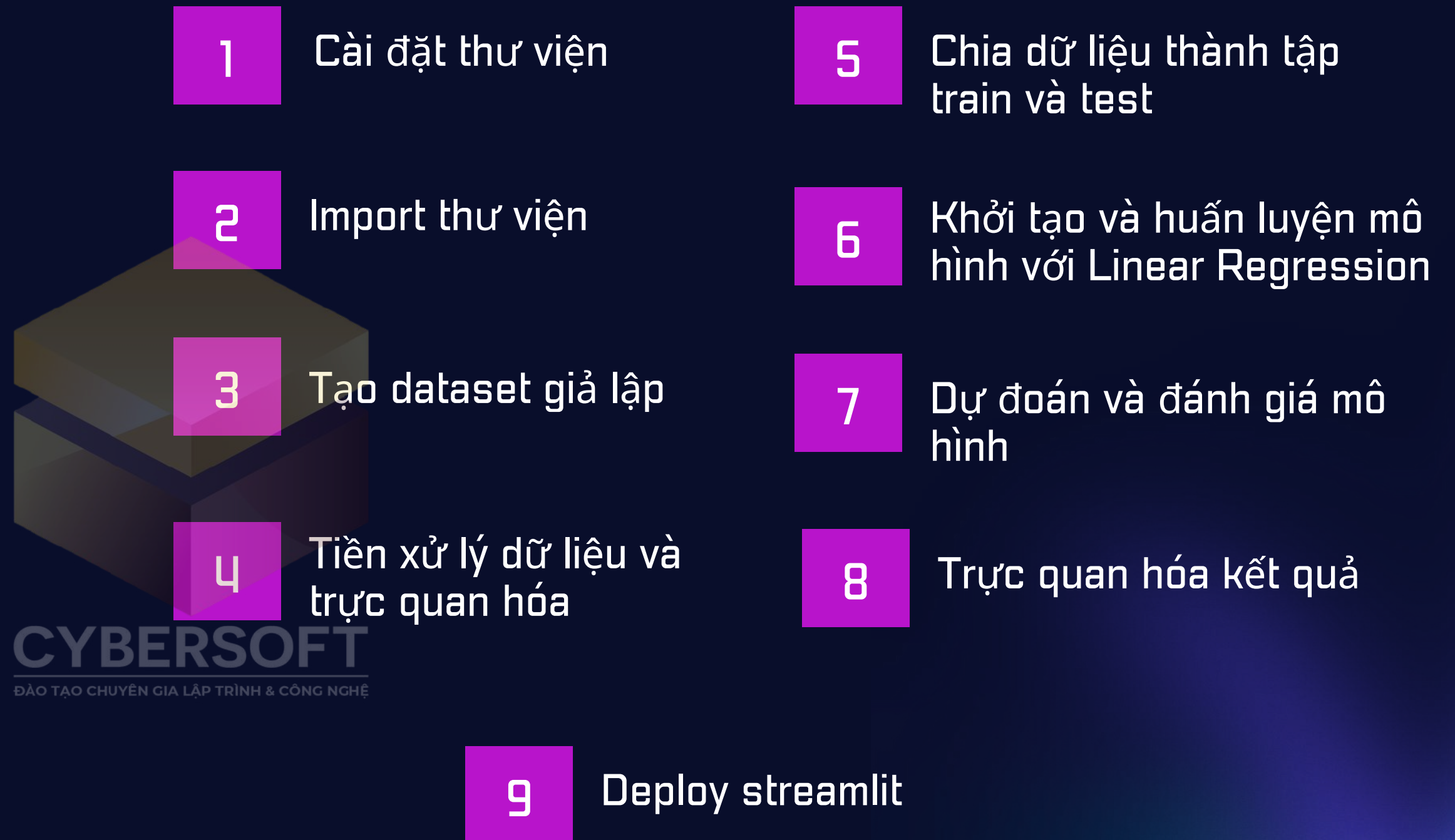
Trực quan hoá kết quả



Thư viện áp dụng



Các bước thực hiện



Dự án: dự đoán điểm số sinh viên

1. Cài đặt thư viện

```
pip install numpy==1.24.3 pandas matplotlib seaborn joblib scikit-learn
```



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Dự án: dự đoán điểm số sinh viên

2. Import thư viện

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Dự án: dự đoán điểm số sinh viên

3. Tạo dataset giả lập

```
# 2. Tạo dataset giả lập
np.random.seed(42)
n_samples = 1000000
X_data = np.random.uniform(1, 10, (n_samples, 10)) # 10 đặc trưng ảnh hưởng đến điểm số

# Định nghĩa các cột đặc trưng
columns = [
    'Hours_Studied',      # Số giờ học
    'Sleep_Hours',        # Số giờ ngủ trung bình
    'Extracurriculars',   # Hoạt động ngoại khóa (số lượng)
    'Health_Index',       # Chỉ số sức khỏe (từ 1-10)
    'Parental_Involvement', # Sự hỗ trợ của phụ huynh (từ 1-10)
    'Tutoring_Hours',     # Số giờ học thêm
    'Internet_Usage',     # Số giờ sử dụng internet
    'Self_Study',         # Số giờ tự học
    'Attendance',         # Tỷ lệ đi học (%)
    'Motivation_Level'    # Mức độ động lực học tập (từ 1-10)
]

data = pd.DataFrame(X_data, columns=columns)

data['Grades'] = (
    5 * data['Hours_Studied'] +
    3 * data['Self_Study'] +
    2 * data['Parental_Involvement'] +
    1.5 * data['Tutoring_Hours'] -
    2 * data['Internet_Usage'] +
    np.random.normal(0, 5, n_samples) # Công thức điểm + nhiễu
)

data.head()
```

→ Số lượng dòng dữ liệu được tạo ra

Định nghĩa & Tạo Dataset:

- Mã Python định nghĩa các cột đặc trưng như Hours_Studied, Sleep_Hours, Motivation_Level, v.v.
- Sau đó tạo DataFrame data và tính điểm (Grades) dựa trên công thức kết hợp nhiều yếu tố, có thêm nhiễu (np.random.normal).

```
# Lưu dataset
data.to_csv('student_scores.csv', index=False)
print("Dataset đã được lưu thành student_scores.csv")
```


Dự án: dự đoán điểm số sinh viên

3. Tạo dataset giả lập

	Hours_Studied	Sleep_Hours	Extracurriculars	Health_Index	Parental_Involvement	Tutoring_Hours	Internet_Usage	Self_Study	Attendance	Motivation_Level	Grades
0	4.370861	9.556429	7.587945	6.387926	2.404168	2.403951	1.522753	8.795585	6.410035	7.372653	52.680718
1	1.185260	9.729189	8.491984	2.911052	2.636425	2.650641	3.738180	5.722808	4.887505	3.621062	28.238799
2	6.506676	2.255445	3.629302	4.297257	5.104630	8.066584	2.797064	5.628110	6.331731	1.418054	67.915446
3	6.467904	2.534717	1.585464	9.539970	9.690688	8.275576	3.741524	1.879049	7.158097	4.961372	64.821061
4	2.098344	5.456592	1.309497	9.183884	3.329020	6.962701	3.805400	5.680612	5.920393	2.663690	35.421047

Bảng dữ liệu mẫu:

- Hiển thị 5 dòng đầu tiên của dataset đã tạo.
- Mỗi dòng là 1 học sinh với các đặc trưng như: số giờ học, điểm sức khỏe, động lực học tập,... và điểm tổng kết Grades.

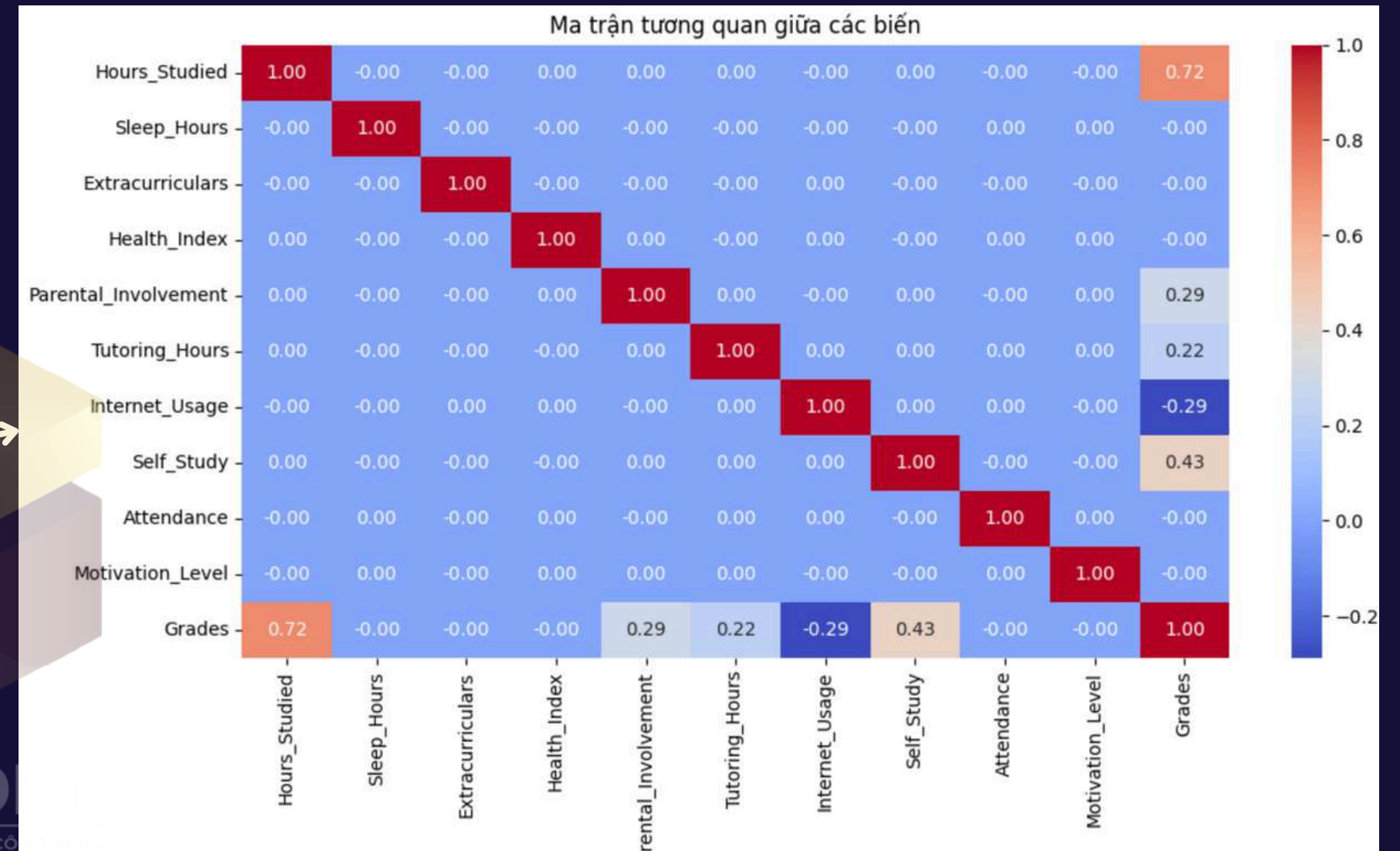
Dự án: dự đoán điểm số sinh viên

4. Tiền xử lý dữ liệu và trực quan hóa

```
# Hiển thị thông tin dataset
print("Thông tin dataset:")
print(data.info())
print("\nMô tả thống kê dataset:")
print(data.describe())
print("\nDữ liệu mẫu:")
print(data.head())

# Trực quan hóa dữ liệu
plt.figure(figsize=(12, 6))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Ma trận tương quan giữa các biến")
plt.show()

# Phân phối điểm số
plt.figure(figsize=(12, 6))
sns.histplot(data['Grades'], bins=50, kde=True)
plt.xlabel("Grades")
plt.ylabel("Tần suất")
plt.title("Phân phối điểm số sinh viên")
plt.show()
```



Ma trận tương quan:

- Biểu đồ heatmap thể hiện mức độ tương quan giữa các đặc trưng.
- Ví dụ: Hours_Studied có tương quan dương mạnh với Grades, trong khi Internet_Usage có tương quan âm.

Dự án: dự đoán điểm số sinh viên

4. Tiền xử lý dữ liệu và trực quan hóa

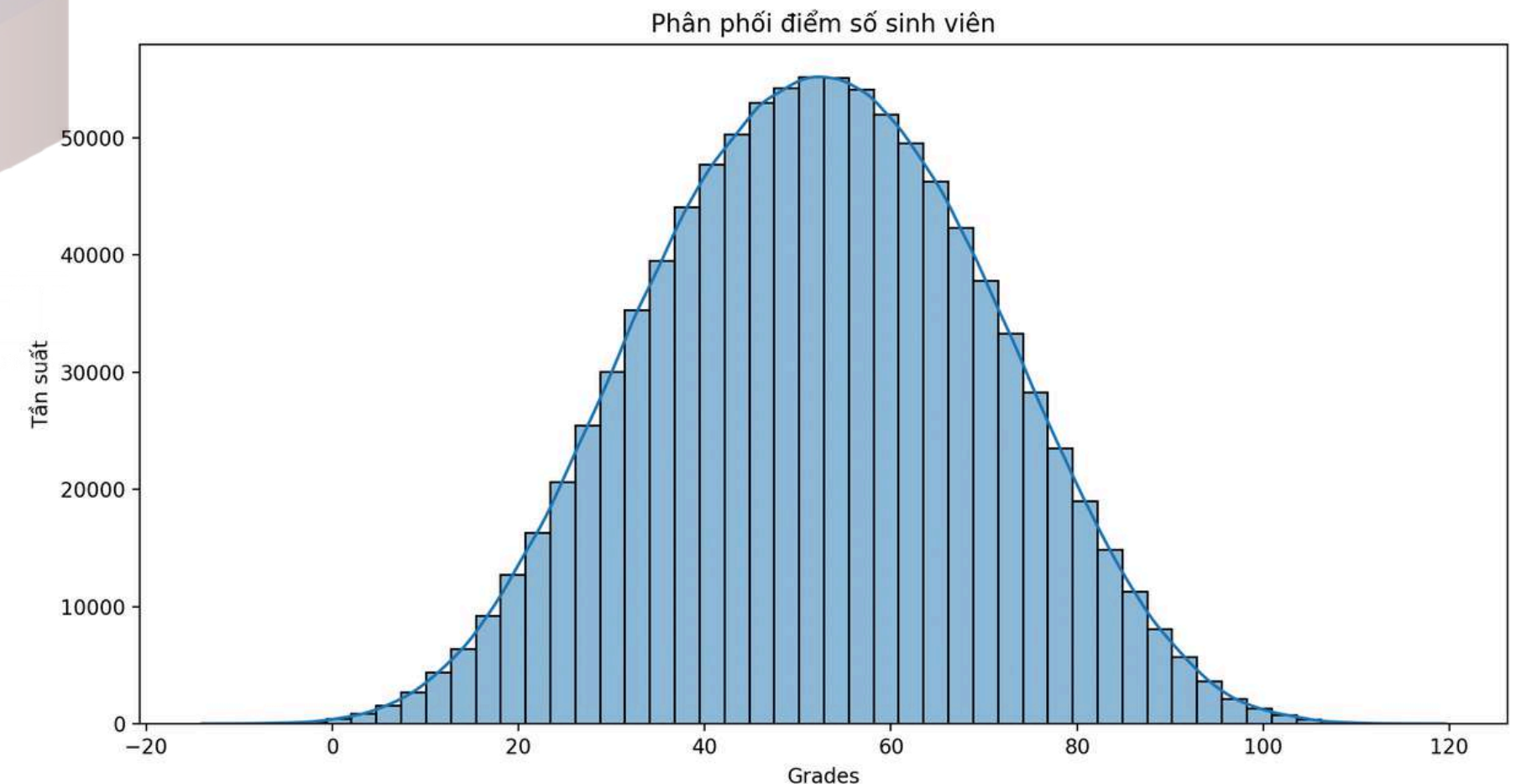
```
# Hiển thị thông tin dataset
print("Thông tin dataset:")
print(data.info())
print("\nMô tả thống kê dataset:")
print(data.describe())
print("\nDữ liệu mẫu:")
print(data.head())
```

```
# Trực quan hóa dữ liệu
plt.figure(figsize=(12, 6))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Ma trận tương quan giữa các biến")
plt.show()
```

```
# Phân phối điểm số
plt.figure(figsize=(12, 6))
sns.histplot(data['Grades'], bins=50, kde=True)
plt.xlabel("Grades")
plt.ylabel("Tần suất")
plt.title("Phân phối điểm số sinh viên")
plt.show()
```

Biểu đồ histogram:

- Biểu đồ có dạng hình chuông, cho thấy phân phối gần giống với phân phối chuẩn (normal distribution), với đỉnh cao nhất khoảng 50-60 điểm, nghĩa là phần lớn sinh viên có điểm số trong khoảng này.
- Đường cong mượt mà trên đỉnh histogram có thể là một đường ước lượng phân phối chuẩn.



Dự án: dự đoán điểm số sinh viên

5. Chia dữ liệu thành tập train và test

```
X = data.drop(columns=['Grades'])  
y = data['Grades']  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Chia dữ liệu ban đầu thành tập huấn luyện (để dạy mô hình) và tập kiểm tra (để đánh giá mô hình) bằng cách tách các biến đầu vào (X) và biến mục tiêu (y).

Dự án: dự đoán điểm số sinh viên

6. Khởi tạo và huấn luyện mô hình với Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)

# Lưu mô hình
joblib.dump(model, 'student_score_model.pkl')
print("Mô hình đã được lưu thành student_score_model.pkl")
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Dự án: dự đoán điểm số sinh viên

7. Dự đoán và đánh giá mô hình

```
# Dự đoán  
y_pred = model.predict(X_test)
```

```
# Đánh giá mô hình  
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
r2 = r2_score(y_test, y_pred)  
print(f"MAE: {mae:.2f}, MSE: {mse:.2f}, RMSE: {rmse:.2f}, R2 Score: {r2:.2f}")
```

MAE: 4.00, MSE: 25.13, RMSE: 5.01, R2 Score: 0.92

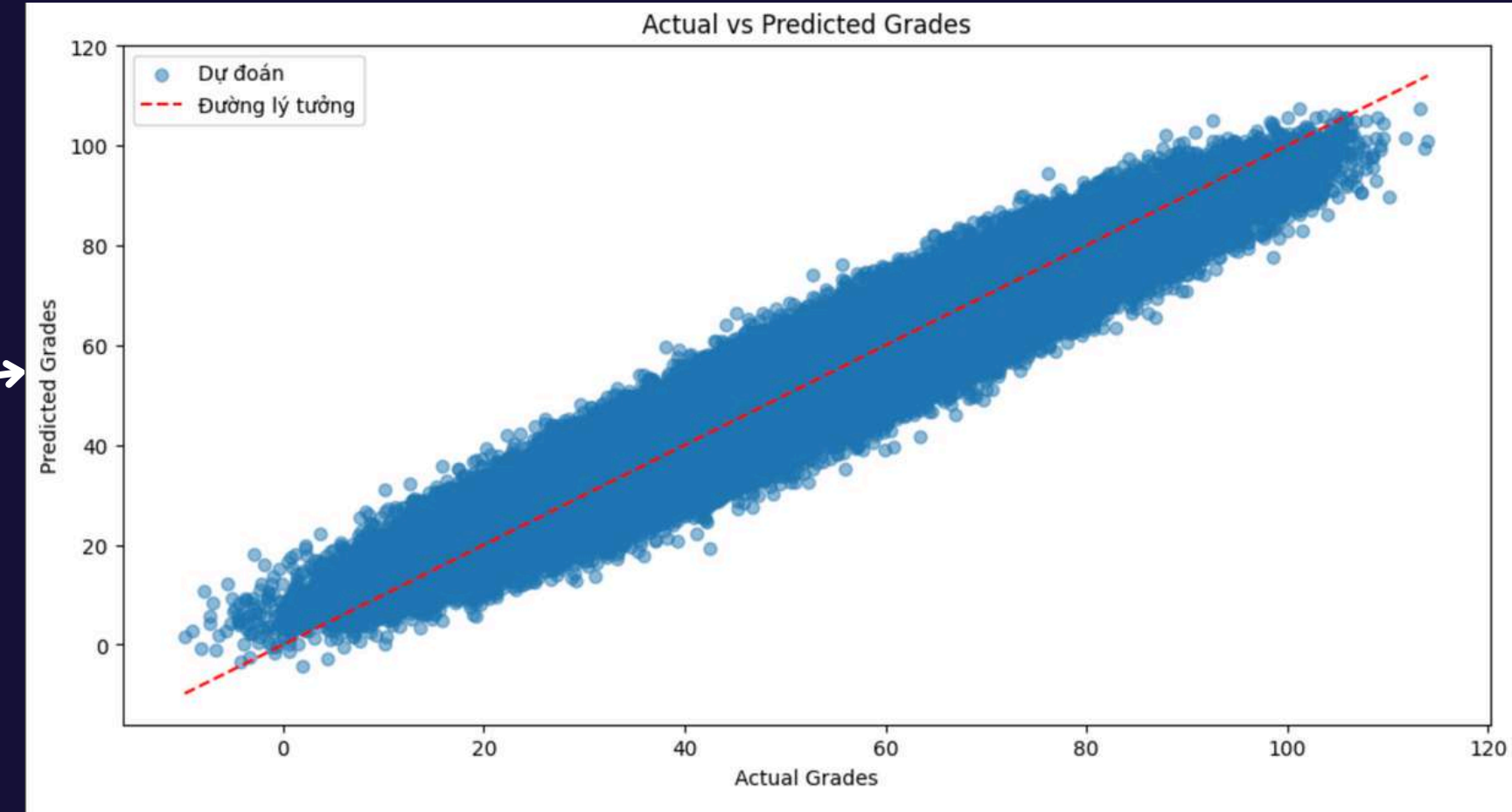
Dự án: dự đoán điểm số sinh viên

8. Trực quan hóa kết quả

```
plt.figure(figsize=(12, 6))
plt.scatter(y_test, y_pred, alpha=0.5, label="Dự đoán")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label="Đường lý tưởng")
plt.xlabel('Actual Grades')
plt.ylabel('Predicted Grades')
plt.title('Actual vs Predicted Grades')
plt.legend()
plt.show()

# Biểu đồ phân phối sai số
errors = y_test - y_pred
plt.figure(figsize=(12, 6))
sns.histplot(errors, bins=50, kde=True)
plt.xlabel("Sai số dự đoán")
plt.ylabel("Tần suất")
plt.title("Phân phối sai số dự đoán")
plt.show()
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



Actual vs Predicted Grades

- Trục X: Giá trị thực tế (Actual Grades).
- Trục Y: Giá trị dự đoán (Predicted Grades).
- Điểm xanh: Các điểm dữ liệu (dự đoán vs. thực tế).
- Đường đỏ: Đường lý tưởng (dự đoán = thực tế).
- 🖱 Ý nghĩa: Dự đoán gần khớp tốt với thực tế nếu các điểm nằm gần đường đỏ.

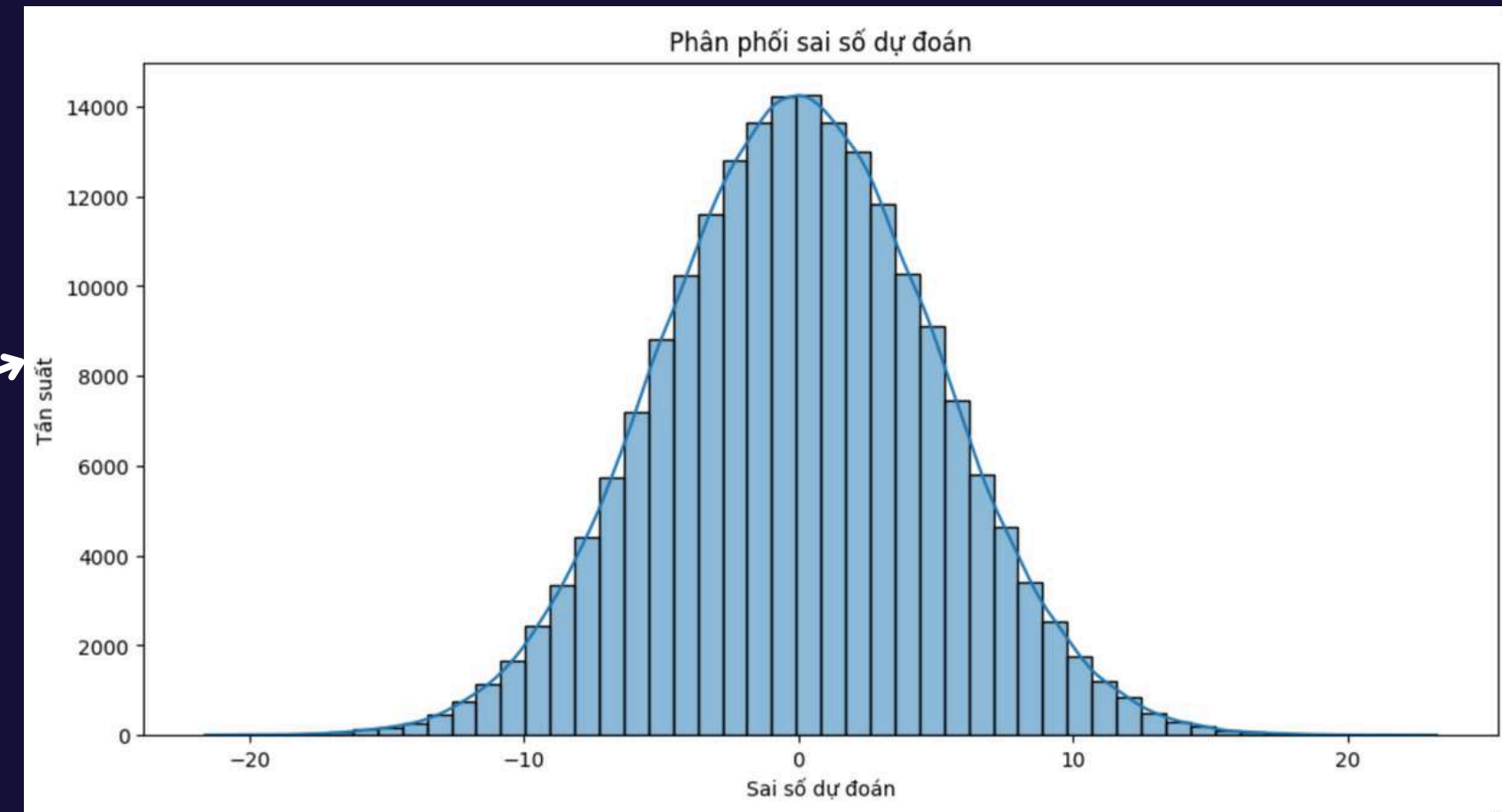
Dự án: dự đoán điểm số sinh viên

8. Trực quan hóa kết quả

```
plt.figure(figsize=(12, 6))
plt.scatter(y_test, y_pred, alpha=0.5, label="Dự đoán")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label="Đường lý tưởng")
plt.xlabel('Actual Grades')
plt.ylabel('Predicted Grades')
plt.title('Actual vs Predicted Grades')
plt.legend()
plt.show()

# Biểu đồ phân phối sai số
errors = y_test - y_pred
plt.figure(figsize=(12, 6))
sns.histplot(errors, bins=50, kde=True)
plt.xlabel("Sai số dự đoán")
plt.ylabel("Tần suất")
plt.title("Phân phối sai số dự đoán")
plt.show()
```

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ



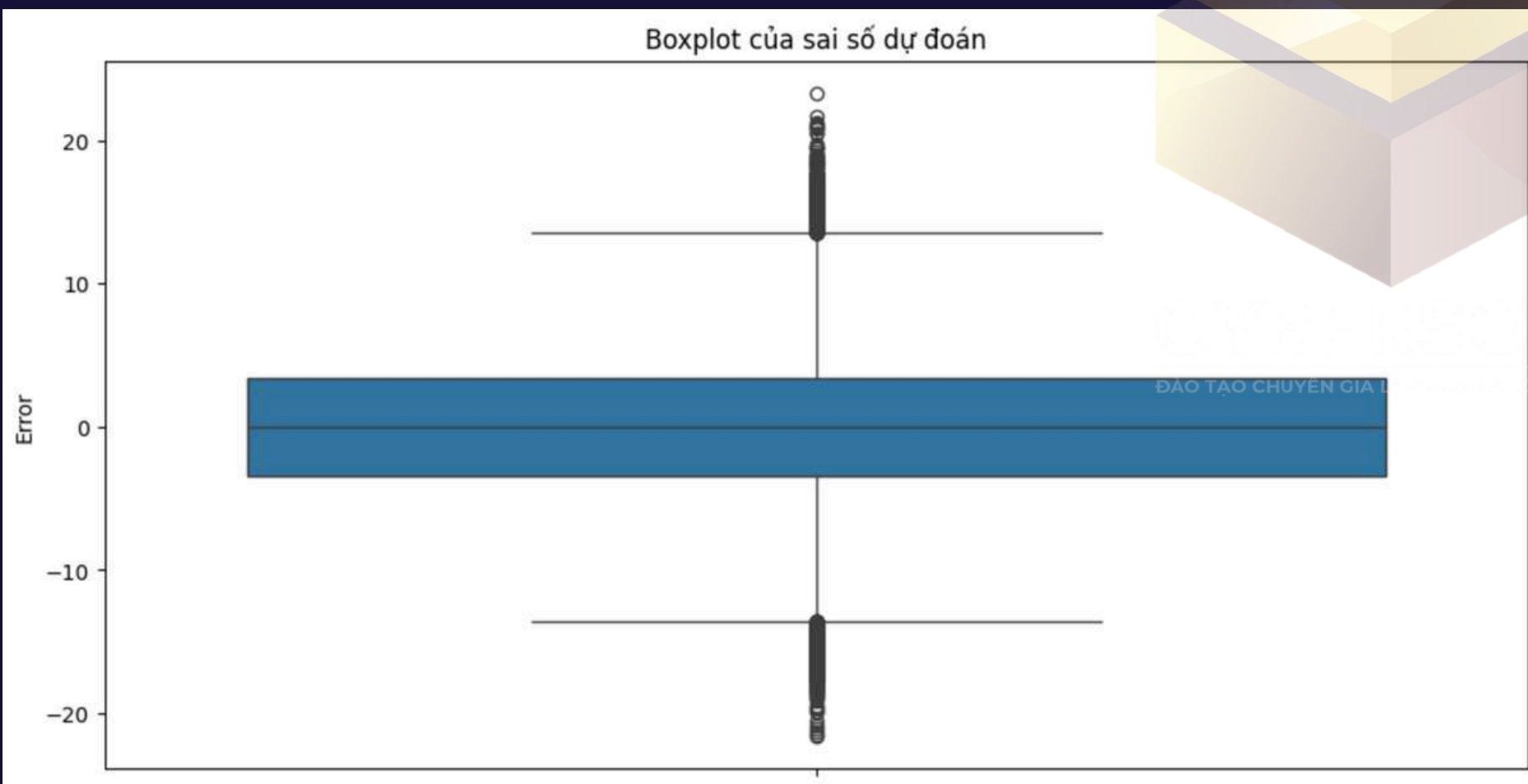
Phân phối sai số dự đoán

- Trục X: Sai số dự đoán (Predicted – Actual).
- Trục Y: Tần suất xuất hiện của sai số đó.
- Dạng chuông đối xứng → sai số có phân phối chuẩn.
- 🖱️ Ý nghĩa: Mô hình có sai số trung bình gần 0, phân bố ổn định, không lệch mạnh.

Dự án: dự đoán điểm số sinh viên

8. Trực quan hóa kết quả

```
# Biểu đồ hộp cho thấy mức độ phân tán lỗi
data_error = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred, 'Error': errors})
plt.figure(figsize=(12, 6))
sns.boxplot(y=data_error['Error'])
plt.title("Boxplot của sai số dự đoán")
plt.show()
```



	Actual	Predicted
0	52.680718	53.605161
1	28.238799	24.871910
2	67.915446	66.140625
3	64.821061	62.313843
4	35.421047	37.022660
..	...	...
95	48.811146	41.163174
96	21.817568	18.300306
97	25.627742	34.134853
98	43.621599	43.694292
99	36.701060	40.402197

Boxplot của sai số dự đoán

- Trục Y: Sai số dự đoán ($\text{Error} = \text{Predicted} - \text{Actual}$).
- Hộp xanh:
 - Phần giữa: 50% sai số trung vị ($Q1 \rightarrow Q3$).
 - Đường giữa hộp: sai số trung vị ($\approx 0 \rightarrow$ không lệch trái/phải).
- Râu (whiskers): Khoảng biến thiên chính (không tính ngoại lệ).
- Chấm tròn ngoài râu: Outliers – các điểm sai số quá lớn hoặc quá nhỏ.

➡ Ý nghĩa:

- Mô hình có phân phối sai số khá cân đối quanh 0.
- Tuy nhiên vẫn tồn tại một số điểm ngoại lệ, cần kiểm tra kỹ dữ liệu hoặc mô hình nếu nhiều.

Dự án: dự đoán điểm số sinh viên

9. Deploy streamlit

Import thư viện

Load mô hình đã lưu

Dự đoán, tính sai số dự đoán

```
import joblib
import numpy as np
import pandas as pd
import streamlit as st
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def load_model(model_path='student_score_model.pkl'):
    """Load mô hình đã lưu"""
    return joblib.load(model_path)

def evaluate_model(model, X_test, y_test):
    """Đánh giá mô hình trên dữ liệu test"""
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    return mae, mse, rmse, r2
```

Dự án: dự đoán điểm số sinh viên

9. Deploy streamlit

Định nghĩa hàm chính của ứng dụng

```
def main():
    st.title("Dự đoán và Đánh giá điểm số học sinh")
    model = load_model()

    columns = [
        'Hours_Studied',      # Số giờ học
        'Sleep_Hours',        # Số giờ ngủ trung bình
        'Extracurriculars',   # Hoạt động ngoại khóa (số lượng)
        'Health_Index',       # Chỉ số sức khỏe (từ 1-10)
        'Parental_Involvement', # Sự hỗ trợ của phụ huynh (từ 1-10)
        'Tutoring_Hours',     # Số giờ học thêm
        'Internet_Usage',     # Số giờ sử dụng internet
        'Self_Study',         # Số giờ tự học
        'Attendance',         # Tỷ lệ đi học (%)
        'Motivation_Level'    # Mức độ động lực học tập (từ 1-10)
    ]

    st.subheader("Nhập dữ liệu đầu vào:")
    user_data = [st.number_input(col, min_value=0.0, step=0.1) for col in columns]

    # Tạo nút "Dự đoán" và kiểm tra xem người dùng có nhấn nút không
    if st.button("Dự đoán"):
        user_input = np.array(user_data).reshape(1, -1)
        predicted_grade = model.predict(user_input)[0]
        st.success(f"Dự đoán điểm số của học sinh: {predicted_grade:.2f}")

    st.subheader("Tải lên dataset để đánh giá mô hình")
    uploaded_file = st.file_uploader("Chọn file CSV", type=["csv"])
```


Dự án: dự đoán điểm số sinh viên

9. Deploy streamlit

streamlit run deploy.py

Xử lý dữ liệu từ file mô hình để hiển thị lên giao diện

```
if uploaded_file is not None:
    data = pd.read_csv(uploaded_file) # Đọc file CSV thành DataFrame
    st.write("Dữ liệu mẫu:")
    st.write(data.head())

    if 'Grades' in data.columns:
        X_test = data[colums] # Lấy các cột đặc trưng từ DataFrame
        y_test = data['Grades'] # Lấy cột 'Grades' làm giá trị thực tế
        mae, mse, rmse, r2 = evaluate_model(model, X_test, y_test) # Gọi hàm đánh giá mô hình

        st.subheader("Đánh giá mô hình:") # Hiển thị tiêu đề phụ cho kết quả đánh giá
        st.write(f"MAE: {mae:.2f}") # Hiển thị giá trị MAE với 2 chữ số thập phân
        st.write(f"MSE: {mse:.2f}") # Hiển thị giá trị MSE với 2 chữ số thập phân
        st.write(f"RMSE: {rmse:.2f}") # Hiển thị giá trị RMSE với 2 chữ số thập phân
        st.write(f"R2 Score: {r2:.2f}") # Hiển thị giá trị R2 với 2 chữ số thập phân
    else: # Nếu không có cột 'Grades'
        st.error("Dataset không chứa cột 'Grades'. Hãy đảm bảo dữ liệu đầu vào đúng định dạng.")

if __name__ == "__main__":
    main() # Gọi hàm main để chạy ứng dụng
```

Dự đoán và Đánh giá điểm số học sinh

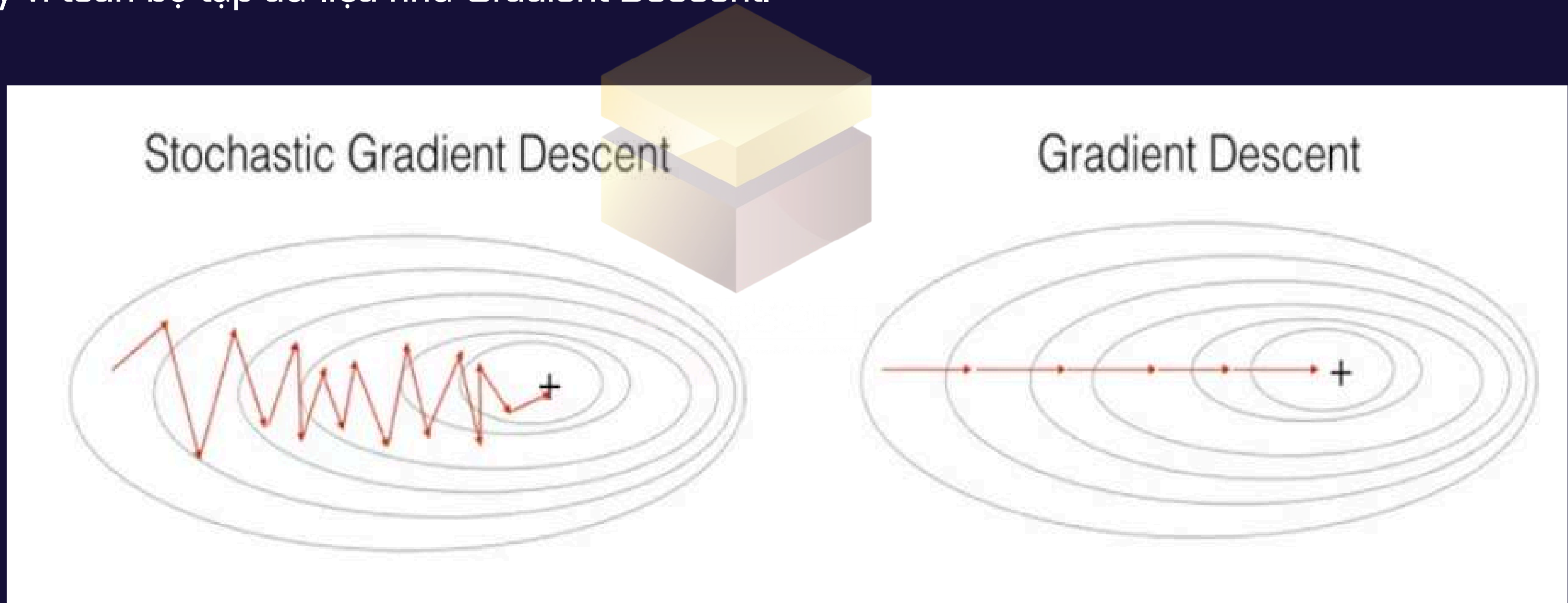
Nhập dữ liệu đầu vào:

Hours_Studied	0,00	-	+
Sleep_Hours	0,00	-	+
Extracurriculars	0,00	-	+
Health_Index	0,00	-	+
Parental_Involvement	0,00	-	+
Tutoring_Hours	0,00	-	+
Internet_Usage	0,00	-	+
Self_Study	0,00	-	+
Attendance	0,00	-	+
Motivation_Level	0,00	-	+
Dự đoán			

Kiến thức biết thêm

Stochastic Gradient Descent (SGD)

Gradient được tính chỉ dựa trên một mẫu dữ liệu duy nhất (hoặc một nhóm nhỏ – mini-batch), thay vì toàn bộ tập dữ liệu như Gradient Descent.



Quy Trình Stochastic Gradient Descent

Khởi tạo tham số
ban đầu w

Tính gradient

Giá trị tham số tối ưu



Logistic Regression



Trải qua Linear Regression chúng ta có thể dự đoán các bài toán như

- Thời gian đến đích
- Dự đoán giá nhà
- Dự đoán doanh thu bán hàng
- Dự đoán điểm số học sinh

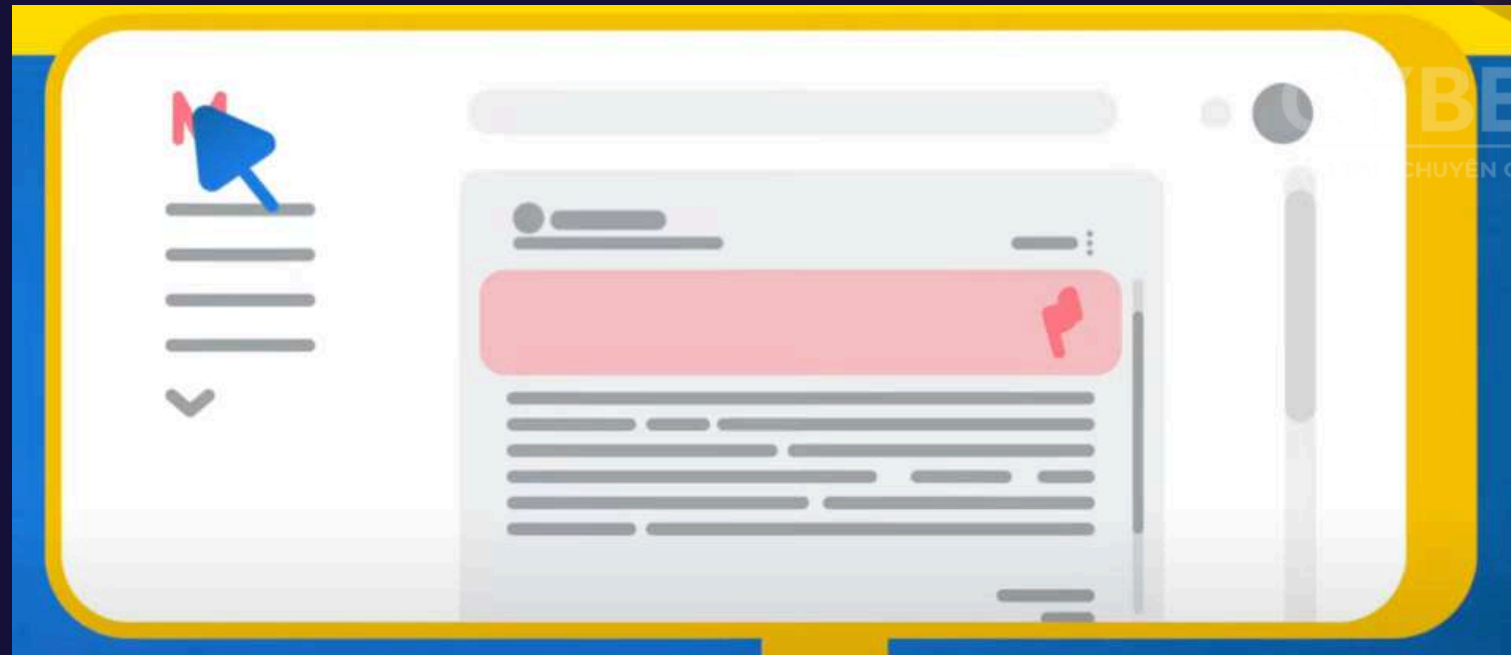


Còn các bài toán như

➤ Dự đoán thời tiết



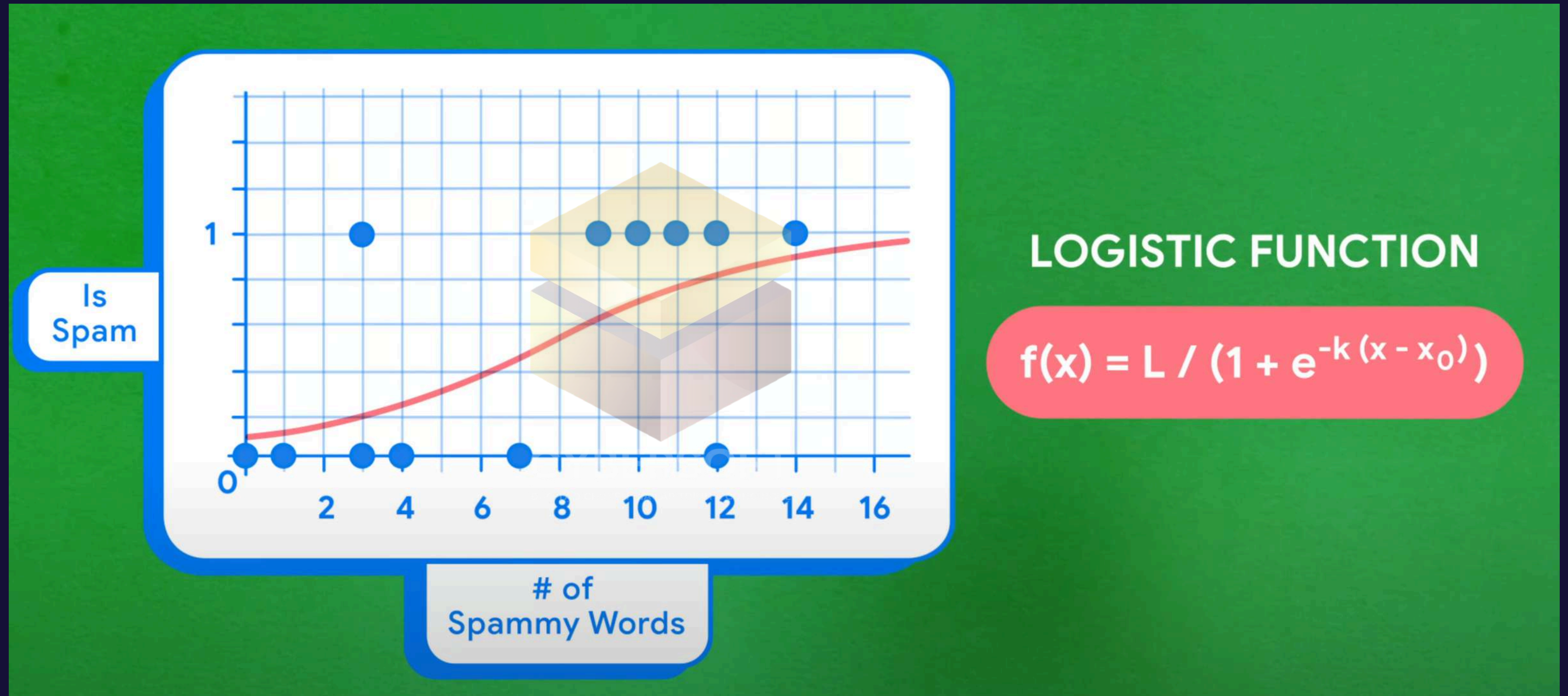
➤ Dự đoán học sinh thi đỗ



➤ Dự đoán email là spam

Thì làm sao chúng ta có thể dự đoán được ????

Logistic Regression



Logistic Regression

Giả sử ta có một nhóm 10 sinh viên dành thời gian trong khoảng từ 0 đến 6 giờ cho việc ôn thi. Thời gian ôn thi này ảnh hưởng đến xác suất sinh viên vượt qua kỳ thi như thế nào?

#Hours

0.5

.75

1.75

4.25

3.5

2

4.75

5.5

2.75

1

#Pass

0

0

1

1

1

0

1

1

1

0

CYBERSOFT

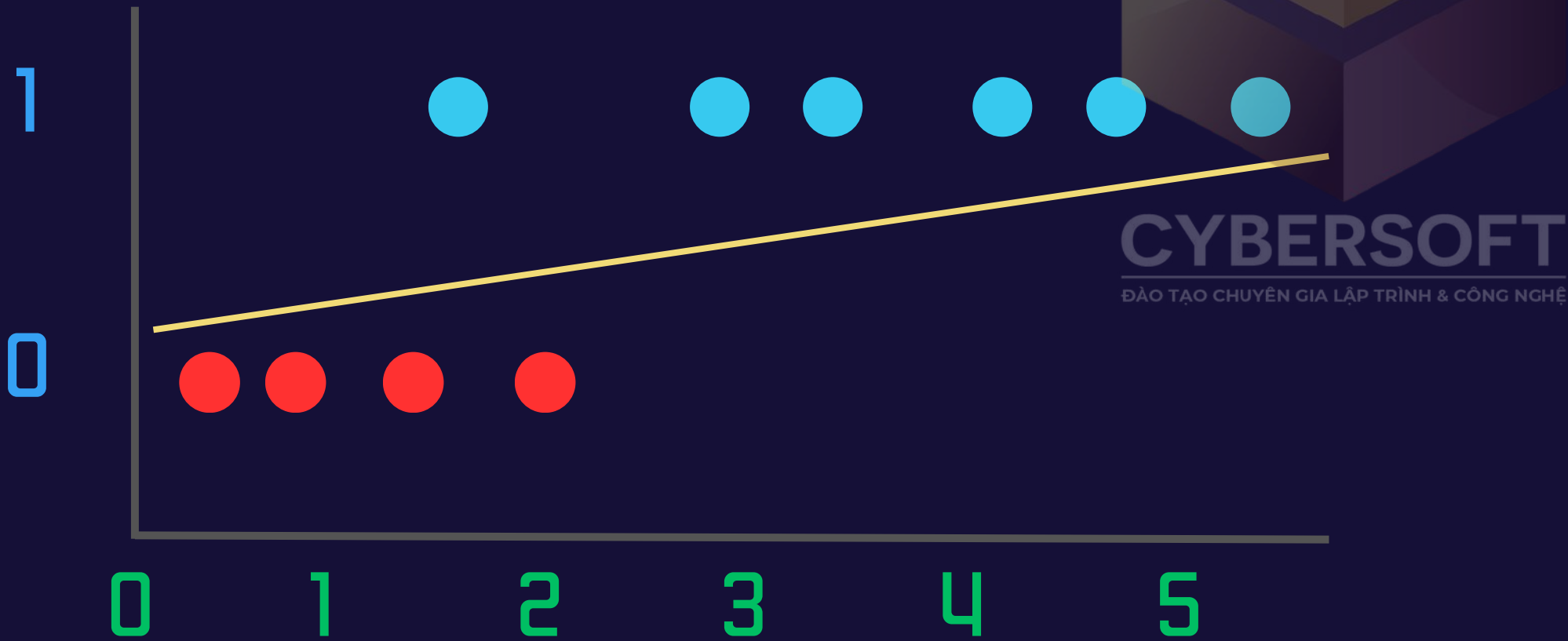
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Logistic Regression

#Hours

#Pass

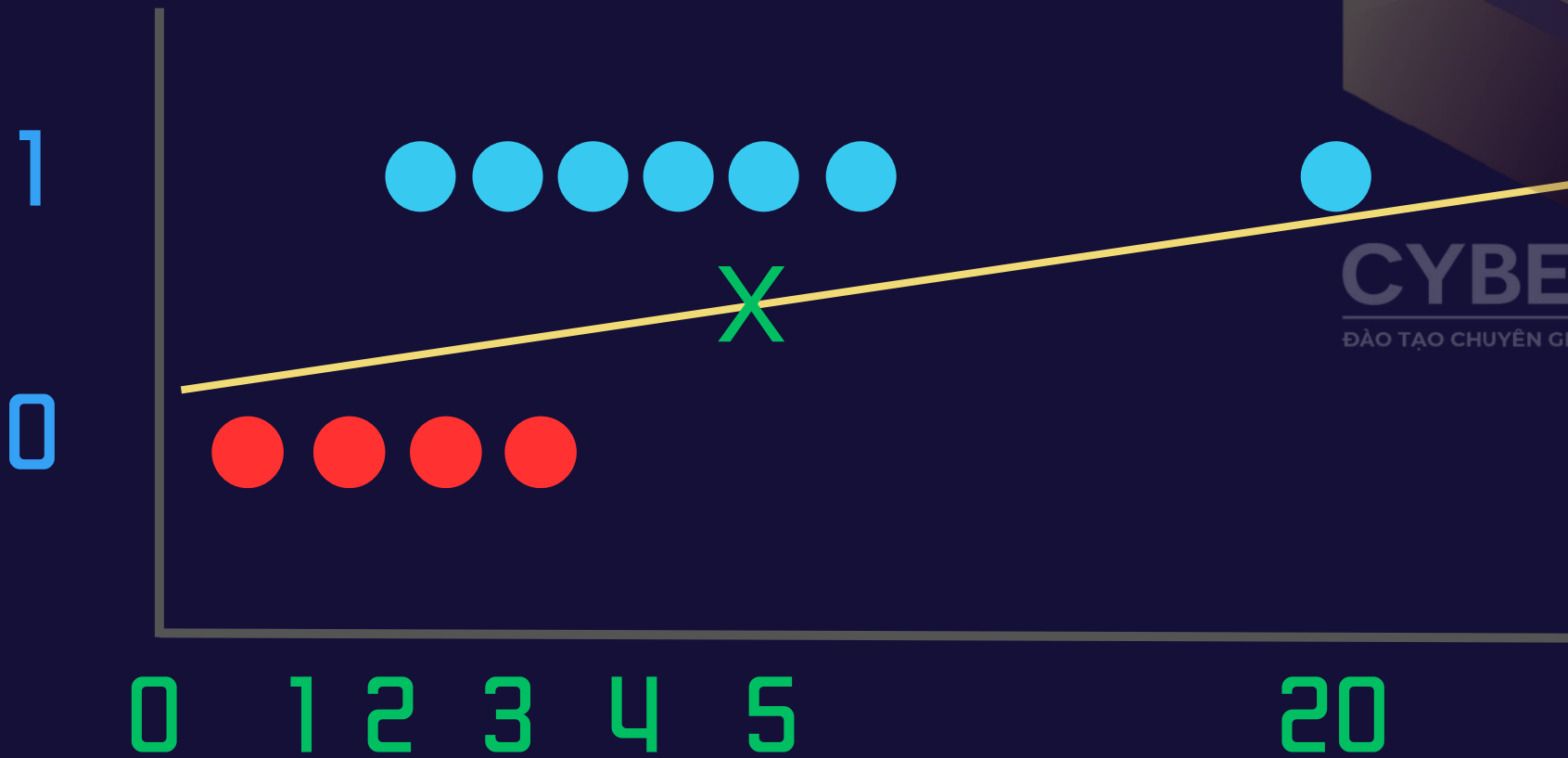
0.5	.75	1.75	4.25	3.5	2	4.75	5.5	2.75	1.25
0	0	1	1	1	0	1	1	1	0



Linear Regression

Logistic Regression

#Hours	0.5	.75	1.75	4.25	3.5	2	4.75	5.5	2.75	1.25
#Pass	0	0	1	1	1	0	1	1	1	0



Ta thấy đối với bài toán này việc dùng **Linear** sẽ không phù hợp với bài toán này

Giả sử có thêm vài bạn sinh viên tiêu biểu ôn tập đến 20 giờ và tất nhiên thi đỗ

Rất nhiều sinh viên thi đỗ cũng được dự đoán là trượt

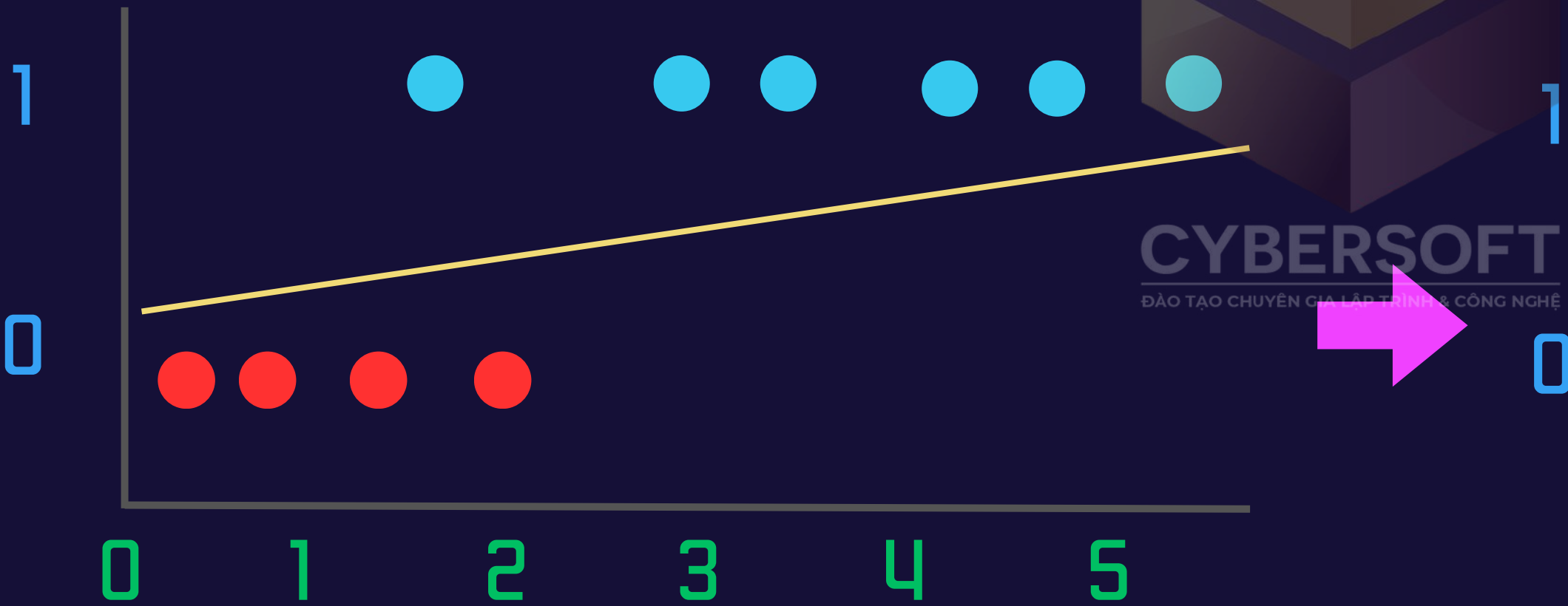
Linear Regression

Logistic Regression

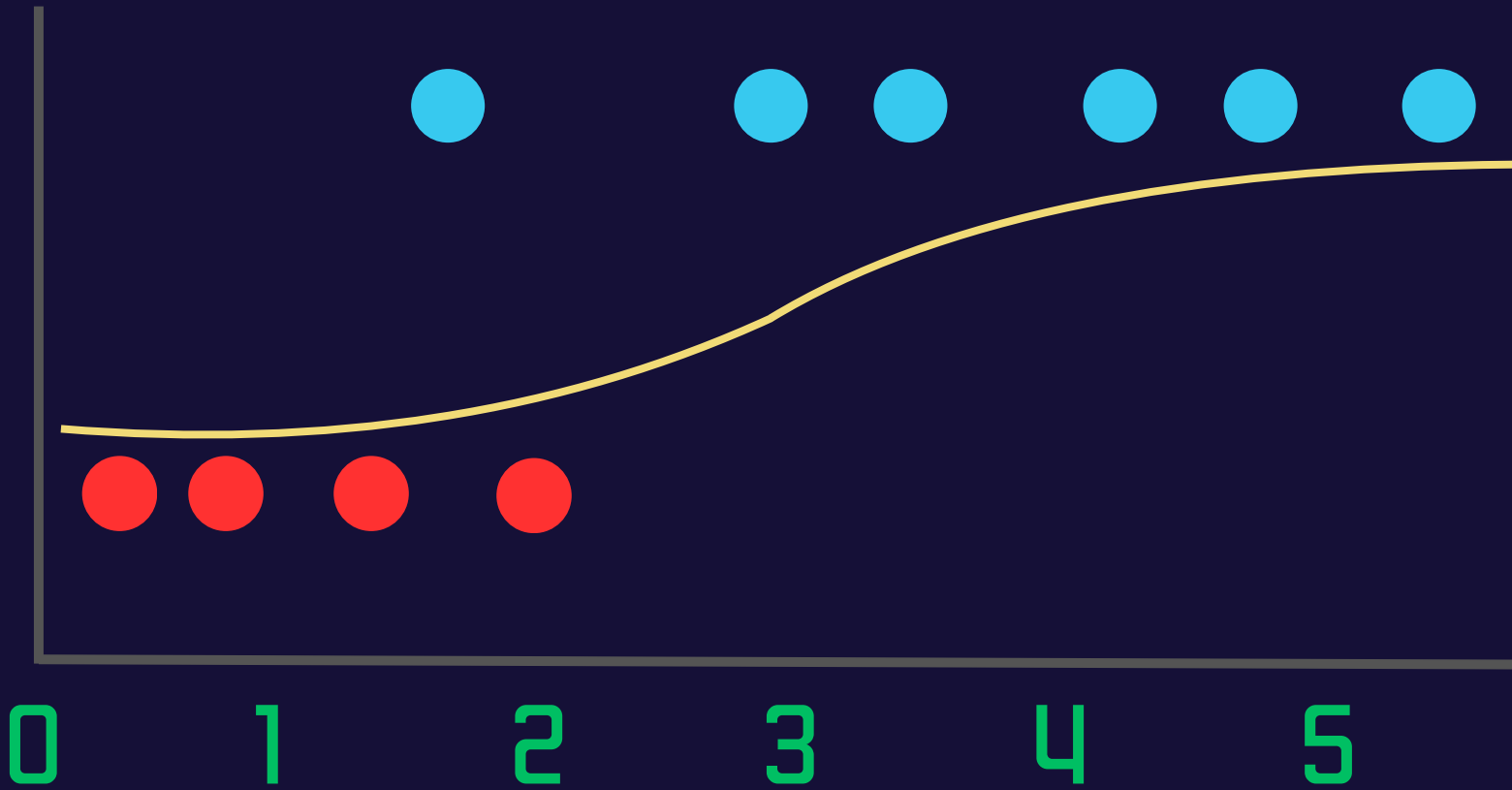
#Hours

0.5	.75	1.75	4.25	3.5	2	4.75	5.5	2.75	1.25
0	0	1	1	1	0	1	1	1	0

#Pass



Linear Regression



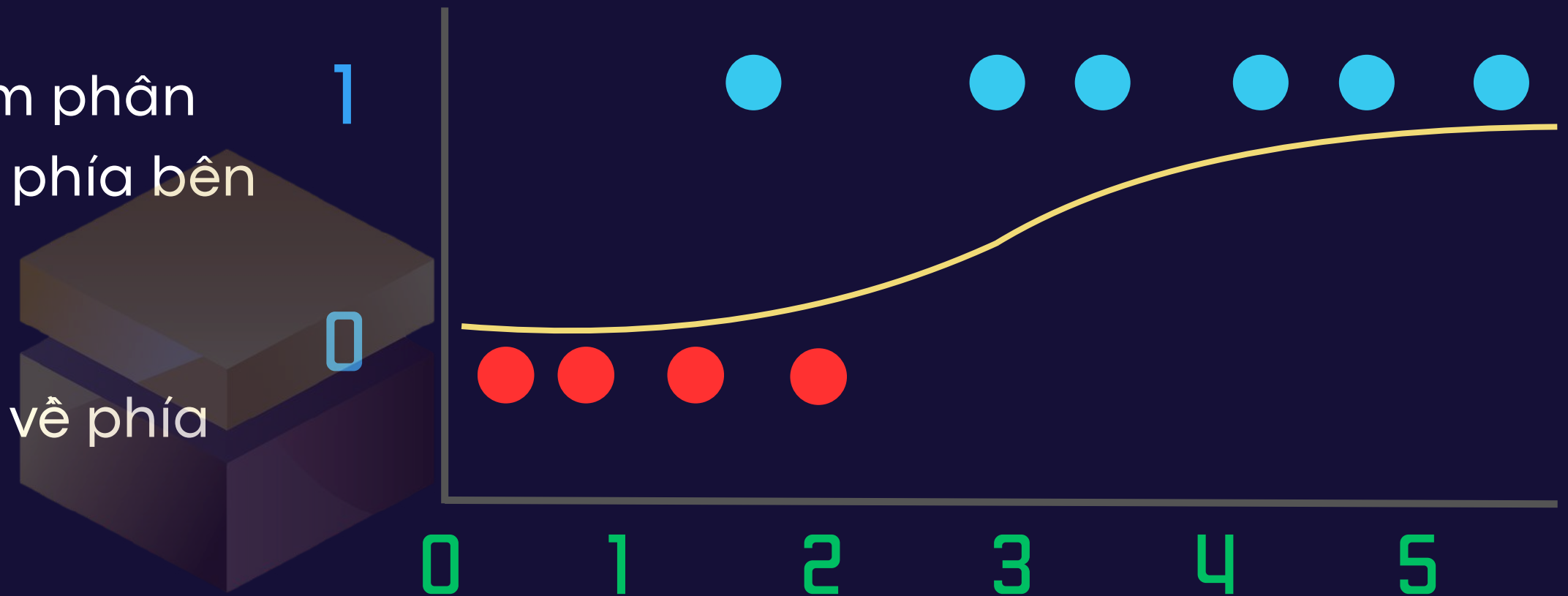
Logistic Regression

Logistic Regression

Nếu coi điểm có tung độ là $1/2$ làm điểm phân chia thì các điểm càng xa điểm này về phía bên trái có giá trị càng gần 0.

Ngược lại, các điểm càng xa điểm này về phía phải có giá trị càng gần 1.

Điều này khớp với nhận xét rằng học càng nhiều thì xác suất đồ càng cao và ngược lại.



Logistic Regression

Sigmoid

Bạn có thể tự hỏi làm thế nào một mô hình hồi quy logistic có thể đảm bảo đầu ra của nó biểu diễn một xác suất, luôn đưa ra một giá trị giữa 0 và 1.

Thực tế là có một họ các hàm được gọi là hàm logistic có đầu ra có cùng các đặc điểm đó.

Hàm logistic chuẩn, còn được gọi là hàm sigmoid (sigmoid có nghĩa là "hình chữ s"), có công thức:


$$f(x) = \frac{1}{1 + e^{-x}}$$

Biến đổi bằng hàm Sigmoid

Các phần trước ta đã có công thức biểu diễn mô hình tuyến tính như sau:

$$z = b + w_1 x_1 + w_2 x_2 + \dots + w_N x_N$$

$$\hat{y} = \sigma(z)$$

$$\sigma = \hat{y} = \frac{1}{1 + e^{-z}}$$

Từ đó ta có công thức cho mô hình Logistic Regression

Áp dụng Sigmoid

Với dữ liệu:

- $x = \text{số giờ học} = 4.9$, $y = 1$ (đậu) or 0 (rớt)
- Sau khi huấn luyện, giả sử mô hình cho ra: $Z = 4.9 - 2 = 2.9$
- Tính Z cho 4.9 giờ

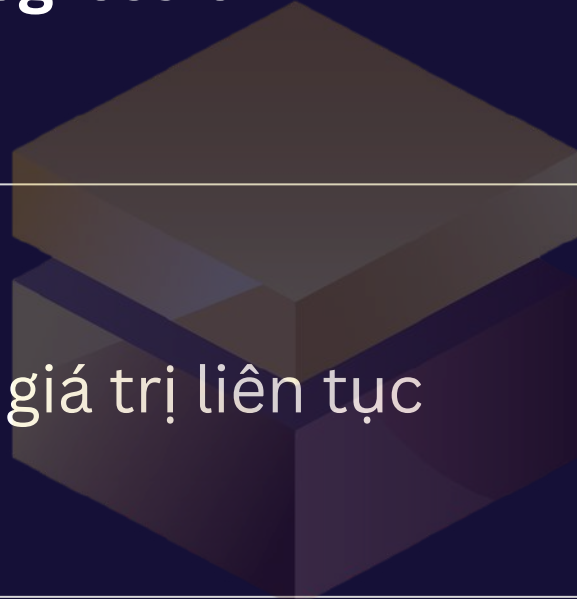
$$\hat{y} = \frac{1}{1 + e^{-2.9}} = 0.9477$$

CYBERSOFT

ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Với #Hours = 4.9, xác suất dự đoán ≈ 0.948 (tức là 94.8%) $\Rightarrow$ #Pass = 1 (vượt qua)

So sánh logistic và Linear

Đặc điểm	Linear Regression	Logistic Regression
Mục tiêu	Dự đoán giá trị liên tục	Dự đoán xác suất (phân loại nhị phân)
Công thức đầu ra	 $\hat{y} = \mathbf{w}^T \mathbf{x} + b$	$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$

Bài tập nhỏ về Logistic Regression

Một công ty muốn dự đoán khả năng một khách hàng sẽ mua sản phẩm dựa trên tuổi (#Age). Dữ liệu thu thập từ 10 khách hàng như sau:

- #Age: [20, 25, 30, 35, 40, 45, 50, 55, 60, 65]
- #Buy: [0, 0, 0, 1, 1, 1, 1, 0, 0, 0]
- (0 = Không mua, 1 = Mua)

Mô hình Logistic Regression đã được huấn luyện và cho ra công thức:

Hướng dẫn giải (Tự làm)

- Bước 1: Tính z với #Age = 48 bằng công thức $z = -10 + 0.3 \cdot \text{\#Age}$.
- Bước 2: Tính xác suất $P(y = 1)$ bằng hàm sigmoid $\frac{1}{1+e^{-z}}$ (sử dụng giá trị $e \approx 2.718$).
- Bước 3: So sánh với ngưỡng 0.5 để đưa ra dự đoán (1 nếu $P \geq 0.5$, 0 nếu $P < 0.5$).

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Dữ liệu
X = np.array([[20], [25], [30], [35], [40], [45], [50], [55], [60], [65]]) # #Age
y = np.array([0, 0, 0, 1, 1, 1, 1, 0, 0, 0]) # #Buy

# Khởi tạo và huấn luyện mô hình Logistic Regression
model = LogisticRegression(solver='lbfgs')
model.fit(X, y)

# Lấy tham số w_0 (intercept) và w_1 (coefficient)
w_0 = model.intercept_[0]
w_1 = model.coef_[0][0]
print(f"z = {w_0:.4f} + {w_1:.4f} * #Age")

# Dự đoán xác suất cho #Age = 48
age_to_predict = np.array([[48]])
prob = model.predict_proba(age_to_predict)[0][1] # Xác suất #Buy = 1
prediction = 1 if prob >= 0.5 else 0

print(f"Xác suất mua với tuổi 48: {prob:.4f}")
print(f"Dự đoán: {'Mua' if prediction == 1 else 'Không mua'}")
```

Bài tập nhỏ về Logistic Regression

Một công ty muốn dự đoán khả năng một khách hàng sẽ mua sản phẩm dựa trên tuổi (#Age). Dữ liệu thu thập từ 10 khách hàng như sau:

- #Age: [20, 25, 30, 35, 40, 45, 50, 55, 60, 65]
- #Buy: [0, 0, 0, 1, 1, 1, 1, 0, 0, 0]
- (0 = Không mua, 1 = Mua)

Mô hình Logistic Regression đã được huấn luyện và cho ra công thức:



Hướng dẫn giải (Tự làm)

- Bước 1: Tính z với #Age = 48 bằng công thức $z = -10 + 0.3 \cdot \text{\#Age}$.
- Bước 2: Tính xác suất $P(y = 1)$ bằng hàm sigmoid $\frac{1}{1+e^{-z}}$ (sử dụng giá trị $e \approx 2.718$).
- Bước 3: So sánh với ngưỡng 0.5 để đưa ra dự đoán (1 nếu $P \geq 0.5$, 0 nếu $P < 0.5$).

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Unsupervised Learning

t-SNE

Ta có một bộ dữ liệu gồm các điểm dữ liệu mô tả hành vi người dùng trên website và ta muốn xem các nhóm hành vi có thể chia ra như thế nào.



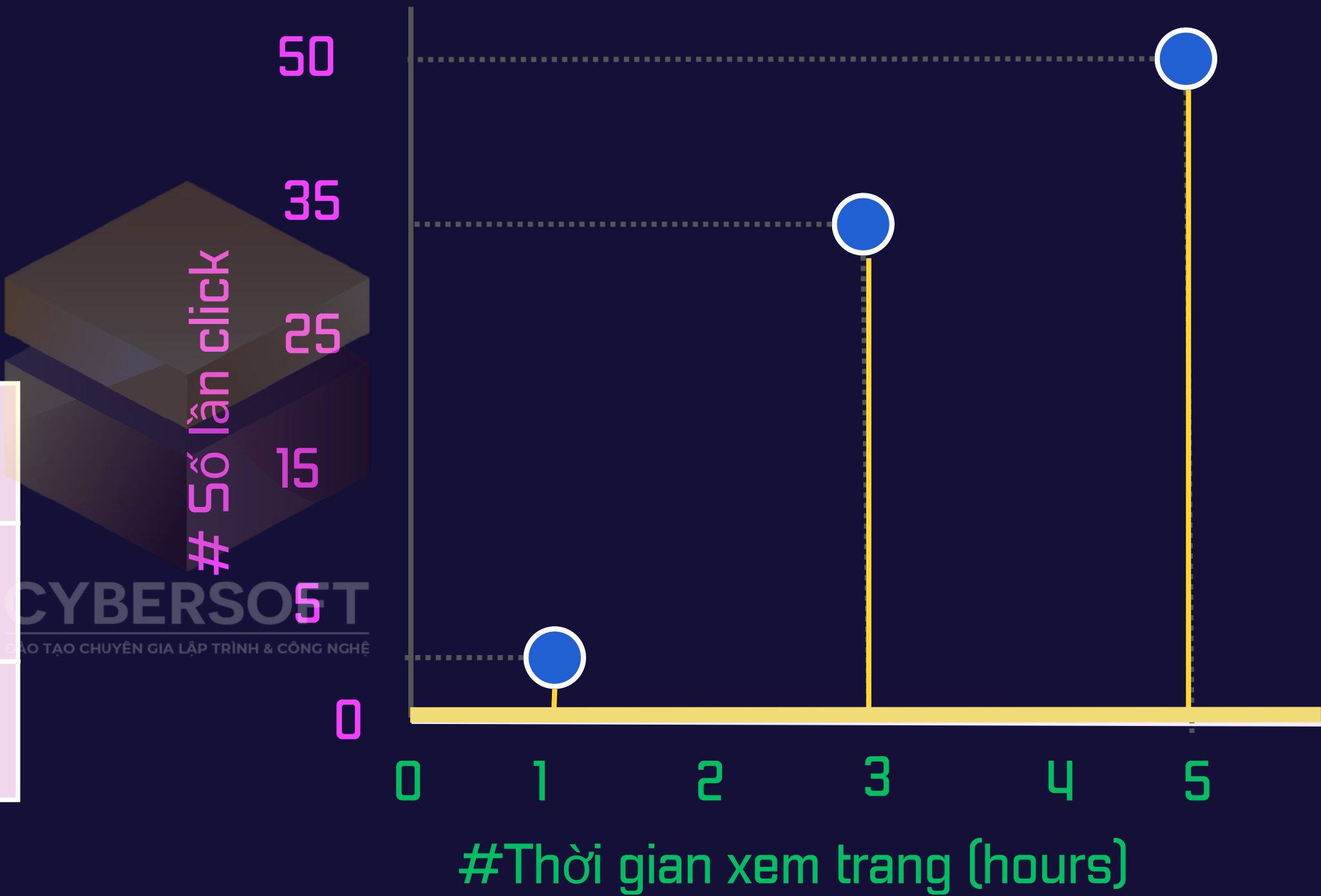
Nếu chỉ có đặc trưng mà không có nhãn, làm sao để "nhìn thấy" các cụm hành vi?

Unsupervised Learning

t-SNE

Features

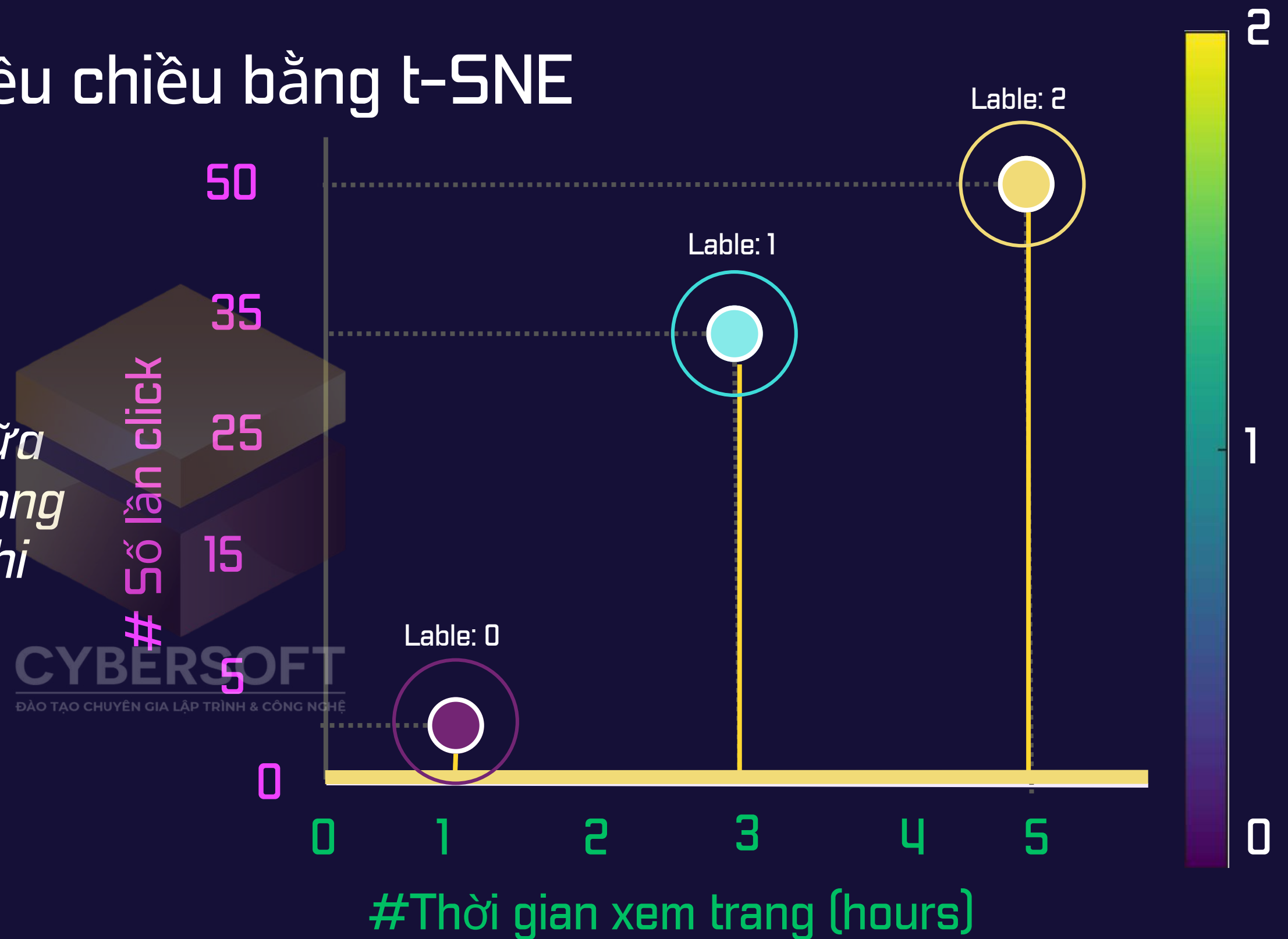
#Thời gian xem trang (hours)	#Số lần click
3	35
1	2
5	50



Unsupervised Learning

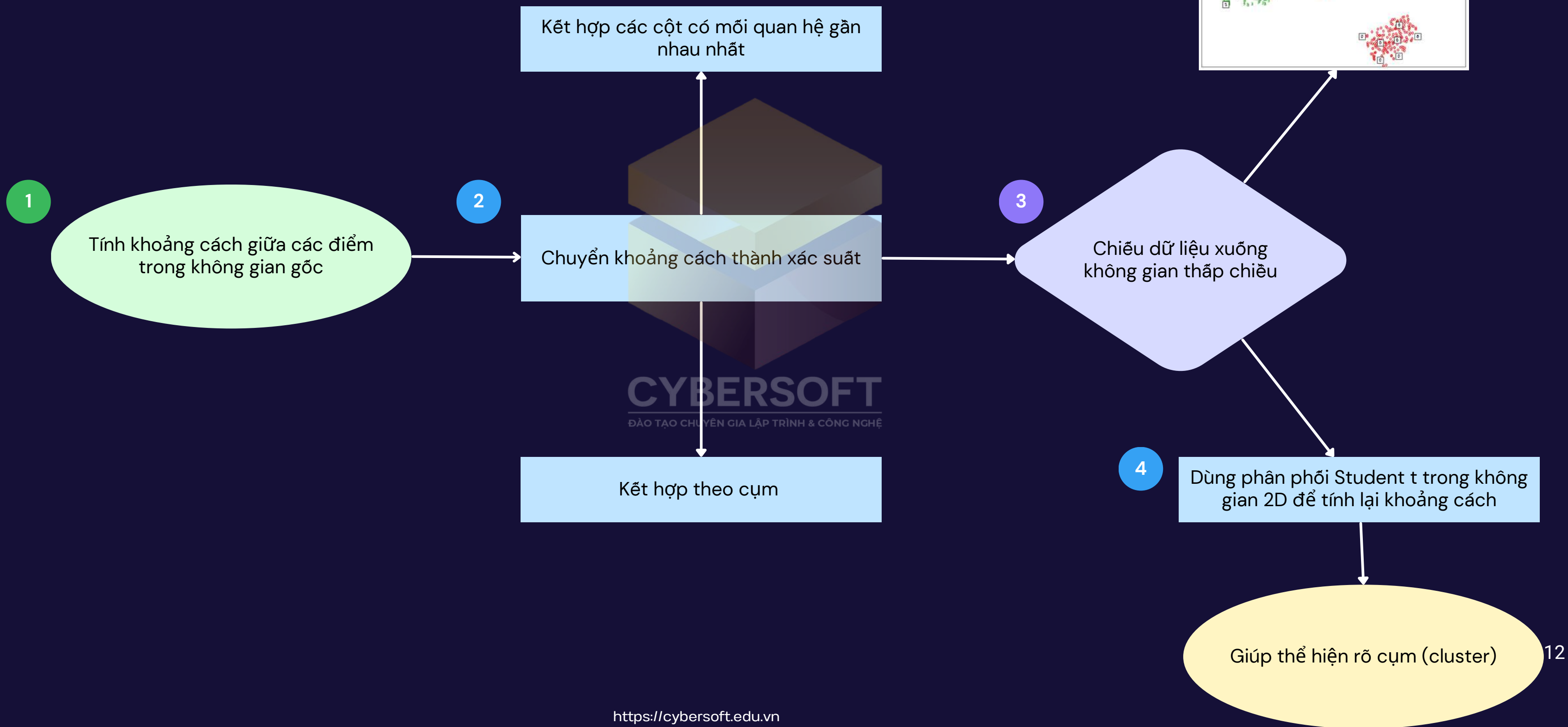
Trực quan hoá dữ liệu nhiều chiều bằng t-SNE

t-SNE bảo toàn mối quan hệ lân cận giữa các điểm dữ liệu (các điểm gần nhau trong không gian gốc sẽ vẫn gần nhau sau khi giảm chiều).



Unsupervised Learning

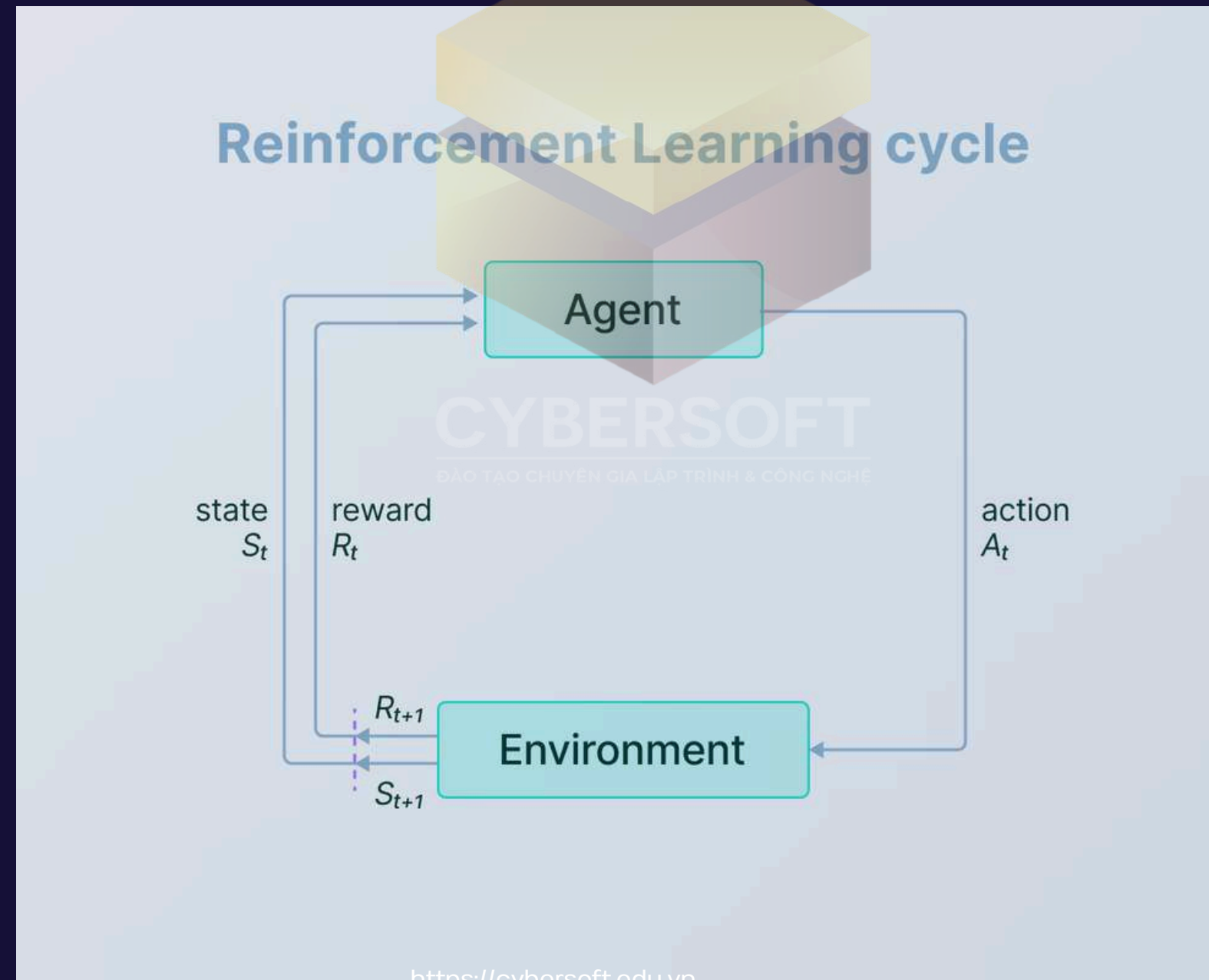
Cách t-SNE hoạt động



Reinforcement Learning

Khái niệm cốt lõi

Là một nhánh của học máy (machine learning) tập trung vào cách một tác nhân (agent) học cách tối ưu hành vi trong một môi trường để tối đa hóa phần thưởng tích lũy theo thời gian.



Reinforcement Learning

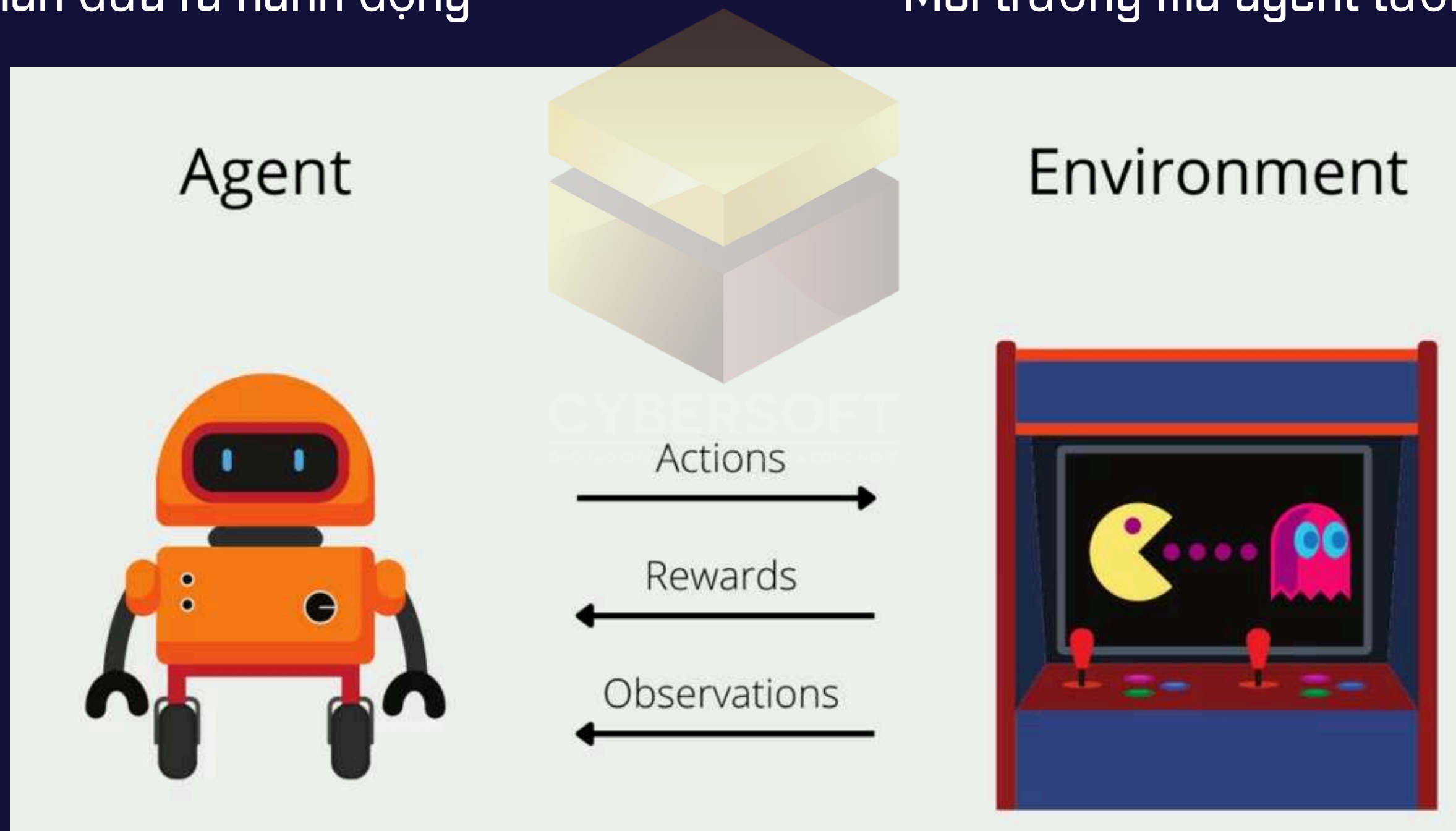
Các thành phần

Agent

Tác nhân đưa ra hành động

Environment

Môi trường mà agent tương tác



Reinforcement Learning

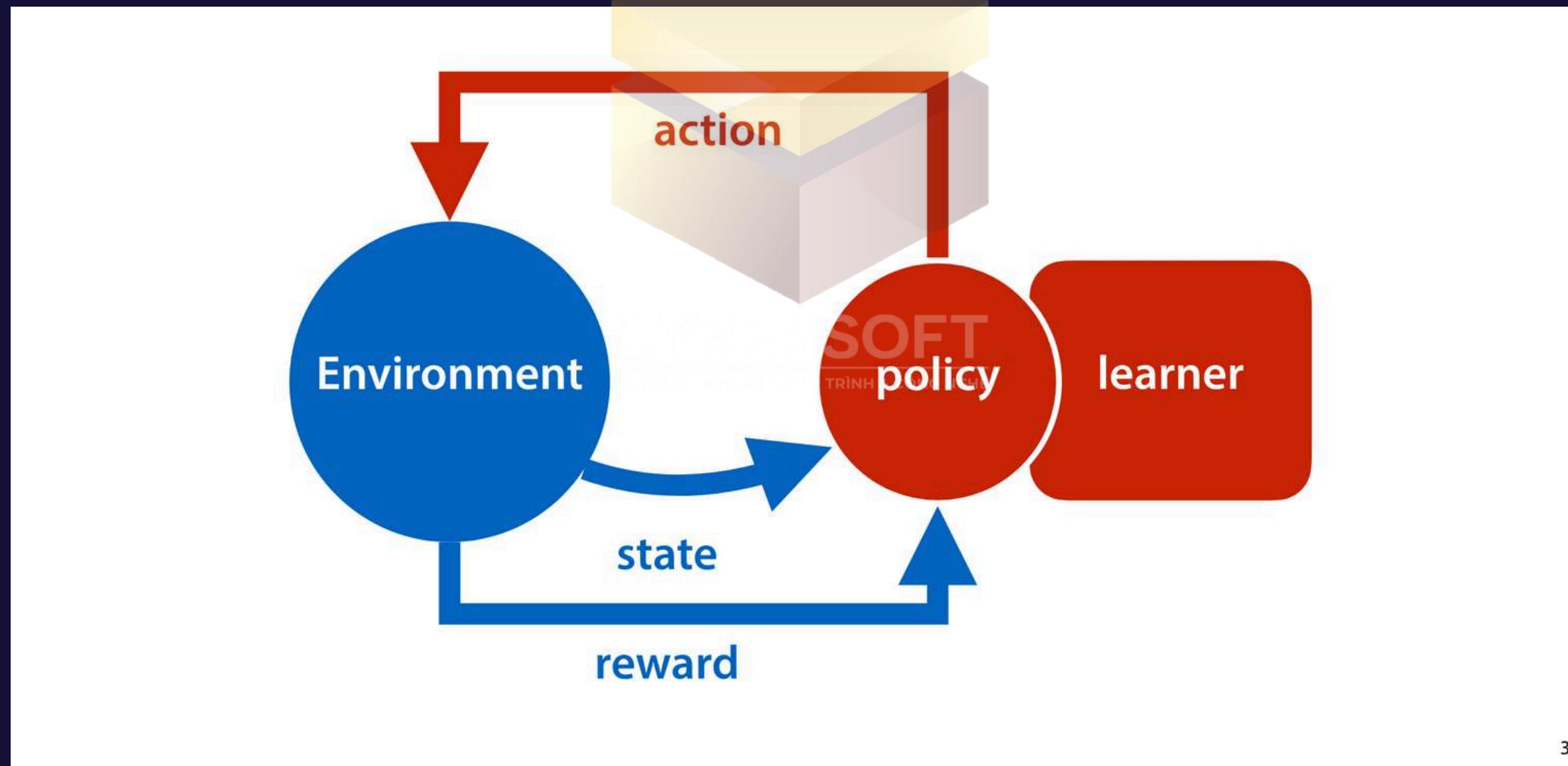
Các thành phần

State (s)

Trạng thái hiện tại của môi trường

Action (a)

Hành động mà agent có thể thực hiện



Reinforcement Learning

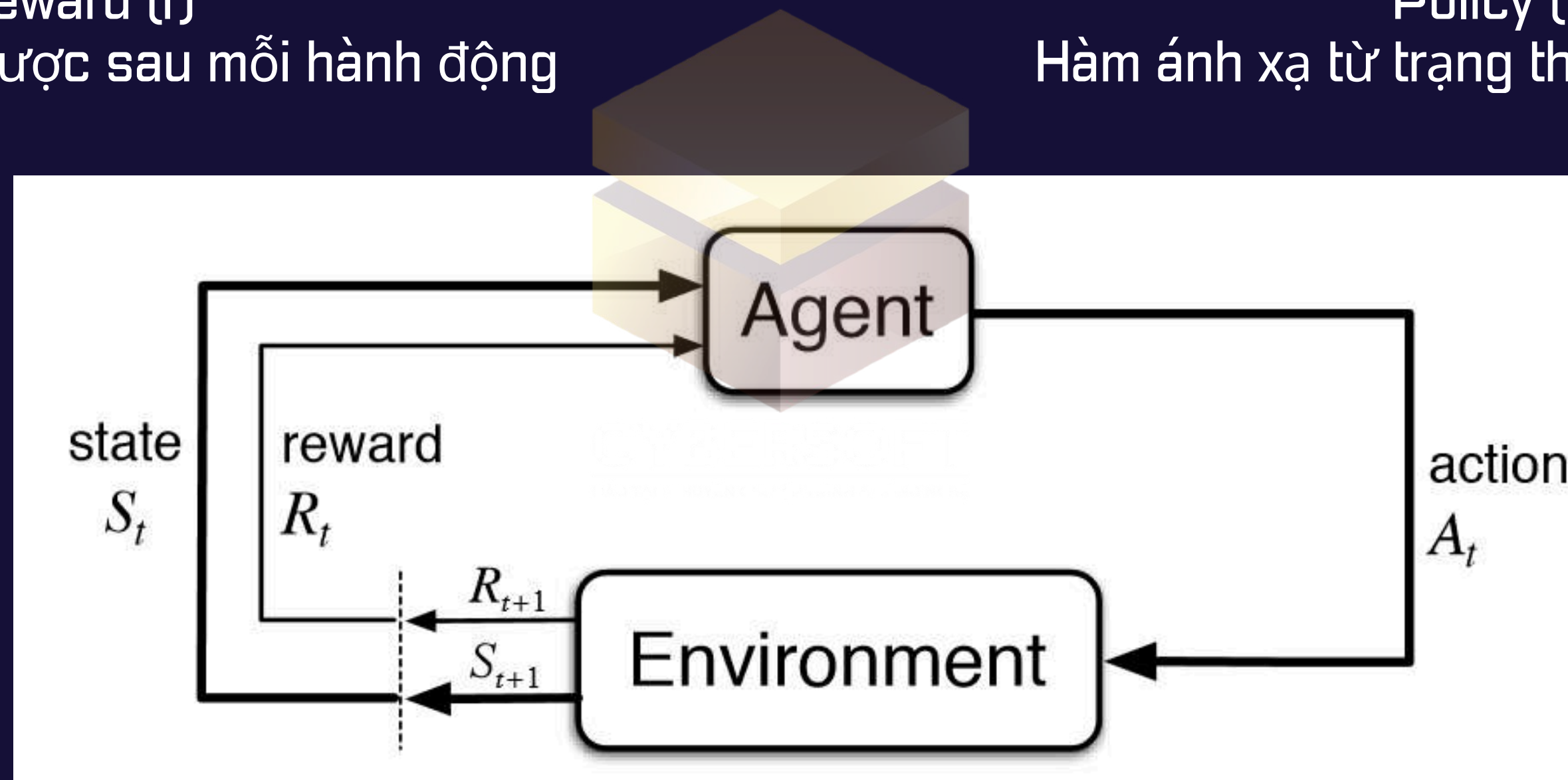
Các thành phần

Reward (r)

Giá trị số nhận được sau mỗi hành động

Policy (π)

Hàm ánh xạ từ trạng thái đến hành động



Reinforcement Learning

Các thành phần

Value function (V, Q)

Hàm đánh giá giá trị kỳ vọng của trạng thái hoặc cặp trạng thái-hành động

Action-value function

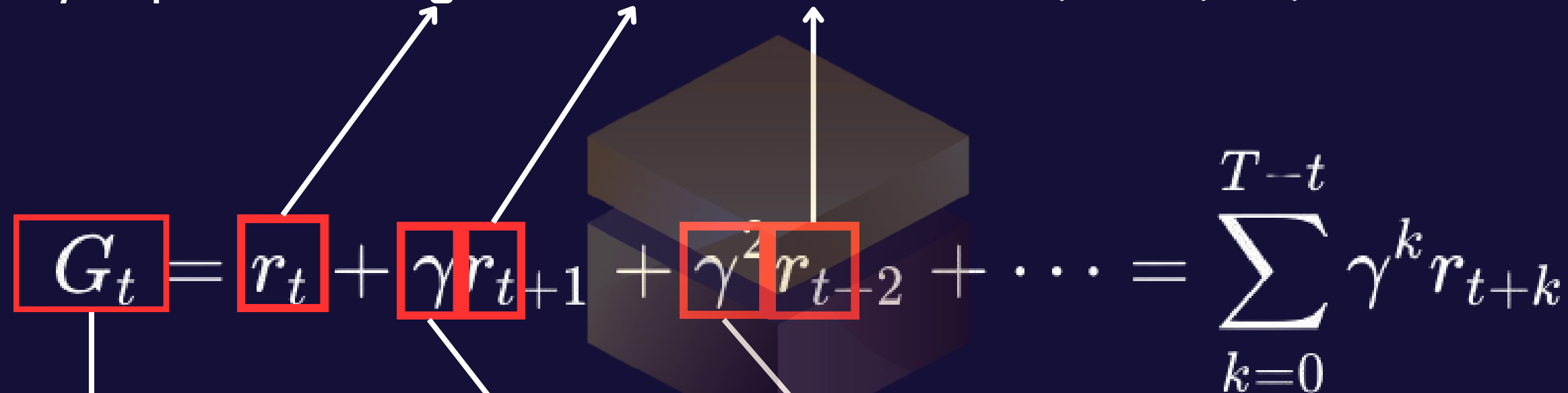
- Before we used the action-value function $Q(a)$
- Now we introduced $V_{\pi}(s)$ filling a similar role
- We can also introduce the **action-value function** $Q_{\pi}(s, a)$
- Basically this accounts for the **transition probabilities**

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} [g_t | s_t, a_t] = \mathbb{E}_{\pi} \left[\sum_{k=t+1}^T \gamma^{k-t-1} r_k | s_t, a_t \right]$$

Reinforcement Learning

Các thành phần

Đây là phần thưởng (reward) mà tác nhân nhận được tại thời điểm t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$


Đây là tổng lợi nhuận tích lũy

Hệ số chiết khấu (discount factor) điều chỉnh tầm quan trọng của các phần thưởng tương lai so với phần thưởng hiện tại.

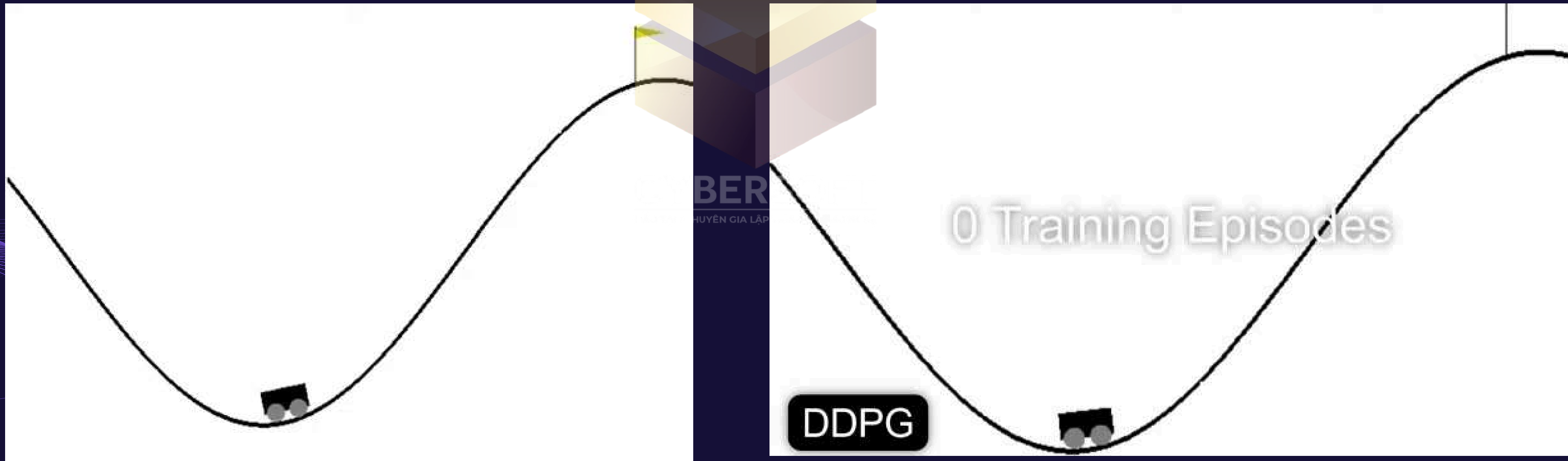
Bài tập trên lớp

Reinforcement Learning

Mọi người bắt đầu thực hành với bài tập *Mountain Car Continuous*

Kết quả thực hành:

- Học tăng cường (Reinforcement Learning) thường cần ước lượng giá trị trạng thái $V(s)$ để đánh giá chất lượng của trạng thái.



Flow dự án

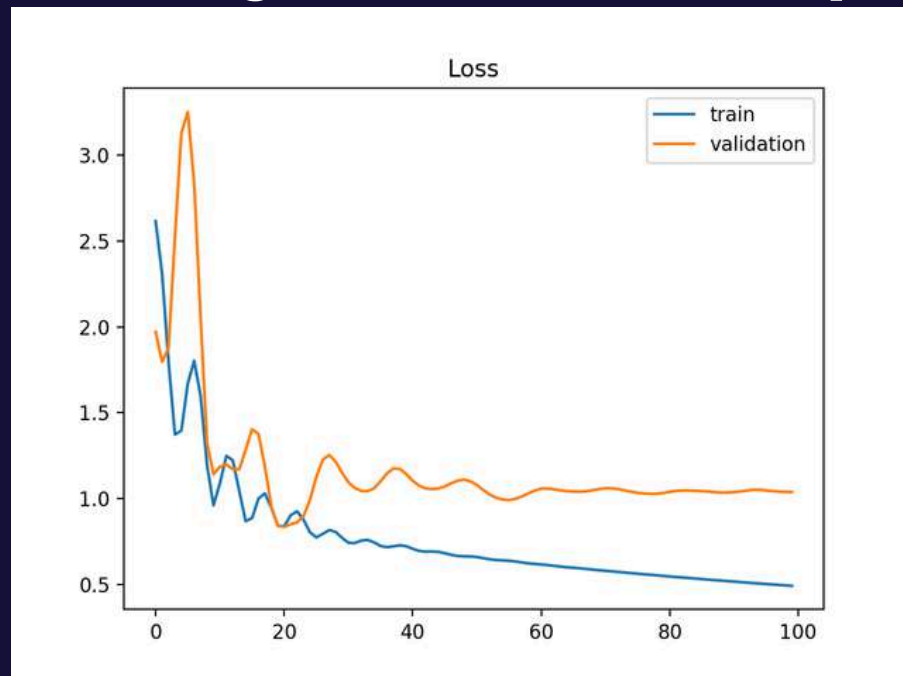
Khởi tạo môi trường & scaler



Thu thập dữ liệu để fit scaler



Learning Curve (MSE vs Episode)



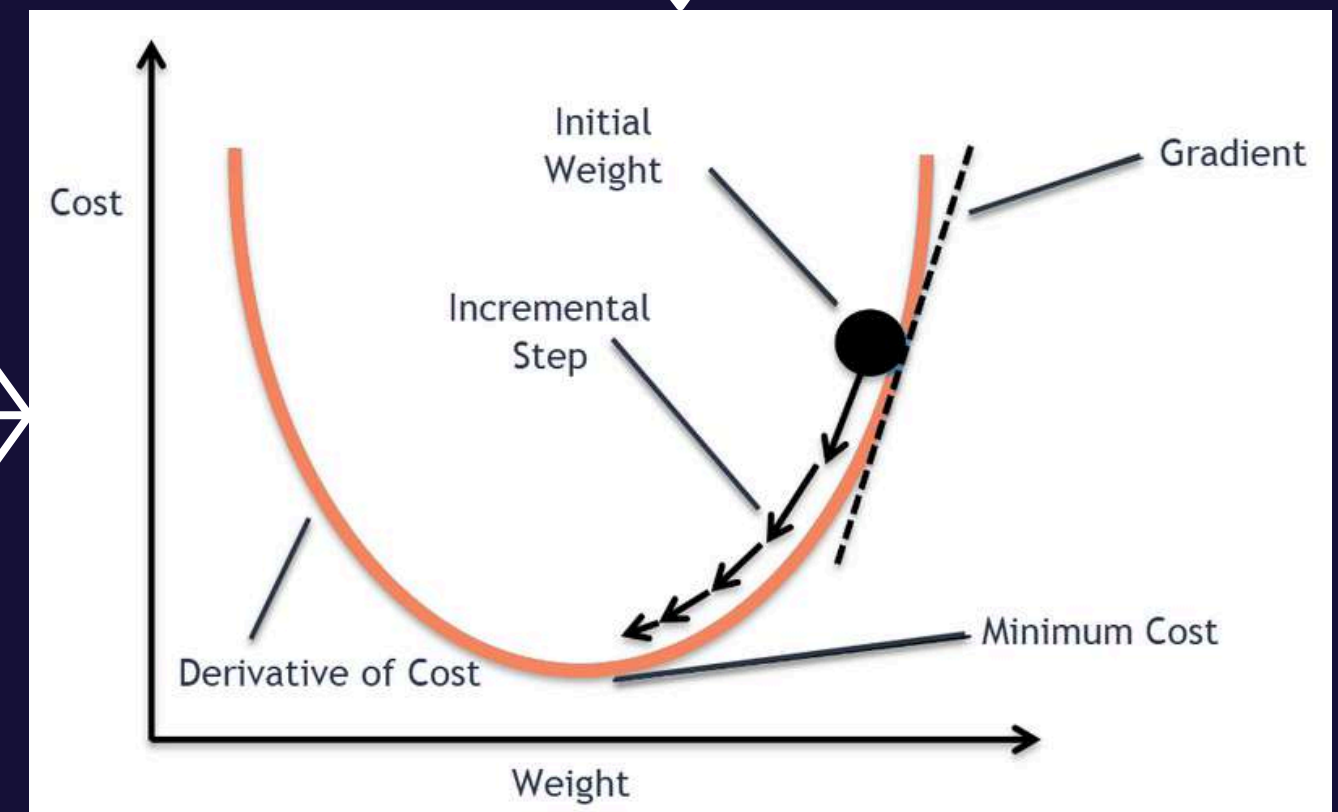
Lưu state và reward

CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Tính Monte Carlo
return G

Tính MSE giữa
 y_{pred} và G

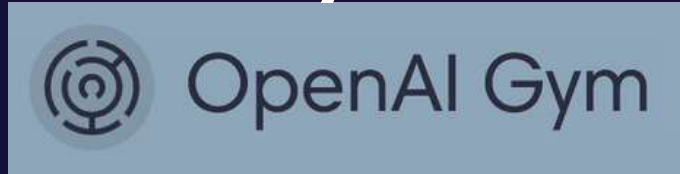
<https://cybersoft.edu.vn>



Khởi tạo Linear Regression

Thư viện áp dụng

Gym



matplotlib



Numpy



Scikit learn



CYBERSOFT
ĐÀO TẠO CHUYÊN GIA LẬP TRÌNH & CÔNG NGHỆ

Bài tập trên lớp: Mountain Car Continuous

1. Cài đặt thư viện

```
pip install gym matplotlib scikit-learn numpy==1.24.3
```

Bài tập trên lớp: Mountain Car Continuous

1.Import thư viện

Khởi tạo môi trường

```
# 1. Khởi tạo môi trường  
env = gym.make('MountainCarContinuous-v0')  
scaler = StandardScaler()
```

```
n_episodes = 200  
gamma = 0.99
```

Các tham số học tăng cường

Tham số gamma này quyết định tầm quan trọng của các phần thưởng trong tương lai so với phần thưởng hiện tại

Thu thập dữ liệu trạng thái

Khởi tạo một danh sách rỗng

```
states = []  
for _ in range(10):  
    state = env.reset()  
    done = False  
    while not done:  
        action = env.action_space.sample()  
        next_state, reward, done, _ = env.step(action)  
        states.append(state)  
        state = next_state  
scaler.fit(states)
```

Trong mỗi lần lặp, nó mô phỏng một lần chơi hoặc một "episode" trong môi trường học tăng cường

Trong mỗi episode, nó cho tác nhân thực hiện các hành động ngẫu nhiên và ghi lại các trạng thái mà tác nhân gặp phải vào danh sách states

Khởi tạo Linear Regression

SGDRegressor là một lớp thực hiện hồi quy tuyến tính bằng cách sử dụng thuật toán Stochastic Gradient Descent (SGD)

```
# 3 Khởi tạo Linear Regression
model = SGDRegressor(max_iter=1, learning_rate="constant", eta0=0.01, penalty=None)
model.partial_fit([scaler.transform([states[0]])[0]], [0.0]) # Dummy init
```

Thực hiện một bước "khởi tạo giả" cho mô hình bằng cách cho nó thấy một ví dụ trạng thái đầu tiên (đã được chuẩn hóa) với một giá trị mục tiêu bằng 0

Hàm ước lượng hàm giá trị

Khởi tạo danh sách rỗng để lưu trữ phần thưởng cho mỗi Episode

```
for episode in range(n_episodes):  
    state = env.reset()  
    episode_states = []  
    episode_rewards = []  
    done = False
```

```
    while not done:  
        action = env.action_space.sample()  
        next_state, reward, done, _ = env.step(action)  
        episode_states.append(state)  
        episode_rewards.append(reward)  
        state = next_state
```

Vòng lặp tương tác với môi trường (Collecting Episode Data)

Hàm ước lượng hàm giá trị

Tính toán "return" (hay "tổng phần thưởng chiết khấu") cho mỗi bước thời gian trong episode.

```
# Monte Carlo return
G = 0
returns = []
for r in reversed(episode_rewards):
    G = r + gamma * G
    returns.insert(0, G)
```

```
X = scaler.transform(episode_states)
y = np.array(returns)
model.partial_fit(X, y)

y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
mse_list.append(mse)
```

Cập nhật mô hình Linear Regression

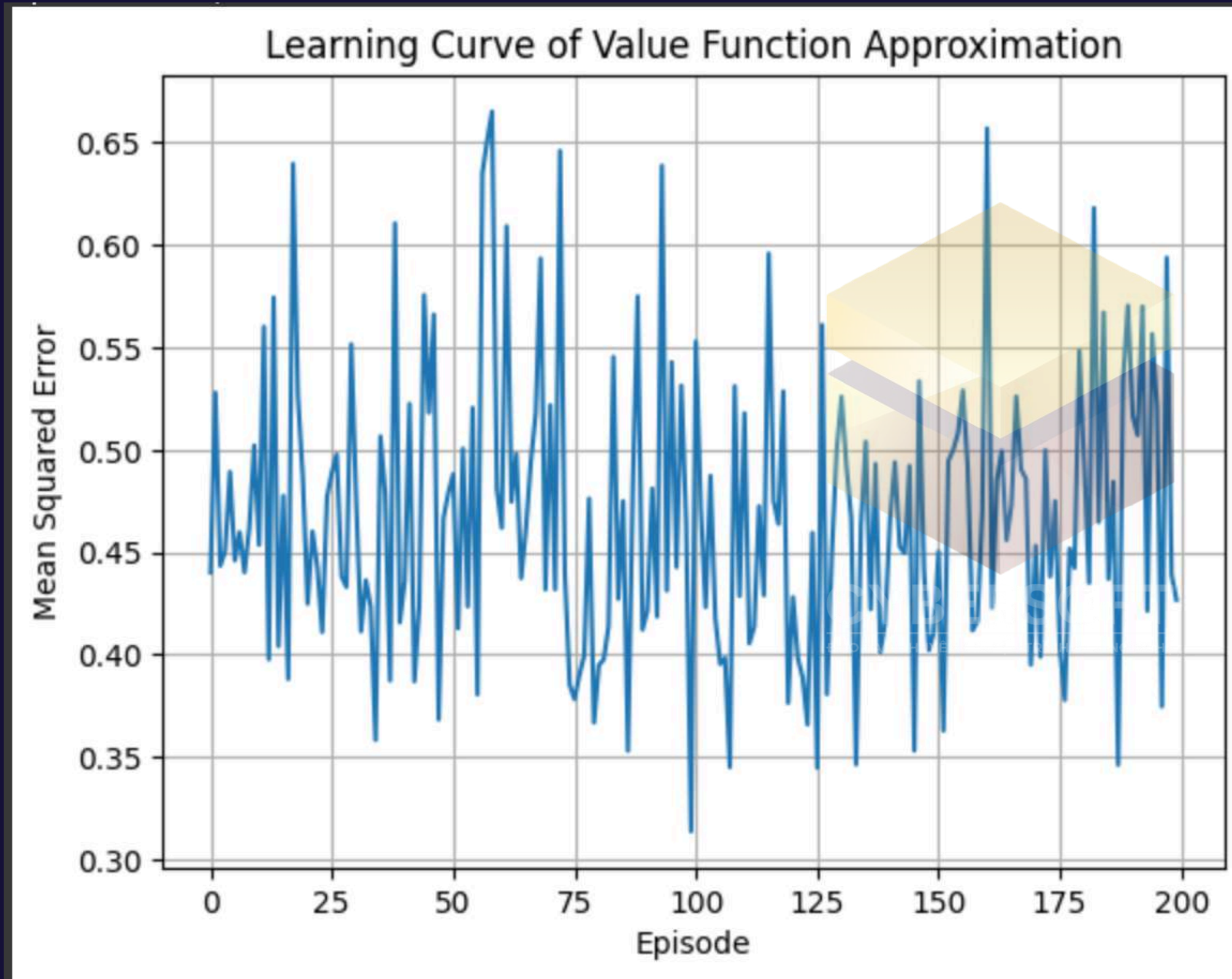
Kết quả đạt được

```
Episode 10, MSE: 0.5021
Episode 20, MSE: 0.4868
Episode 30, MSE: 0.5516
Episode 40, MSE: 0.4156
Episode 50, MSE: 0.4790
Episode 60, MSE: 0.4808
Episode 70, MSE: 0.4317
Episode 80, MSE: 0.3667
Episode 90, MSE: 0.4121
Episode 100, MSE: 0.3136
Episode 110, MSE: 0.4285
Episode 120, MSE: 0.3763
Episode 130, MSE: 0.5000
Episode 140, MSE: 0.4160
Episode 150, MSE: 0.4106
Episode 160, MSE: 0.4695
Episode 170, MSE: 0.3950
Episode 180, MSE: 0.5483
Episode 190, MSE: 0.5704
Episode 200, MSE: 0.4267
```



Output là giá trị lỗi trung bình qua từng epochs

Kết quả đạt được



Mô hình không đứng yên, mà liên tục điều chỉnh và học hỏi qua từng episode. Các dao động của MSE cho thấy nó đang tích cực phản ứng với dữ liệu mới từ môi trường.